VLAAMS SUPERCOMPUTER CENTRUM | *Innovative Computing for A Smarter Flanders*

HPC Tutorial

Last updated: August 26 2021

For Windows Users

Authors:

Franky Backeljauw[5], Stefan Becuwe[5], Geert Jan Bex[3], Geert Borstlap[5], Jasper Devreker[2], Stijn De Weirdt[2], Andy Georges[2], Balázs Hajgató[1,2], Kenneth Hoste[2], Kurt Lust[5], Samuel Moors[1], Ward Poelmans[1], Mag Selwa[4], Álvaro Simón García[2], Bert Tijskens[5], Jens Timmerman[2], Kenneth Waegeman[2], Toon Willems[2]

Acknowledgement: VSCentrum.be

[1]Free University of Brussels
[2]Ghent University
[3]Hasselt University
[4]KU Leuven
[5]University of Antwerp

Audience:

This HPC Tutorial is designed for **researchers** at the **University of Antwerp** and affiliated institutes who are in need of computational power (computer resources) and wish to explore and use the High Performance Computing (HPC) core facilities of the Flemish Supercomputing Centre (VSC) to execute their computationally intensive tasks.

The audience may be completely unaware of the UAntwerpen-HPC concepts but must have some basic understanding of computers and computer programming.

Contents:

This **Beginners Part** of this tutorial gives answers to the typical questions that a new UAntwerpen-HPC user has. The aim is to learn how to make use of the HPC.

Beginners Part		
Questions	**chapter**	**title**
What is a UAntwerpen-HPC exactly? Can it solve my computational needs?	**1**	Introduction to HPC
How to get an account?	**2**	Getting an HPC Account
How do I connect to the UAntwerpen-HPC and transfer my files and programs?	**3**	Connecting to the HPC infrastructure
How to start background jobs?	**4**	Running batch jobs
How to start jobs with user interaction?	**5**	Running interactive jobs
Where do the input and output go? Where to collect my results?	**6**	Running jobs with input/output data
Can I speed up my program by exploring parallel programming techniques?	**7**	Multi core jobs/Parallel Computing
Troubleshooting	**8**	Troubleshooting
What are the rules and priorities of jobs?	**9**	HPC Policies
FAQ	**10**	Frequently Asked Questions

The **Advanced Part** focuses on in-depth issues. The aim is to assist the end-users in running their own software on the UAntwerpen-HPC.

Advanced Part		
Questions	**chapter**	**title**
What are the optimal Job Specifications?	**11**	Fine-tuning Job Specifications
Can I start many jobs at once?	**12**	Multi-job submission
Can I compile my software on the UAntwerpen-HPC?	**13**	Compiling and testing your software on the HPC
Do you have more program examples for me?	**14**	Program examples
Do you have more job script examples for me?	**15**	Job script examples
Any more advice?	**16**	Best Practices

The **Annexes** contains some useful reference guides.

Annex	
Title	**chapter**
HPC Quick Reference Guide	**A**
TORQUE options	**B**
Useful Linux Commands	**C**

Notification:

In this tutorial specific commands are separated from the accompanying text:

```
$ commands
```

These should be entered by the reader at a command line in a terminal on the UAntwerpen-HPC. They appear in all exercises preceded by a $ and printed in **bold**. You'll find those actions in a grey frame.

Button are menus, buttons or drop down boxes to be pressed or selected.

"Directory" is the notation for directories (called "folders" in Windows terminology) or specific files. (e.g., "/user/antwerpen/201/vsc20167")

"Text" Is the notation for text to be entered.

Tip: A "Tip" paragraph is used for remarks or tips.

More support:

Before starting the course, the example programs and configuration files used in this Tutorial must be copied to your home directory, so that you can work with your personal copy. If you have received a new VSC-account, all examples are present in an "/apps/antwerpen/tutorials/Intro-HPC/examples" directory.

```
$ cp -r /apps/antwerpen/tutorials/Intro-HPC/examples ~/
$ cd
$ ls
```

They can also be downloaded from the VSC website at https://www.vscentrum.be. Apart from this UAntwerpen-HPC Tutorial, the documentation on the VSC website will serve as a reference for all the operations.

Tip: The users are advised to get self-organised. There are only limited resources available at the UAntwerpen-HPC staff, which are best effort based. The UAntwerpen-HPC staff cannot give support for code fixing. The user applications and own developed software remain solely the responsibility of the end-user.

More documentation can be found at:

1. VSC documentation: https://www.vscentrum.be/user-portal
2. CalcUA Core Facility web pages: https://www.uantwerpen.be/hpc
3. External documentation (TORQUE, Moab): http://docs.adaptivecomputing.com

This tutorial is intended for users who want to connect and work on the HPC of the **University of Antwerp**.

This tutorial is available in a Windows, Mac or Linux version.

This tutorial is available for UAntwerpen, UGent, KU Leuven, UHasselt and VUB users.

Request your appropriate version at hpc@uantwerpen.be.

Contact Information:

We welcome your feedback, comments and suggestions for improving the UAntwerpen-HPC Tutorial (contact: hpc@uantwerpen.be).

For all technical questions, please contact the UAntwerpen-HPC staff:

1. By e-mail: hpc@uantwerpen.be
2. By phone: 03/2653860 (Stefan), 03/2653855 (Franky), 03/2653879 (Bert), 03/2653852 (Kurt), 03/2658980 (Carl)
3. In real: Campus Middelheim, Building G, Rooms G.309, G.310 and G.311

Mailing-lists:

1. Announcements: calcua-announce@supersympa.ua.ac.be (for official announcements and communications)

New users are automatically added to the mailing list. You can check the archives on https://sympa.ua.ac.be/sympa/, but you will need to generate a password first on that site using your main e-mail address (on which you receive the mailing list).

Glossary

NFS Network File System, a protocol for sharing file systems across a network, often used on Unix(-like) operating systems.

NIC Network Interface Controller, a (virtualized) hardware component that connects a computer to a network.

REST REpresentational State Transfer is a software architectural style that defines a set of constraints to be used for creating web services.

YAML A human-readable text-based data serialization format.

cluster A group of compute nodes.

compute node The computational units on which batch or interactive jobs are processed. A compute node is pretty much comparable to a single personal computer. It contains one or more sockets, each holding a single CPU. Some nodes also contain one or more GPGPUs. The compute node is equipped with memory (RAM) that is accessible by all its CPUs.

core An individual compute unit inside a CPU. A CPU typically contains one or more cores.

CPU A central processing unit. A CPU is a consumable resource. A compute node typically contains one or more CPUs.

distributed memory system Computing system consisting of many compute nodes connected by a network, each with their own memory. Accessing memory on a neighbouring node is possible but requires explicit communication.

FLOPS FLOPS is short for "Floating-point Operations Per second", i.e., the number of (floating-point) computations that a processor can perform per second.

FTP File Transfer Protocol, used to copy files between distinct machines (over a network.) FTP is unencrypted, and as such blocked on certain systems. SFTP or SCP are secure alternatives to FTP.

GPGPU A general purpose graphical processing unit. A GPGPU is a consumable resource. A GPGPU is a GPU that is used for highly parallel general purpose calculations. A compute node may contain zero, one or more GPGPUs.

grid A group of clusters.

Heat Heat is the OpenStack orchestration service, which can manage multiple composite cloud applications using templates, through both an OpenStack-native REST API and a CloudFormation-compatible Query API.

Heat Orchestration Template A Heat Orchestration Template (HOT) is a text file which describes the infrastructure for a cloud application. Because HOT files are text files in a YAML-based format, they are readable and writable by humans, and can be managed using a version control system. HOT is one of the template formats supported by Heat, along with the older CloudFormation-compatible CFN format.

Horizon Horizon is the name of the OpenStack Dashboard.

HPC High Performance Computing, high performance computing and multiple-task computing on a supercomputer. The UAntwerpen-HPC is the HPC infrastructure at the University of Antwerp.

InfiniBand A high speed switched fabric computer network communications link used in UAntwerpen-HPC.

job constraints A set of conditions that must be fulfilled for the job to start.

L1d Level 1 data cache, often called **primary cache**, is a static memory integrated with the CPU core that is used to store data recently accessed by a CPU and also data which may be required by the next operations.

L2d Level 2 data cache, also called **secondary cache**, is a memory that is used to store recently accessed data and also data, which may be required for the next operations. The goal of having the level 2 cache is to reduce data access time in cases when the same data was already accessed before..

L3d Level 3 data cache. Extra cache level built into motherboards between the microprocessor and the main memory.

LAN Local Area Network.

Linux An operating system, similar to UNIX.

LLC The Last Level Cache is the last level in the memory hierarchy before main memory. Any memory requests missing here must be serviced by local or remote DRAM, with significant increase in latency when compared with data serviced by the cache memory.

login node On UAntwerpen-HPC clusters, login nodes serve multiple functions. From a login node you can submit and monitor batch jobs, analyse computational results, run editors, plots, debuggers, compilers, do housekeeping chores as adjust shell settings, copy files and in general manage your account. You connect to these servers when want to start working on the UAntwerpen-HPC.

memory A quantity of physical memory (RAM). Memory is provided by compute nodes. It is required as a constraint or consumed as a consumable resource by jobs. Within Moab, memory is tracked and reported in megabytes (MB).

metrics A measure of some property, activity or performance of a computer sub-system. These metrics are visualised by graphs in, e.g., Ganglia.

Moab Moab is a job scheduler, which allocates resources for jobs that are requesting resources.

modules UAntwerpen-HPC uses an open source software package called "Environment Modules" (Modules for short) which allows you to add various path definitions to your shell environment.

MPI MPI stands for Message-Passing Interface. It supports a parallel programming method designed for distributed memory systems, but can also be used well on shared memory systems.

node See compute node.

node attribute A node attribute is a non-quantitative aspect of a node. Attributes typically describe the node itself or possibly aspects of various node resources such as processors or memory. While it is probably not optimal to aggregate node and resource attributes together in this manner, it is common practice. Common node attributes include processor architecture, operating system, and processor speed. Jobs often specify that resources be allocated from nodes possessing certain node attributes.

OpenStack OpenStack (https://www.openstack.org) is a free and open-source software platform for cloud computing, mostly deployed as infrastructure-as-a-service (IaaS), whereby virtual servers and other resources are made available to customers.

OpenStack Dashboard OpenStack Dashboard (Horizon) provides administrators and users with a graphical interface to access, provision, and automate deployment of cloud-based resources. The design accommodates third party products and services, such as billing, monitoring, and additional management tools. The dashboard is also brand-able for service providers and other commercial vendors who want to make use of it. The dashboard is one of several ways users can interact with OpenStack resources. Developers can automate access or build tools to manage resources using the native OpenStack API or the EC2 compatibility API..

OpenStack Identity OpenStack Identity (Keystone) provides a central directory of users mapped to the OpenStack services they can access. It acts as a common authentication system across the cloud operating system and can integrate with existing backend directory services. It supports multiple forms of authentication including standard username/password credentials and token-based systems. In the VSC cloud, it is integrated with the VSC account system..

OpenStack Image OpenStack Image (Glance) provides discovery, registration, and delivery services for disk and server images. Stored images can be used as a template. It can also be used to store and catalog an unlimited number of backups. The Image Service can store disk and server images in a variety of back-ends, including Swift..

OpenStack Instance OpenStack Instances are virtual machines, which are instances of a system image that is created upon request and which is configured when launched. With traditional virtualization technology, the state of the virtual machine is persistent, whereas OpenStack supports both persistent and ephemeral image creation.

OpenStack Volume OpenStack Volume is a detachable block storage device. Each volume can be attached to only one instance at a time.

PBS, TORQUE or **OpenPBS** are Open Source resource managers, which are responsible for collecting status and health information from compute nodes and keeping track of jobs running in the system. It is also responsible for spawning the actual executable that is associated with a job, e.g., running the executable on the corresponding compute node. Client commands for submitting and managing jobs can be installed on any host, but in general are installed and used from the Login nodes.

processor A processing unit. A processor is a consumable resource. A processor can be a CPU or a (GP)GPU.

queue PBS/TORQUE queues, or "classes" as Moab refers to them, represent groups of computing resources with specific parameters. A queue with a 12 hour runtime or "walltime" would allow jobs requesting 12 hours or less to use this queue.

scp Secure Copy is a protocol to copy files between distinct machines. SCP or scp is used extensively on UAntwerpen-HPC clusters to stage in data from outside resources.

scratch Supercomputers generally have what is called scratch space: storage available for temporary use. Use the scratch filesystem when, for example you are downloading and uncompressing applications, reading and writing input/output data during a batch job, or when you work with large datasets. Scratch is generally a lot faster then the Data or Home filesystem.

sftp Secure File Transfer Protocol, used to copy files between distinct machines.

share A share is a remote, mountable file system. Users can mount and access a share on several hosts at a time.

shared memory system Computing system in which all of the CPUs share one global memory space. However, access times from a CPU to different regions of memory are not necessarily uniform. This is called NUMA: Non-uniform memory access. Memory that is closer to the CPU your process is running on will generally be faster to access than memory that is closer to a different CPU. You can pin processes to a certain CPU to ensure they always use the same memory.

SSD Solid-State Drive. This is a kind of storage device that is faster than a traditional hard disk drive.

SSH Secure Shell (SSH), sometimes known as Secure Socket Shell, is a Unix-based command interface and protocol for securely getting access to a remote computer. It is widely used by network administrators to control Web and other kinds of servers remotely. SSH is actually a suite of three utilities (slogin, ssh, and scp) that are secure versions of the earlier UNIX utilities, rlogin, rsh, and rcp. SSH commands are encrypted and secure in several ways. Both ends of the client/server connection are authenticated using a digital certificate, and passwords are protected by encryption. Popular implementations include OpenSSH on Linux/Mac and Putty on Windows.

ssh-keys OpenSSH is a network connectivity tool, which encrypts all traffic including passwords to effectively eliminate eavesdropping, connection hijacking, and other network-level attacks. SSH-keys are part of the OpenSSH bundle. On UAntwerpen-HPC clusters, ssh-keys allow password-less access between compute nodes while running batch or interactive parallel jobs.

stack In the context of OpenStack, a stack is a collection of cloud resources which can be managed using the Heat orchestration engine.

supercomputer A computer with an extremely high processing capacity or processing power.

swap space A quantity of virtual memory available for use by batch jobs. Swap is a consumable resource provided by nodes and consumed by jobs.

TLB Translation Look-aside Buffer, a table in the CPU's memory that contains information about the virtual memory pages the CPU has accessed recently. The table cross-references a program's virtual addresses with the corresponding absolute addresses in physical memory that the program has most recently used. The TLB enables faster computing because it allows the address processing to take place independent of the normal address-translation pipeline.

walltime Walltime is the length of time specified in the job script for which the job will run on a batch system, you can visualise walltime as the time measured by a wall mounted clock (or your digital wrist watch). This is a computational resource.

worker node See compute node.

Contents

Part I

Beginner's Guide

Chapter 1

Introduction to HPC

1.1 What is HPC?

"**High Performance Computing**" (HPC) is computing on a "*Supercomputer*", a computer with at the frontline of contemporary processing capacity – particularly speed of calculation and available memory.

While the supercomputers in the early days (around 1970) used only a few processors, in the 1990s machines with thousands of processors began to appear and, by the end of the 20th century, massively parallel supercomputers with tens of thousands of "off-the-shelf" processors were the norm. A large number of dedicated processors are placed in close proximity to each other in a computer cluster.

A **computer cluster** consists of a set of loosely or tightly connected computers that work together so that in many respects they can be viewed as a single system.

The components of a cluster are usually connected to each other through fast local area networks ("LAN") with each *node* (computer used as a server) running its own instance of an operating system. Computer clusters emerged as a result of convergence of a number of computing trends including the availability of low cost microprocessors, high-speed networks, and software for high performance distributed computing.

Compute clusters are usually deployed to improve performance and availability over that of a single computer, while typically being more cost-effective than single computers of comparable speed or availability.

Supercomputers play an important role in the field of computational science, and are used for a wide range of computationally intensive tasks in various fields, including quantum mechanics, weather forecasting, climate research, oil and gas exploration, molecular modelling (computing the structures and properties of chemical compounds, biological macromolecules, polymers, and crystals), and physical simulations (such as simulations of the early moments of the universe, airplane and spacecraft aerodynamics, the detonation of nuclear weapons, and nuclear fusion).[1]

[1] Wikipedia: http://en.wikipedia.org/wiki/Supercomputer

1.2 What is the UAntwerpen-HPC?

The UAntwerpen-HPC is a collection of computers with AMD and/or Intel CPUs, running a Linux operating system, shaped like pizza boxes and stored above and next to each other in racks, interconnected with copper and fiber cables. Their number crunching power is (presently) measured in hundreds of billions of floating point operations (gigaflops) and even in teraflops.

The UAntwerpen-HPC relies on parallel-processing technology to offer University of Antwerp researchers an extremely fast solution for all their data processing needs.

The UAntwerpen-HPC consists of:

In technical terms	**... in human terms**
over 280 nodes and over 11000 cores	... or the equivalent of 2750 quad-core PCs
over 500 Terabyte of online storage	... or the equivalent of over 60000 DVDs
up to 100 Gbit InfiniBand fiber connections	... or allowing to transfer 3 DVDs per second

The UAntwerpen-HPC currently consists of:

Leibniz:

1. 144 compute nodes with 2 14-core Intel E5-2680v4 CPUs (Broadwell generation, 2.4 GHz) and 128 GB RAM, 120 GB local disk
2. 8 compute nodes with 2 14-core Intel E5-2680v4 CPUs (Broadwell generation, 2.4 GHz) and 256 GB RAM, 120 GB local disk
3. 24 “hopper” compute nodes (recovered from the former Hopper cluster) with 2 ten core Intel E5-2680v2 CPUs (Ivy Bridge generation, 2.8 GHz), 256 GB memory, 500 GB local disk
4. 2 GPGPU nodes with 2 14-core Intel E5-2680v4 CPUs (Broadwell generation, 2.4 GHz), 128 GB RAM and two NVIDIA Tesla P100 GPUs with 16 GB HBM2 memory per GPU, 120 GB local disk
5. 1 vector computing node with 1 12-core Intel Xeon Gold 6126 (Skylake generation, 2.6 GHz), 96 GB RAM and 2 NEC SX-Aurora Vector Engines type 10B (per card 8 cores @1.4 GHz, 48 GB HBM2), 240 GB local disk

6. 1 Xeon Phi node with 2 14-core Intel E5-2680v4 CPUs (Broadwell generation, 2.4 GHz), 128 GB RAM and Intel Xeon Phi 7220P PCIe card with 16 GB of RAM, 120 GB local disk

7. 1 visualisation node with 2 14-core Intel E5-2680v4 CPUs (Broadwell generation, 2.4 GHz), 256 GB RAM and with a NVIDIA P5000 GPU, 120 GB local disk

The nodes are connected using an InfiniBand EDR network except for the "hopper" compute nodes that utilize FDR10 InfiniBand.

Vaughan:

1. 104 compute nodes with 2 32-core AMD Epyc 7452 (2.35 GHz) and 256 GB RAM, 240 GB local disk

The nodes are connected using an InfiniBand HDR100 network.

All the nodes in the UAntwerpen-HPC run under the "CentOS Linux release 7.8.2003 (Core)" operating system, which is a clone of "RedHat Enterprise Linux", with *cgroups* support.

Two tools perform the **job management** and **job scheduling**:

1. TORQUE: a resource manager (based on PBS);

2. Moab: job scheduler and management tools.

For maintenance and monitoring, we use:

1. Ganglia: monitoring software;

2. Icinga and Nagios: alert manager.

1.3 What the HPC infrastucture is *not*

The HPC infrastructure is *not* a magic computer that automatically:

1. runs your PC-applications much faster for bigger problems;

2. develops your applications;

3. solves your bugs;

4. does your thinking;

5. ...

6. allows you to play games even faster.

The UAntwerpen-HPC does not replace your desktop computer.

1.4 Is the UAntwerpen-HPC a solution for my computational needs?

1.4.1 Batch or interactive mode?

Typically, the strength of a supercomputer comes from its ability to run a huge number of programs (i.e., executables) in parallel without any user interaction in real time. This is what is called "running in batch mode".

It is also possible to run programs at the UAntwerpen-HPC, which require user interaction. (pushing buttons, entering input data, etc.). Although technically possible, the use of the UAntwerpen-HPC might not always be the best and smartest option to run those interactive programs. Each time some user interaction is needed, the computer will wait for user input. The available computer resources (CPU, storage, network, etc.) might not be optimally used in those cases. A more in-depth analysis with the UAntwerpen-HPC staff can unveil whether the UAntwerpen-HPC is the desired solution to run interactive programs. Interactive mode is typically only useful for creating quick visualisations of your data without having to copy your data to your desktop and back.

1.4.2 What are cores, processors and nodes?

In this manual, the terms core, processor and node will be frequently used, so it's useful to understand what they are.

Modern servers, also referred to as *(worker)nodes* in the context of HPC, include one or more sockets, each housing a multi-*core processor* (next to memory, disk(s), network cards, ...). A modern *processor* consists of multiple CPUs or *cores* that are used to execute *computations*.

1.4.3 Parallel or sequential programs?

Parallel programs

Parallel computing is a form of computation in which many calculations are carried out simultaneously. They are based on the principle that large problems can often be divided into smaller ones, which are then solved concurrently ("in parallel").

Parallel computers can be roughly classified according to the level at which the hardware supports parallelism, with multicore computers having multiple processing elements within a single machine, while clusters use multiple computers to work on the same task. Parallel computing has become the dominant computer architecture, mainly in the form of multicore processors.

The two parallel programming paradigms most used in HPC are:

- OpenMP for shared memory systems (multithreading): on multiple cores of a single node
- MPI for distributed memory systems (multiprocessing): on multiple nodes

Parallel programs are more difficult to write than sequential ones, because concurrency introduces several new classes of potential software bugs, of which race conditions are the most

common. Communication and synchronisation between the different subtasks are typically some of the greatest obstacles to getting good parallel program performance.

Sequential programs

Sequential software does not do calculations in parallel, i.e., it only uses *one single core of a single workernode*. **It does not become faster by just throwing more cores at it**: it can only use one core.

It is perfectly possible to also run purely **sequential programs** on the UAntwerpen-HPC.

Running your sequential programs on the most modern and fastest computers in the UAntwerpen-HPC can save you a lot of time. But it also might be possible to run multiple instances of your program (e.g., with different input parameters) on the UAntwerpen-HPC, in order to solve one overall problem (e.g., to perform a parameter sweep). This is another form of running your sequential programs in parallel.

1.4.4 What programming languages can I use?

You can use *any* programming language, *any* software package and *any* library provided it has a version that runs on Linux, specifically, on the version of Linux that is installed on the compute nodes, CentOS Linux release 7.8.2003 (Core).

For the most common **programming languages**, a compiler is available on CentOS Linux release 7.8.2003 (Core). Supported and common programming languages on the UAntwerpen-HPC are C/C++, FORTRAN, Java, Perl, Python, MATLAB, R, etc.

Supported and commonly used compilers are GCC, clang, J2EE and Intel Cluster Studio.

Commonly used software packages are:

- in bioinformatics: beagle, Beast, bowtie, MrBayes, SAMtools
- in chemistry: ABINIT, CP2K, Gaussian, Gromacs, LAMMPS, NWChem, Quantum Espresso, Siesta, VASP
- in engineering: COMSOL, OpenFOAM, Telemac
- in mathematics: JAGS, MATLAB, R
- for visuzalization: Gnuplot, ParaView.

Commonly used libraries are Intel MKL, FFTW, HDF5, PETSc and Intel MPI, OpenMPI.

Additional software can be installed "*on demand*". Please contact the UAntwerpen-HPC staff to see whether the UAntwerpen-HPC can handle your specific requirements.

1.4.5 What operating systems can I use?

All nodes in the UAntwerpen-HPC cluster run under CentOS Linux release 7.8.2003 (Core), which is a specific version of RedHat Enterprise Linux. This means that all programs (executables) should be compiled for CentOS Linux release 7.8.2003 (Core).

Users can connect from any computer in the University of Antwerp network to the UAntwerpen-HPC, regardless of the Operating System that they are using on their personal computer. Users can use any of the common Operating Systems (such as Windows, macOS or any version of Linux/Unix/BSD) and run and control their programs on the UAntwerpen-HPC.

A user does not need to have prior knowledge about Linux; all of the required knowledge is explained in this tutorial.

1.4.6 What does a typical workflow look like?

A typical workflow looks like:

1. Connect to the login nodes with SSH (see section 3.2)
2. Transfer your files to the cluster (see section 3.3)
3. Optional: compile your code and test it (for compiling, see chapter 13)
4. Create a job script and submit your job (see chapter 4)
5. Get some coffee and be patient:
   (a) Your job gets into the queue
   (b) Your job gets executed
   (c) Your job finishes
6. Study the results generated by your jobs, either on the cluster or after downloading them locally.

1.4.7 What is the next step?

When you think that the UAntwerpen-HPC is a useful tool to support your computational needs, we encourage you to acquire a VSC-account (as explained in chapter 2), read chapter 3, "Setting up the environment", and explore chapters 5 to 11 which will help you to transfer and run your programs on the UAntwerpen-HPC cluster.

Do not hesitate to contact the UAntwerpen-HPC staff for any help.

Chapter 2

Getting an HPC Account

2.1 Getting ready to request an account

All users of Antwerp University Association (AUHA) can request an account on the UAntwerpen-HPC, which is part of the Flemish Supercomputing Centre (VSC).

See chapter 9 for more information on who is entitled to an account.

The VSC, abbreviation of Flemish Supercomputer Centre, is a virtual supercomputer centre. It is a partnership between the five Flemish associations: the Association KU Leuven, Ghent University Association, Brussels University Association, Antwerp University Association and the University Colleges-Limburg. The VSC is funded by the Flemish Government.

The UAntwerpen-HPC clusters use public/private key pairs for user authentication (rather than passwords). Technically, the private key is stored on your local computer and always stays there; the public key is stored on the UAntwerpen-HPC. Access to the UAntwerpen-HPC is granted to anyone who can prove to have access to the corresponding private key on his local computer.

2.1.1 How do SSH keys work?

- an SSH public/private key pair can be seen as a lock and a key
- the SSH public key is equivalent with a *lock*: you give it to the VSC and they put it on the *door* that gives access to your account.
- the SSH private key is like a *physical key*: you don't hand it out to other people.
- anyone who has the key (and the optional password) can unlock the *door* and log in to the account.
- the *door* to your VSC account is special: it can have multiple *locks* (SSH public keys) attached to it, and you only need to open one *lock* with the corresponding *key* (SSH private key) to open the *door* (log in to the account).

Since all VSC clusters use Linux as their main operating system, you will need to get acquainted with using the command-line interface and using the terminal.

A typical Windows environment does not come with pre-installed software to connect and run command-line executables on a UAntwerpen-HPC. Some tools need to be installed on your Windows machine first, before we can start the actual work.

2.1.2 Get PuTTY: A free telnet/SSH client

We recommend to use the PuTTY tools package, which is freely available.

You do not need to install PuTTY, you can download the PuTTY and PuTTYgen executable and run it. This can be useful in situations where you do not have the required permissions to install software on the computer you are using. Alternatively, an installation package is also available.

You can download PuTTY from the official address: `https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html`. You probably want the 64-bits version. If you can install software on your computer, you can use the "Package files", if not, you can download and use `putty.exe` and `puttygen.exe` in the "Alternative binary files" section.

The PuTTY package consists of several components, but we'll only use two:

1. **PuTTY**: *the Telnet and SSH client itself* (to login, see subsection 3.2.1)

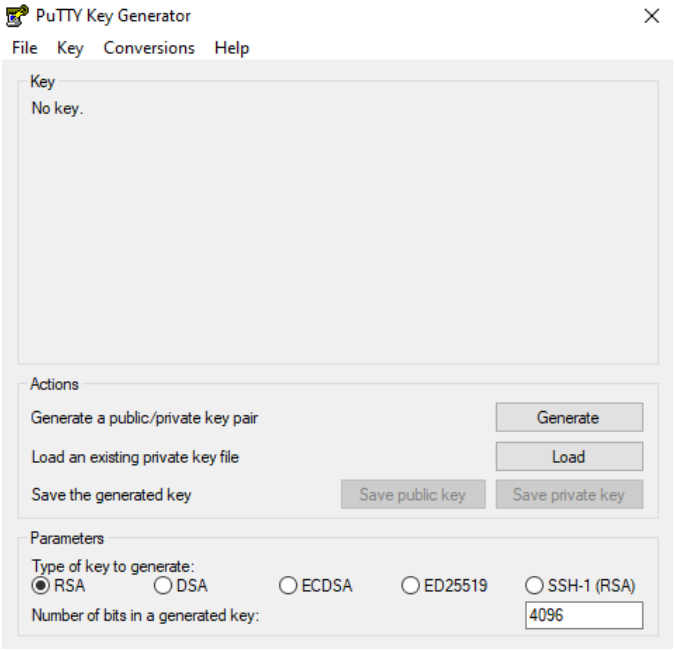
2. **PuTTYgen**: *an RSA and DSA key generation utility* (to generate a key pair, see subsection 2.1.3)

2.1.3 Generating a public/private key pair

Before requesting a VSC account, you need to generate a pair of *ssh* keys. You need 2 keys, a public and a private key. You can visualise the public key as a lock to which only you have the key (your private key). You can send a copy of your lock to anyone without any problems, because only you can open it, as long as you keep your private key secure. To generate a public/private key pair, you can use the PuTTYgen key generator.

Start ***PuTTYgen.exe*** it and follow these steps:

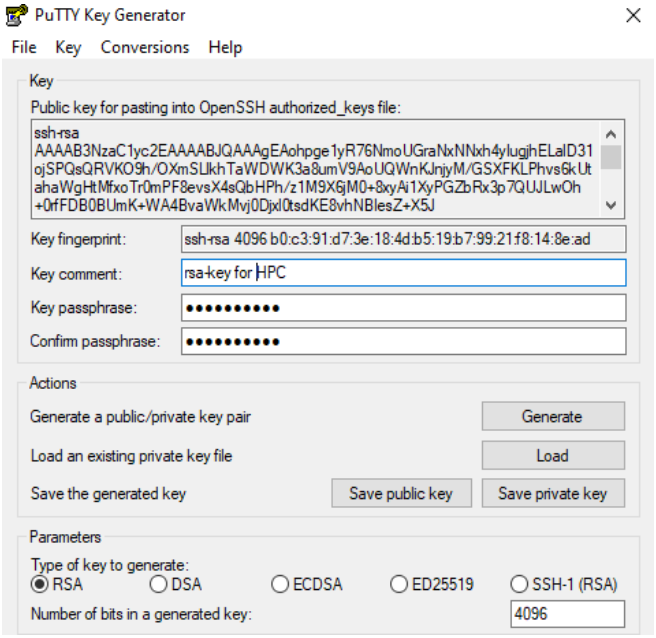
1. In Parameters (at the bottom of the window), choose "RSA" and set the number of bits in the key to 4096.



2. Click on Generate. To generate the key, you must move the mouse cursor over the PuTTYgen window (this generates some random data that PuTTYgen uses to generate the key pair). Once the key pair is generated, your public key is shown in the field Public key for pasting into OpenSSH authorized_keys file.

3. Next, it is advised to fill in the Key comment field to make it easier identifiable afterwards.

4. Next, you should specify a passphrase in the Key passphrase field and retype it in the Confirm passphrase field. Remember, the passphrase protects the private key against unauthorised use, so it is best to choose one that is not too easy to guess but that you can still remember. Using a passphrase is not required, but we recommend you to use a good passphrase unless you are certain that your computer's hard disk is encrypted with a decent password. (If you are not sure your disk is encrypted, it probably isn't.)



5. Save both the public and private keys in a folder on your personal computer (We recommend to create and put them in the folder "C:\Users\%USERNAME%\AppData\Local\PuTTY\.ssh") with the buttons Save public key and Save private key. We recommend using the name "**id_rsa.pub**" for the public key, and "**id_rsa.ppk**" for the private key.

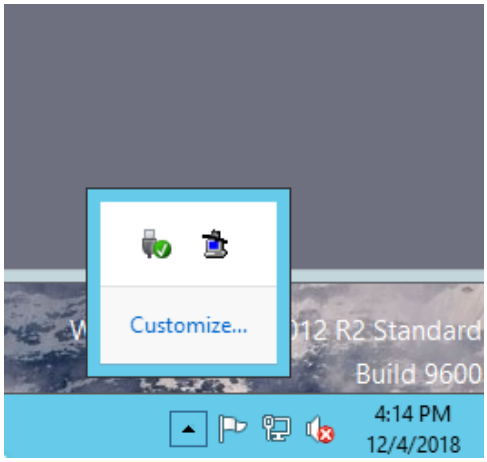
If you use another program to generate a key pair, please remember that they need to be in the OpenSSH format to access the UAntwerpen-HPC clusters.

2.1.4 Using an SSH agent (optional)

It is possible to setup a SSH agent in Windows. This is an optional configuration to help you to keep all your SSH keys (if you have several) stored in the same key ring to avoid to type the SSH key password each time. The SSH agent is also necessary to enable SSH hops with key forwarding from Windows.

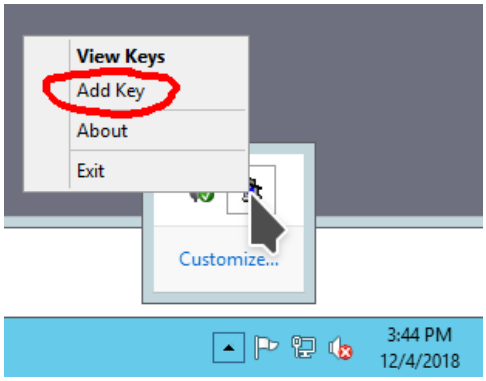
Pageant is the SSH authentication agent used in windows. This agent should be available from the PuTTY installation package https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html or as stand alone binary package.

After the installation just start the Pageant application in Windows, this will start the agent in background. The agent icon will be visible from the Windows panel.

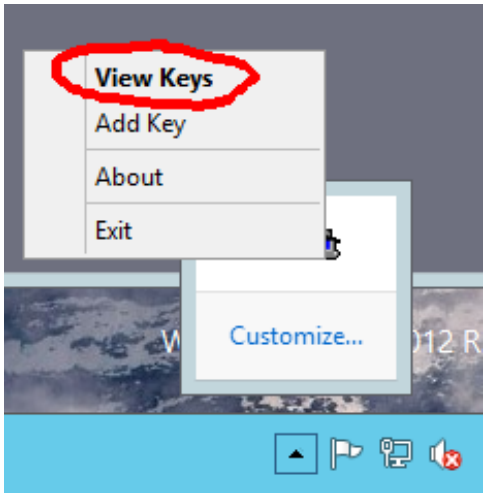


At this point the agent does not contain any private key. You should include the private key(s) generated in the previous section subsection 2.1.3.

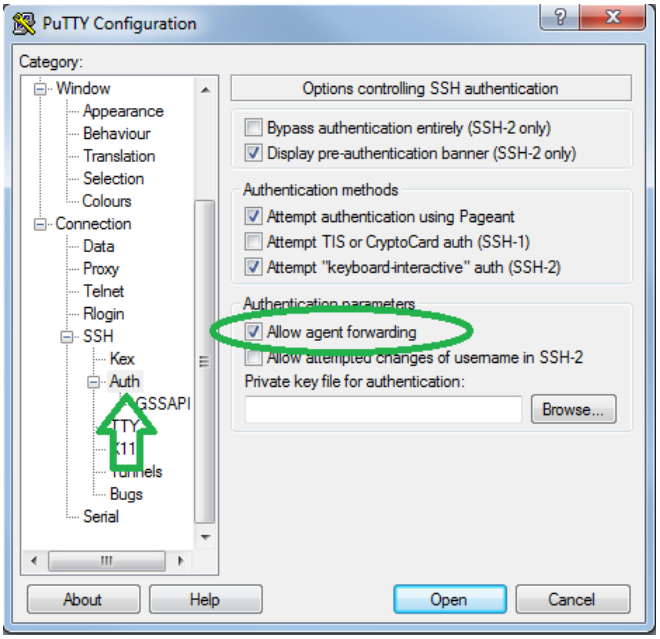
1. Click on Add key



2. Select the private key file generated in subsection 2.1.3 ("**id_rsa.ppk**" by default).
3. Enter the same SSH key password used to generate the key. After this step the new key will be included in Pageant to manage the SSH connections.
4. You can see the SSH key(s) available in the key ring just clicking on View Keys.



5. You can change **PuTTY** setup to use the SSH agent. Open PuTTY and check *Connection > SSH > Auth > Allow agent forwarding.*



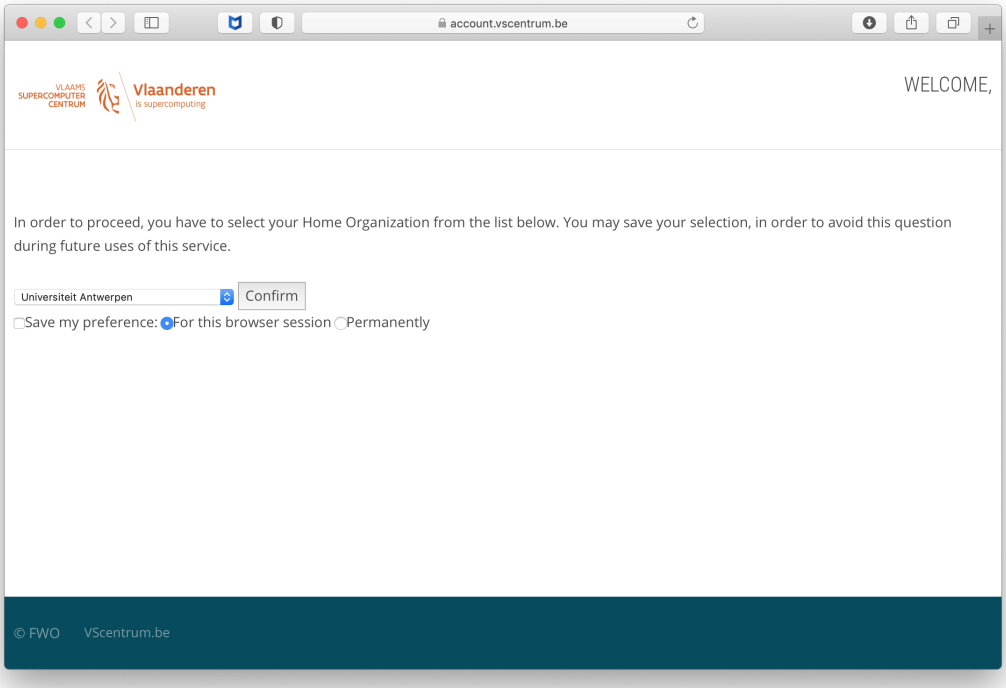
Now you can connect to the login nodes as usual. The SSH agent will know which SSH key should be used and you do not have to type the SSH passwords each time, this task is done by **Pageant** agent automatically.

It is also possible to use **WinSCP** with **Pageant**, see https://winscp.net/eng/docs/ui_pageant for more details.

2.2 Applying for the account

Visit https://account.vscentrum.be/

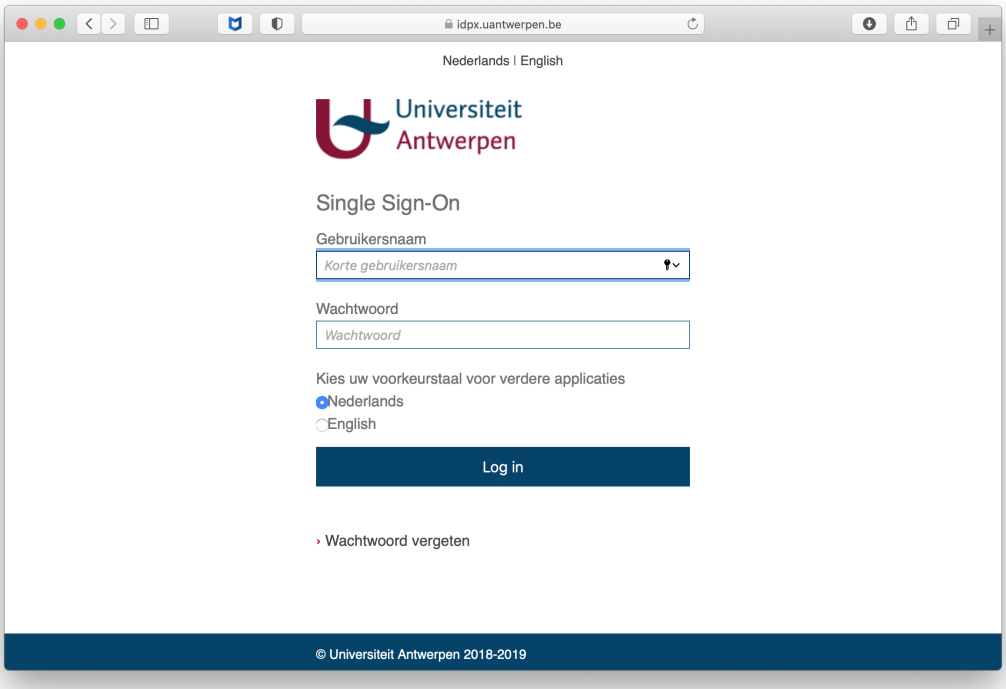
You will be redirected to our WAYF (Where Are You From) service where you have to select your "Home Organisation".



Select “Universiteit Antwerpen” in the dropdown box and optionally select “Save my preference” and “permanently”.

Click Confirm

You will now be taken to the authentication page of your institute.



The site is only accessible from within the University of Antwerp domain, so the page won't load from, e.g., home. However, you can also get external access to the University of Antwerp domain using VPN. We refer to the Pintra pages of the ICT Department for more information.

2.2.1 Users of the Antwerp University Association (AUHA)

All users (researchers, academic staff, etc.) from the higher education institutions associated with University of Antwerp can get a VSC account via the University of Antwerp. There is not yet an automated form to request your personal VSC account.

Please e-mail the UAntwerpen-HPC staff to get an account (see Contacts information). You will have to provide a public *ssh* key generated as described above. Please attach your public key (i.e., the file named `id_rsa.pub`), which you will normally find in your .ssh subdirectory within your HOME Directory. (i.e., `/Users/<username>/.ssh/id_rsa.pub`).

After you log in using your University of Antwerp login and password, you will be asked to upload the file that contains your public key, i.e., the file "id_rsa.pub" which you have generated earlier. Make sure that your public key is actually accepted for upload, beacuse if it is in a wrong format, wrong type or too short, then it will be refused.

This file should have been stored in the directory "C:\Users\%USERNAME%\AppData\Local\PuTTY\.ssh"

After you have uploaded your public key you will receive an e-mail with a link to confirm your e-mail address. After confirming your e-mail address the VSC staff will review and if applicable approve your account.

2.2.2 Welcome e-mail

Within one day, you should receive a Welcome e-mail with your VSC account details.

```
Dear (Username),
Your VSC-account has been approved by an administrator.
Your vsc-username is vsc20167

Your account should be fully active within one hour.

To check or update your account information please visit
https://account.vscentrum.be/

For further info please visit https://www.vscentrum.be/user-portal

Kind regards,
-- The VSC administrators
```

Now, you can start using the UAntwerpen-HPC. You can always look up your VSC id later by visiting
https://account.vscentrum.be.

2.2.3 Adding multiple SSH public keys (optional)

In case you are connecting from different computers to the login nodes, it is advised to use separate SSH public keys to do so. You should follow these steps.

1. Create a new public/private SSH key pair from Putty. Repeat the process described in section 2.1.3.
2. Go to https://account.vscentrum.be/django/account/edit
3. Upload the new SSH public key using the **Add public key** section. Make sure that your public key is actually saved, because a public key will be refused if it is too short, wrong type, or in a wrong format.
4. (optional) If you lost your key, you can delete the old key on the same page. You should keep at least one valid public SSH key in your account.
5. Take into account that it will take some time before the new SSH public key is active in your account on the system; waiting for 15-30 minutes should be sufficient.

2.3 Computation Workflow on the UAntwerpen-HPC

A typical Computation workflow will be:

1. Connect to the UAntwerpen-HPC
2. Transfer your files to the UAntwerpen-HPC

3. Compile your code and test it
4. Create a job script
5. Submit your job
6. Wait while
   (a) your job gets into the queue
   (b) your job gets executed
   (c) your job finishes
7. Move your results

We'll take you through the different tasks one by one in the following chapters.

Chapter 3

Connecting to the HPC infrastructure

Before you can really start using the UAntwerpen-HPC clusters, there are several things you need to do or know:

1. You need to **log on to the cluster** using an SSH client to one of the login nodes. This will give you command-line access. The software you'll need to use on your client system depends on its operating system.
2. Before you can do some work, you'll have to **transfer the files** that you need from your desktop computer to the cluster. At the end of a job, you might want to transfer some files back.
3. Optionally, if you wish to use programs with a **graphical user interface**, you will need an X-server on your client system and log in to the login nodes with X-forwarding enabled.
4. Often several versions of **software packages and libraries** are installed, so you need to select the ones you need. To manage different versions efficiently, the VSC clusters use so-called **modules**, so you will need to select and load the modules that you need.

3.1 Connection restrictions

Since March 20th 2020, restrictions are in place that limit from where you can connect to the VSC HPC infrastructure, in response to security incidents involving several European HPC centres.

VSC login nodes are only directly accessible from within university networks, and from (most) Belgian commercial internet providers.

All other IP domains are blocked by default. If you are connecting from an IP address that is not allowed direct access, you have the following options to get access to VSC login nodes:

- Use an VPN connection to connect to the University of Antwerp network (recommended).
- Register your IP address by accessing `https://firewall.hpc.kuleuven.be` (same URL regardless of the university your affiliated with) and log in with your University of Antwerp account.

  - While this web connection is active new SSH sessions can be started.

  - Active SSH sessions will remain active even when this web page is closed.

- Contact your HPC support team (via hpc@uantwerpen.be) and ask them to whitelist your IP range (e.g., for industry access, automated processes).

Trying to establish an SSH connection from an IP address that does not adhere to these restrictions will result in an immediate failure to connect, with an error message like:

```
ssh_exchange_identification: read: Connection reset by peer
```

3.2 First Time connection to the HPC infrastructure

If you have any issues connecting to the UAntwerpen-HPC after you've followed these steps, see section 8.3 to troubleshoot.

When connecting from outside Belgium, you need a VPN client to connect to the University of Antwerp network first.

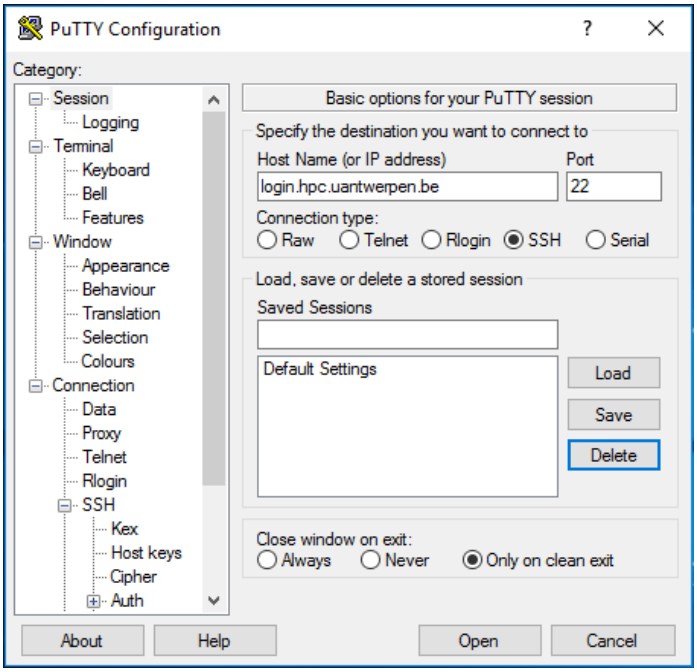
3.2.1 Open a Terminal

You've generated a public/private key pair with PuTTYgen and have an approved account on the VSC clusters. The next step is to setup the connection to (one of) the UAntwerpen-HPC.

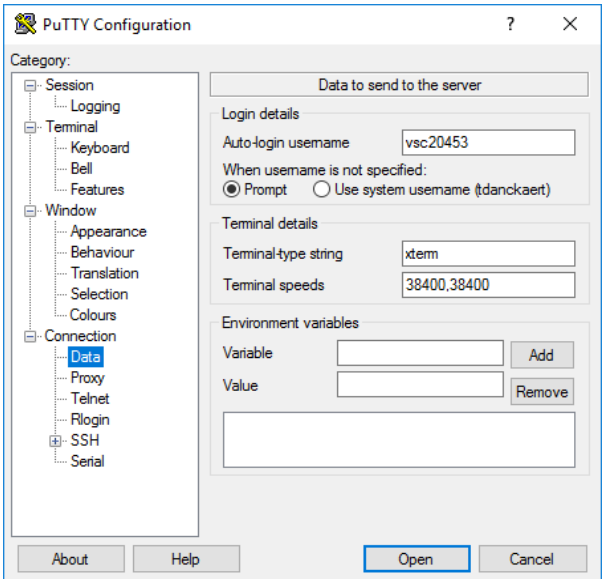
In the screenshots, we show the setup for user "***vsc20167***" to the UAntwerpen-HPC cluster via the login node "***login.hpc.uantwerpen.be***".

1. Start the PuTTY executable `putty.exe` in your directory `C:\Program Files (x86)\PuTTY` and the configuration screen will pop up. As you will often use the PuTTY tool, we recommend adding a shortcut on your desktop.

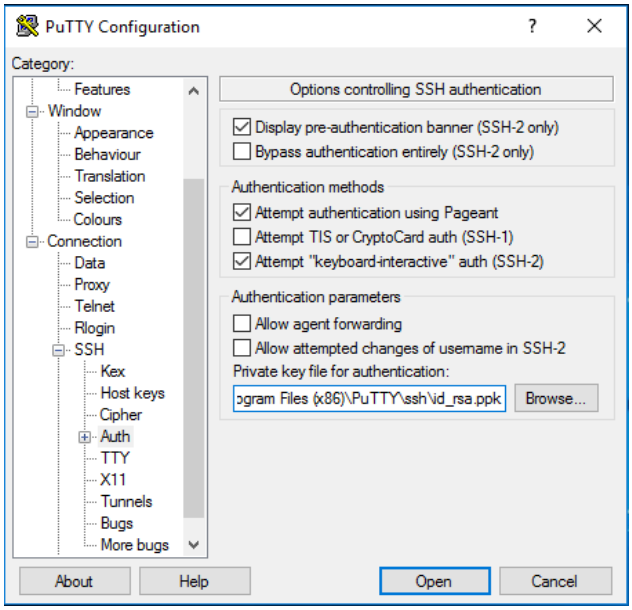
2. Within the category <***Session***>, in the field <***Host Name***>, enter the name of the login node of the UAntwerpen-HPC cluster (i.e., "***login.hpc.uantwerpen.be***") you want to connect to.



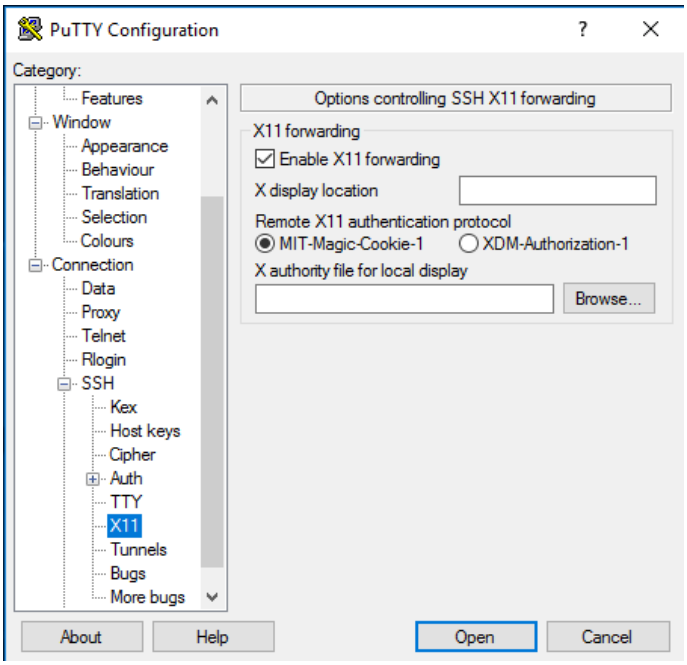
3. In the category Connection » Data, in the field Auto-login username, put in <*vsc20167*>, which is your VSC username that you have received by e-mail after your request was approved.



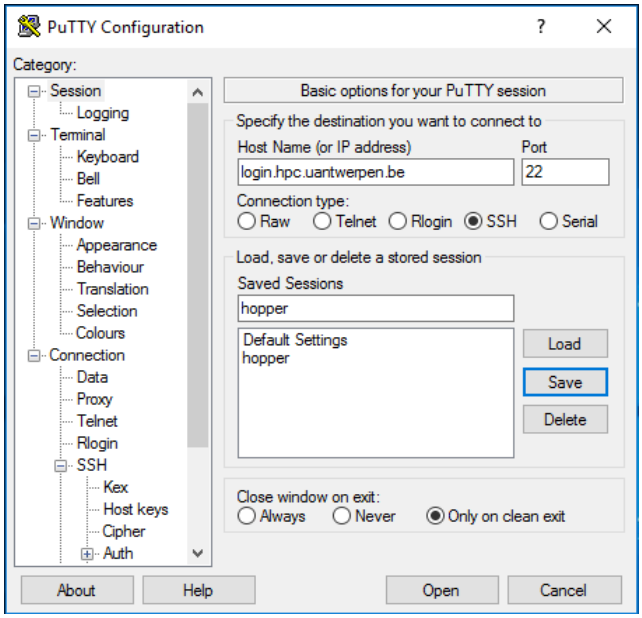
4. In the category Connection » SSH » Auth, in the field Private key file for authentication click on Browse and select the private key (i.e., "id_rsa.ppk") that you generated and saved above.



5. In the category Connection » SSH » X11, click the Enable X11 Forwarding checkbox.



6. Now go back to <Session>, and fill in "*leibniz*" in the Saved Sessions field and press Save to store the session information.



7. Now pressing Open, will open a terminal window and asks for you passphrase.

8. If this is your first time connecting, you will be asked to verify the authenticity of the login node. Please see section 8.6 on how to do this.

9. After entering your correct passphrase, you will be connected to the login-node of the UAntwerpen-HPC.

10. To check you can now "Print the Working Directory" (pwd) and check the name of the computer, where you have logged in (hostname):

```
$ pwd
/user/antwerpen/201/vsc20167
$ hostname -f
ln2.leibniz.uantwerpen.vsc
```

11. For future PuTTY sessions, just select your saved session (i.e. "*leibniz*") from the list, Load it and press Open.

Congratulations, you're on the UAntwerpen-HPC infrastructure now! To find out where you have landed you can print the current working directory:

```
$ pwd
/user/antwerpen/201/vsc20167
```

Your new private home directory is "/user/antwerpen/201/vsc20167". Here you can create your own subdirectory structure, copy and prepare your applications, compile and test them and submit your jobs on the UAntwerpen-HPC.

```
$ cd /apps/antwerpen/tutorials
$ ls
Intro-HPC/
```

This directory currently contains all training material for the ***Introduction to the UAntwerpen-HPC***. More relevant training material to work with the UAntwerpen-HPC can always be added later in this directory.

You can now explore the content of this directory with the "ls –l" (**l**ist**s** **l**ong) and the "cd" (**c**hange **d**irectory) commands:

As we are interested in the use of the ***HPC***, move further to ***Intro-HPC*** and explore the contents up to 2 levels deep:

```
$ cd Intro-HPC
$ tree -L 2
.
`-- examples
    |-- Compiling-and-testing-your-software-on-the-HPC
    |-- Fine-tuning-Job-Specifications
    |-- Multi-core-jobs-Parallel-Computing
    |-- Multi-job-submission
    |-- Program-examples
    |-- Running-batch-jobs
    |-- Running-jobs-with-input
    |-- Running-jobs-with-input-output-data
    |-- example.pbs
    `-- example.sh
9 directories, 5 files
```

This directory contains:

1. This ***HPC Tutorial*** (in either a Mac, Linux or Windows version).

2. An ***examples*** subdirectory, containing all the examples that you need in this Tutorial, as well as examples that might be useful for your specific applications.

```
$ cd examples
```

Tip: Typing `cd ex` followed by ⇥ (the Tab-key) will generate the `cd examples` command. **Command-line completion** (also tab completion) is a common feature of the bash command line interpreter, in which the program automatically fills in partially typed commands.

Tip: For more exhaustive tutorials about Linux usage, see Appendix C

The first action is to copy the contents of the UAntwerpen-HPC examples directory to your home directory, so that you have your own personal copy and that you can start using the examples. The "-r" option of the copy command will also copy the contents of the sub-directories "*recursively*".

```
$ cp -r /apps/antwerpen/tutorials/Intro-HPC/examples ~/
```

Upon connection, you will get a welcome message containing your last login timestamp and some pointers to information about the system. On Leibniz, the system will also show your disk quota.

```
Last login: Mon Feb  2 17:58:13 2015 from mylaptop.uantwerpen.be

------------------------------------------------------------------

Welcome to LEIBNIZ !

Useful links:
  https://vscdocumentation.readthedocs.io
  https://vscdocumentation.readthedocs.io/en/latest/antwerp/tier2_hardware.html
  https://www.uantwerpen.be/hpc

Questions or problems? Do not hesitate and contact us:
  hpc@uantwerpen.be

Happy computing!

------------------------------------------------------------------

Your quota is:

                  Block Limits
   Filesystem       used      quota      limit    grace
   user              740M         3G       3.3G     none
   data            3.153G        25G      27.5G     none
   scratch         12.38M        25G      27.5G     none
   small           20.09M        25G      27.5G     none

                  File Limits
   Filesystem      files      quota      limit    grace
   user            14471      20000      25000     none
   data             5183     100000     150000     none
   scratch            59     100000     150000     none
   small            1389     100000     110000     none

------------------------------------------------------------------
```

You can exit the connection at anytime by entering:

```
$ exit
logout
Connection to login.hpc.uantwerpen.be closed.
```

Tip: Setting your Language right:

You may encounter a warning message similar to the following one during connecting:

```
perl: warning: Setting locale failed.
perl: warning: Please check that your locale settings:
LANGUAGE = (unset),
LC_ALL = (unset),
LC_CTYPE = "UTF-8",
LANG = (unset)
    are supported and installed on your system.
perl: warning: Falling back to the standard locale ("C").
```

or any other error message complaining about the locale.

This means that the correct "locale" has not yet been properly specified on your local machine. Try:

```
$ locale
LANG=
LC_COLLATE="C"
LC_CTYPE="UTF-8"
LC_MESSAGES="C"
LC_MONETARY="C"
LC_NUMERIC="C"
LC_TIME="C"
LC_ALL=
```

A **locale** is a set of parameters that defines the user's language, country and any special variant preferences that the user wants to see in their user interface. Usually a locale identifier consists of at least a language identifier and a region identifier.

3.3 Transfer Files to/from the UAntwerpen-HPC

Before you can do some work, you'll have to **transfer the files** you need from your desktop or department to the cluster. At the end of a job, you might want to transfer some files back.

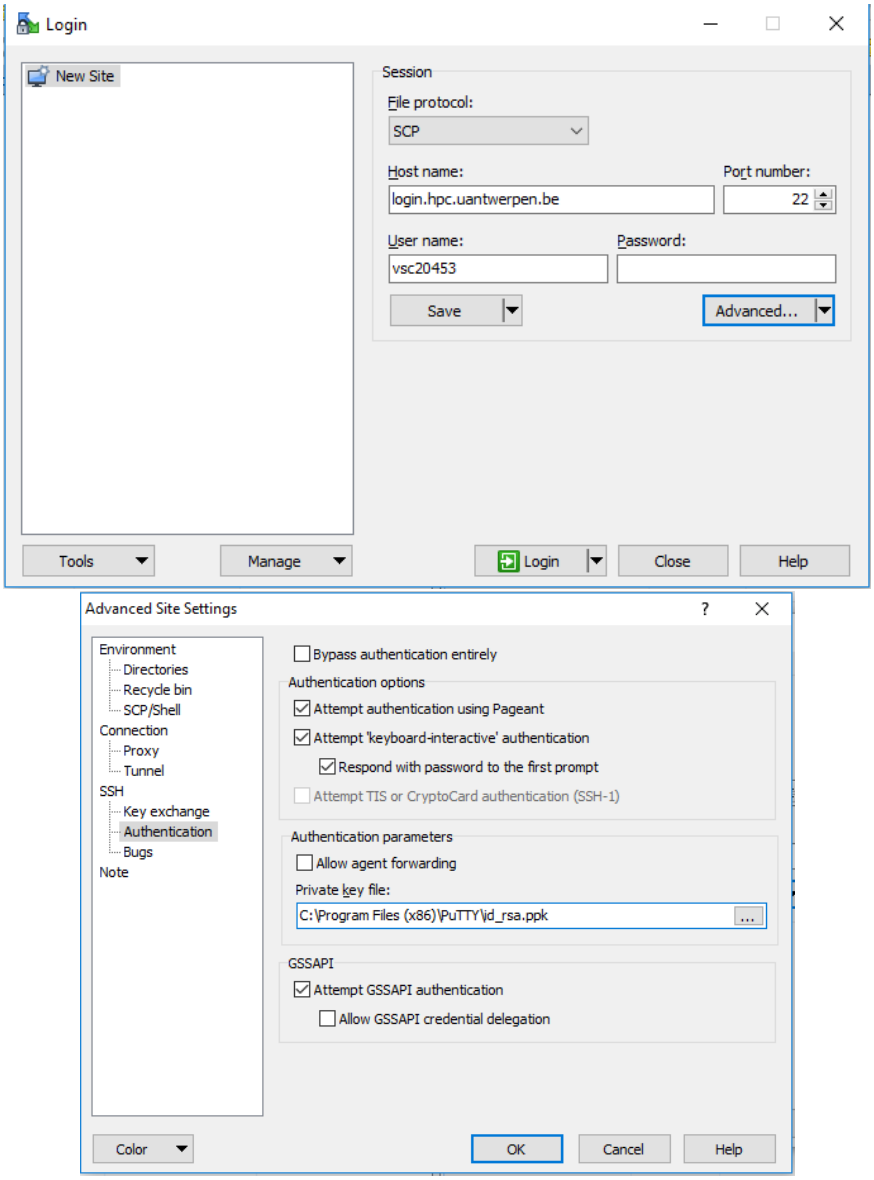
3.3.1 WinSCP

To transfer files to and from the cluster, we recommend the use of WinSCP, a graphical file management tool which can transfer files using secure protocols such as SFTP and SCP. WinSCP is freely available from http://www.winscp.net.

To transfer your files using WinSCP,

1. Open the program
2. The Login menu is shown automatically (if it is closed, click New Session to open it again). Fill in the necessary fields under Session
   (a) Click New Site.
   (b) Enter "*login.hpc.uantwerpen.be*" in the Host name field.
   (c) Enter your "*vsc-account*" in the User name field.
   (d) Select SCP as the file protocol.

(e) Note that the password field remains empty.



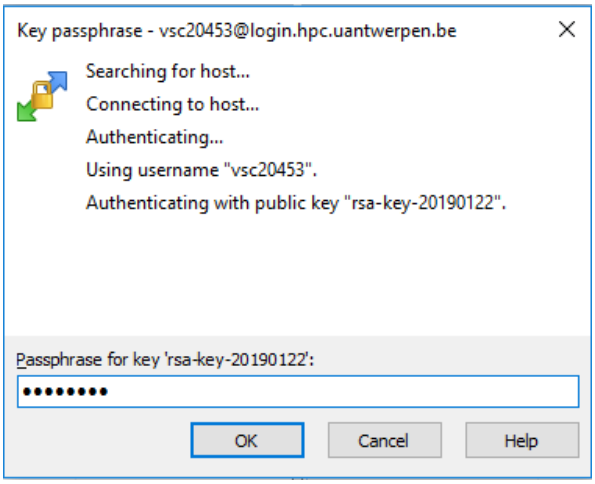
(f) Click Advanced....

(g) Click SSH » Authentication.

(h) Select your private key in the field Private key file.

3. Press the Save button, to save the session under Session » Sites for future access.

4. Finally, when clicking on Login, you will be asked for your key passphrase.



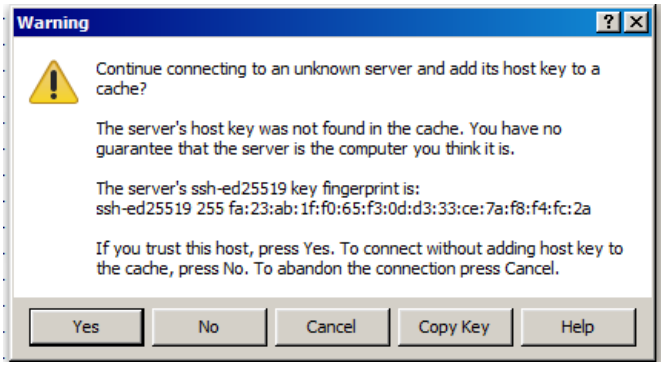
The first time you make a connection to the login node, a Security Alert will appear and you will be asked to verify the authenticity of the login node.

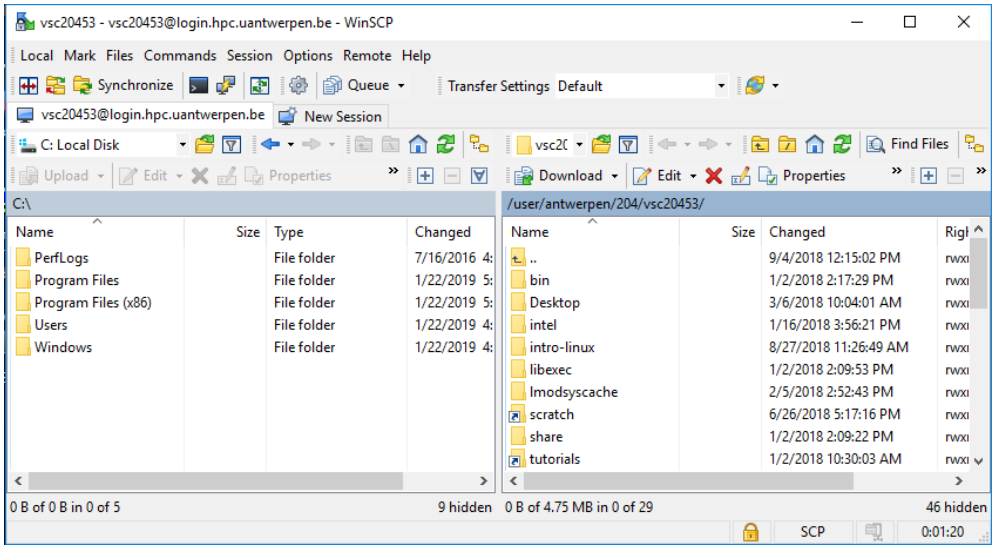
Make sure the fingerprint in the alert matches one of the following:

- `ssh-rsa 2048 5a:40:9d:2a:f4:b7:6c:87:0d:87:30:07:9d:ea:80:11`
- `sha-rsa 2048 SHA256:W8HRHTZpPd2GIhoNU2xj2oUKhcFr2bjIzZsKzY+1PCA`
- `ssh-ecdsa 256 f9:4f:19:9b:fb:40:c5:6c:6f:b9:64:2e:33:0a:8d:26`
- `ssh-ecdsa 256 SHA256:DlsP+jFrTqSdr9VquUpDj17Uy99zFdFN/LxVhaQQzbo`
- `ssh-ed25519 255 df:0c:61:b9:26:51:0f:b4:ca:43:ac:f6:ee:d2:a1:29`
- `ssh-ed25519 255 SHA256:+vlrkJui34B4iumxGVHd447K3W8wzgE1n1h2/Ic0WlE`

If it does, press ***Yes***, if it doesn't, please contact hpc@uantwerpen.be.

Note: it is possible that the `ssh-ed25519` fingerprint starts with "`ssh-ed25519 255`" rather than "`ssh-ed25519 256`" (or vice versa), depending on the PuTTY version you are using. It is safe to ignore this `255` versus `256` difference, but the part after should be **identical**.





Now, try out whether you can transfer an arbitrary file from your local machine to the HPC and back.

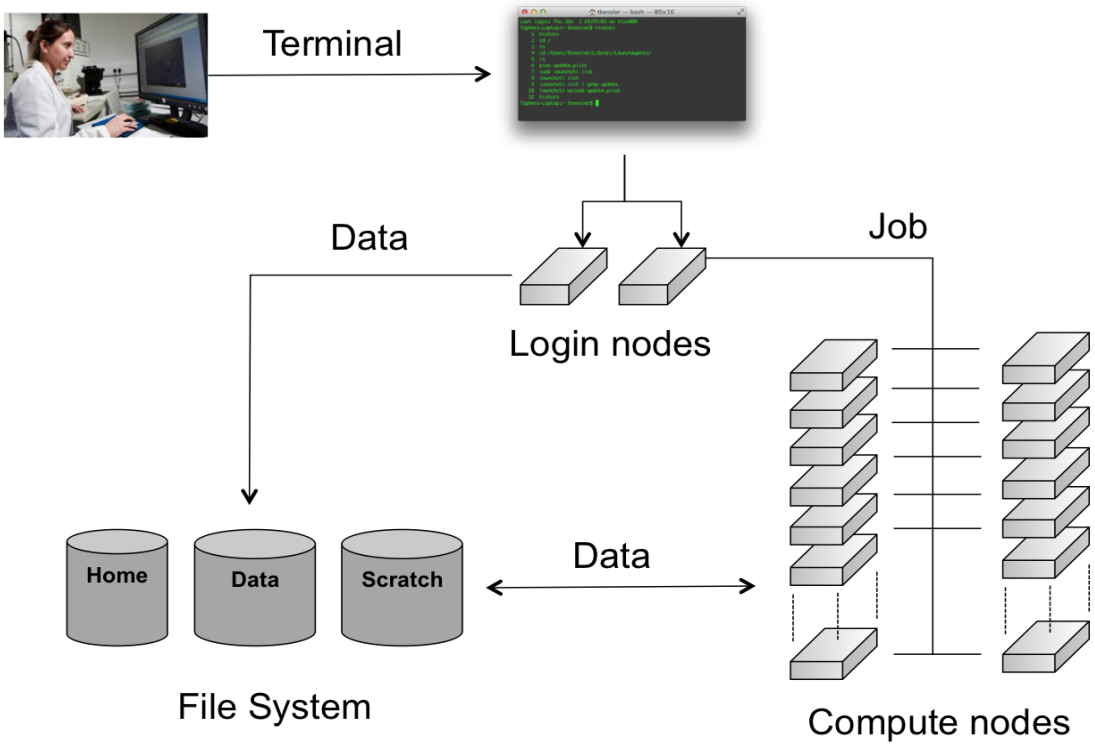
3.3.2 Fast file transfer for large datasets

See the section on `rsync` in chapter 5 of the Linux intro manual.

Chapter 4

Running batch jobs

In order to have access to the compute nodes of a cluster, you have to use the job system. The system software that handles your batch jobs consists of two pieces: the queue- and resource manager **TORQUE** and the scheduler **Moab**. Together, TORQUE and Moab provide a suite of commands for submitting jobs, altering some of the properties of waiting jobs (such as reordering or deleting them), monitoring their progress and killing ones that are having problems or are no longer needed. Only the most commonly used commands are mentioned here.



When you connect to the UAntwerpen-HPC, you have access to (one of) the **login nodes** of the cluster. There you can prepare the work you want to get done on the cluster by, e.g., installing or compiling programs, setting up data sets, etc. The computations however, should not be performed on this login node. The actual work is done on the cluster's **compute nodes**. Each

compute node contains a number of CPU **cores**. The compute nodes are managed by the job scheduling software (Moab) and a Resource Manager (TORQUE), which decides when and on which compute nodes the jobs can run. It is usually not necessary to log on to the compute nodes directly . Users can (and should) monitor their jobs periodically as they run, but do not have to remain connected to the UAntwerpen-HPC the entire time.

The documentation in this "Running batch jobs" section includes a description of the general features of job scripts, how to submit them for execution and how to monitor their progress.

4.1 Modules

Software installation and maintenance on a UAntwerpen-HPC cluster such as the VSC clusters poses a number of challenges not encountered on a workstation or a departmental cluster. We therefore need a system on the UAntwerpen-HPC, which is able to easily activate or deactivate the software packages that you require for your program execution.

4.1.1 Environment Variables

The program environment on the UAntwerpen-HPC is controlled by pre-defined settings, which are stored in environment (or shell) variables. For more information about environment variables, see the chapter "Getting started", section "Variables" in the intro to Linux.

All the software packages that are installed on the UAntwerpen-HPC cluster require different settings. These packages include compilers, interpreters, mathematical software such as MATLAB and SAS, as well as other applications and libraries.

4.1.2 The module command

In order to administer the active software and their environment variables, the module system has been developed, which:

1. Activates or deactivates *software packages* and their dependencies.
2. Allows setting and unsetting of *environment variables*, including adding and deleting entries from list-like environment variables.
3. Does this in a *shell-independent* fashion (necessary information is stored in the accompanying module file).
4. Takes care of *versioning aspects*: For many libraries, multiple versions are installed and maintained. The module system also takes care of the versioning of software packages. For instance, it does not allow multiple versions to be loaded at same time.
5. Takes care of *dependencies*: Another issue arises when one considers library versions and the dependencies they require. Some software requires an older version of a particular library to run correctly (or at all). Hence a variety of version numbers is available for important libraries. Modules typically load the required dependencies automatically.

This is all managed with the `module` command, which is explained in the next sections.

4.1.3 Available modules

A large number of software packages are installed on the UAntwerpen-HPC clusters. A list of all currently available software can be obtained by typing:

```
$ module available
```

It's also possible to execute `module av` or `module avail`, these are shorter to type and will do the same thing.

This will give some output such as:

```
$ module av 2>&1 | more
------------- /apps/antwerpen/modules/hopper/2015a/all ------------
ABINIT/7.10.2-intel-2015a
ADF/2014.05
Advisor/2015_update1
Bison/3.0.4-intel-2015a
Boost/1.57.0-foss-2015a-Python-2.7.9
Boost/1.57.0-intel-2015a-Python-2.7.9
bzip2/1.0.6-foss-2015a
bzip2/1.0.6-intel-2015a
...
```

Or when you want to check whether some specific software, some compiler or some application (e.g., LAMMPS) is installed on the UAntwerpen-HPC.

```
$ module av 2>&1 | grep -i -e "LAMMPS"
LAMMPS/9Dec14-intel-2015a
LAMMPS/30Oct14-intel-2014a
LAMMPS/5Sep14-intel-2014a
```

As you are not aware of the capitals letters in the module name, we looked for a case-insensitive name with the "-i" option.

This gives a full list of software packages that can be loaded.

The casing of module names is important: lowercase and uppercase letters matter in module names.

4.1.4 Organisation of modules in toolchains

The amount of modules on the VSC systems can be overwhelming, and it is not always immediately clear which modules can be loaded safely together if you need to combine multiple programs in a single job to get your work done.

Therefore the VSC has defined so-called **toolchains**. A toolchain contains a C/C++ and Fortran compiler, a MPI library and some basic math libraries for (dense matrix) linear algebra and FFT. Two toolchains are defined on most VSC systems. One, the `intel` toolchain, consists of the Intel compilers, MPI library and math libraries. The other one, the `foss` toolchain, consists of Open Source components: the GNU compilers, OpenMPI, OpenBLAS and the standard LAPACK and ScaLAPACK libraries for the linear algebra operations and the FFTW library for FFT. The toolchains are refreshed twice a year, which is reflected in their name.

E.g., `foss/2021a` is the first version of the `foss` toolchain in 2021.

The toolchains are then used to compile a lot of the software installed on the VSC clusters. You can recognise those packages easily as they all contain the name of the toolchain after the version number in their name (e.g., `Python/2.7.12-intel-2016b`). Only packages compiled with the same toolchain name and version can work together without conflicts.

4.1.5 Loading and unloading modules

module load

To "activate" a software package, you load the corresponding module file using the `module load` command:

```
$ module load example
```

This will load the most recent version of *example*.

For some packages, multiple versions are installed; the load command will automatically choose the default version (if it was set by the system administrators) or the most recent version otherwise (i.e., the lexicographical last after the `/`).

However, you should specify a particular version to avoid surprises when newer versions are installed:

```
$ module load secondexample/2.7-intel-2016b
```

Modules need not be loaded one by one; the two `module load` commands can be combined as follows:

```
$ module load example/1.2.3 secondexample/2.7-intel-2016b
```

This will load the two modules as well as their dependencies (unless there are conflicts between both modules).

module list

Obviously, you need to be able to keep track of the modules that are currently loaded. Assuming you have run the `module load` commands stated above, you will get the following:

```
$ module list
Currently Loaded Modulefiles:
1) example/1.2.3                                    6) imkl/11.3.3.210-iimpi
   -2016b
2) GCCcore/5.4.0                                    7) intel/2016b
3) icc/2016.3.210-GCC-5.4.0-2.26                    8) examplelib/1.2-intel
   -2016b
4) ifort/2016.3.210-GCC-5.4.0-2.26                  9) secondexample/2.7-intel
   -2016b
5) impi/5.1.3.181-iccifort-2016.3.210-GCC-5.4.0-2.26
```

It is important to note at this point that other modules (e.g., `intel/2016b`) are also listed,

although the user did not explicitly load them. This is because `secondexample/2.7-intel-2016b` depends on it (as indicated in its name), and the system administrator specified that the `intel/2016b` module should be loaded whenever *this* `secondexample` module is loaded. There are advantages and disadvantages to this, so be aware of automatically loaded modules whenever things go wrong: they may have something to do with it!

module unload

To unload a module, one can use the `module unload` command. It works consistently with the `load` command, and reverses the latter's effect. However, the dependencies of the package are NOT automatically unloaded; you will have to unload the packages one by one. When the `secondexample` module is unloaded, only the following modules remain:

```
$ module unload secondexample
$ module list
Currently Loaded Modulefiles:
Currently Loaded Modulefiles:
1) example/1.2.3                                    5) impi/5.1.3.181-iccifort
    -2016.3.210-GCC-5.4.0-2.26
2) GCCcore/5.4.0                                    6) imkl/11.3.3.210-iimpi
    -2016b
3) icc/2016.3.210-GCC-5.4.0-2.26                    7) intel/2016b
4) ifort/2016.3.210-GCC-5.4.0-2.26                  8) examplelib/1.2-intel
    -2016b
```

Notice that the version was not specified: there can only be one version of a module loaded at a time, so unloading modules by name is not ambiguous. However, checking the list of currently loaded modules is always a good idea, since unloading a module that is currently not loaded will *not* result in an error.

4.1.6 Purging all modules

In order to unload all modules at once, and hence be sure to start in a clean state, you can use:

```
$ module purge
```

However, on some VSC clusters you may be left with a very empty list of available modules after executing `module purge`. On those systems, `module av` will show you a list of modules containing the name of a cluster or a particular feature of a section of the cluster, and loading the appropriate module will restore the module list applicable to that particular system.

4.1.7 Using explicit version numbers

Once a module has been installed on the cluster, the executables or libraries it comprises are never modified. This policy ensures that the user's programs will run consistently, at least if the user specifies a specific version. **Failing to specify a version may result in unexpected behaviour**.

Consider the following example: the user decides to use the `example` module and at that point in time, just a single version 1.2.3 is installed on the cluster. The user loads the module using:

```
$ module load example
```

rather than

```
$ module load example/1.2.3
```

Everything works fine, up to the point where a new version of `example` is installed, 4.5.6. From then on, the user's `load` command will load the latter version, rather than the intended one, which may lead to unexpected problems.

Consider the following `example` modules:

```
$ module avail example/
example/1.2.3
example/4.5.6
```

Let's now generate a version conflict with the `example` module, and see what happens.

```
$ module av example/
example/1.2.3              example/4.5.6
$ module load example/1.2.3 example/4.5.6
example/4.5.6(12):ERROR:150: Module 'example/4.5.6' conflicts with the currently
    loaded module(s) 'example/1.2.3'
example/4.5.6(12):ERROR:102: Tcl command execution failed: conflict example
$ module swap example/4.5.6
```

Note: A `module swap` command combines the appropriate `module unload` and `module load` commands.

4.1.8 Get detailed info

To get a list of all possible commands, type:

```
$ module help
```

Or to get more information about one specific module package:

```
$ module help example/1.2.3
----------- Module Specific Help for 'example/1.2.3' ---------------------------
  This is just an example - Homepage: https://example.com/
```

To remove a collection, remove the corresponding file in `$HOME/.lmod.d`:

```
$ rm $HOME/.lmod.d/my-collection
```

4.1.9 Getting module details

To see how a module would change the environment, you can use the `module show` command:

```
$ module show Python/3.2.5-foss-2014a
module-whatis    Description: Python is a programming language that lets you work
    more quickly and integrate your systems more effectively. - Homepage: http://
    python.org/
conflict         Python
module           load foss/2014a
module           load bzip2/1.0.6-foss-2014a
...
prepend-path ...
...
setenv           EBVERSIONPYTHON 3.2.5
setenv           EBEXTSLISTPYTHON distribute-0.6.26,pip-1.1,nose-1.1.2,numpy-1.6.1,
    scipy-0.10.1
```

Here you can see that the `Python/3.2.5-foss-2014a` comes with a whole bunch of extensions: `numpy`, `scipy`, ...

You can also see the modules the `Python/3.2.5-foss-2014a` module loads: `foss/2014a`, `bzip2/1.0.6-foss-2014a`, ...If you're not sure what all of this means: don't worry, you don't have to know; just load the module and try to use the software.

4.2 Getting system information about the HPC infrastructure

4.2.1 Checking the general status of the HPC infrastructure

To check how much jobs are running in what queues, you can use the `qstat -q` command:

```
$ qstat -q
Queue            Memory CPU Time Walltime Node  Run Que Lm  State
---------------- ------ -------- -------- ----  --- --- --  -----
default            --      --       --      --    0   0 --   E R
q72h               --      --    72:00:00   --    0   0 --   E R
long               --      --    72:00:00   --  316  77 --   E R
short              --      --    11:59:59   --   21   4 --   E R
q1h                --      --    01:00:00   --    0   1 --   E R
q24h               --      --    24:00:00   --    0   0 --   E R
                                               ----- -----
                                                337  82
```

Here, there are 316 jobs running on the `long` queue, and 77 jobs queued. We can also see that the `long` queue allows a maximum wall time of 72 hours.

4.3 Defining and submitting your job

Usually, you will want to have your program running in batch mode, as opposed to interactively as you may be accustomed to. The point is that the program must be able to start and run without user intervention, i.e., without you having to enter any information or to press any buttons during program execution. All the necessary input or required options have to be specified on the command line, or needs to be put in input or configuration files.

As an example, we will run a Perl script, which you will find in the examples subdirectory on

the UAntwerpen-HPC. When you received an account to the UAntwerpen-HPC a subdirectory with examples was automatically generated for you.

Remember that you have copied the contents of the HPC examples directory to your home directory, so that you have your **own personal** copy (editable and over-writable) and that you can start using the examples. If you haven't done so already, run these commands now:

```
$ cd
$ cp -r /apps/antwerpen/tutorials/Intro-HPC/examples ~/
```

First go to the directory with the first examples by entering the command:

```
$ cd ~/examples/Running-batch-jobs
```

Each time you want to execute a program on the UAntwerpen-HPC you'll need 2 things:

The executable The program to execute from the end-user, together with its peripheral input files, databases and/or command options.

A batch job script , which will define the computer resource requirements of the program, the required additional software packages and which will start the actual executable. The UAntwerpen-HPC needs to know:

1. the type of compute nodes;
2. the number of CPUs;
3. the amount of memory;
4. the expected duration of the execution time (wall time: Time as measured by a clock on the wall);
5. the name of the files which will contain the output (i.e., stdout) and error (i.e., stderr) messages;
6. what executable to start, and its arguments.

Later on, the UAntwerpen-HPC user shall have to define (or to adapt) his/her own job scripts. For now, all required job scripts for the exercises are provided for you in the examples subdirectories.

List and check the contents with:

```
$ ls -l
total 512
-rw-r--r-- 1 vsc20167 193 Sep 11 10:34 fibo.pbs
-rw-r--r-- 1 vsc20167 609 Sep 11 10:25 fibo.pl
```

In this directory you find a Perl script (named "fibo.pl") and a job script (named "fibo.pbs").

1. The Perl script calculates the first 30 Fibonacci numbers.
2. The job script is actually a standard Unix/Linux shell script that contains a few extra comments at the beginning that specify directives to PBS. These comments all begin with #**PBS**.

We will first execute the program locally (i.e., on your current login-node), so that you can see what the program does.

On the command line, you would run this using:

```
$ ./fibo.pl
[0] -> 0
[1] -> 1
[2] -> 1
[3] -> 2
[4] -> 3
[5] -> 5
[6] -> 8
[7] -> 13
[8] -> 21
[9] -> 34
[10] -> 55
[11] -> 89
[12] -> 144
[13] -> 233
[14] -> 377
[15] -> 610
[16] -> 987
[17] -> 1597
[18] -> 2584
[19] -> 4181
[20] -> 6765
[21] -> 10946
[22] -> 17711
[23] -> 28657
[24] -> 46368
[25] -> 75025
[26] -> 121393
[27] -> 196418
[28] -> 317811
[29] -> 514229
```

Remark: Recall that you have now executed the Perl script locally on one of the login-nodes of the UAntwerpen-HPC cluster. Of course, this is not our final intention; we want to run the script on any of the compute nodes. Also, it is not considered as good practice, if you "abuse" the login-nodes for testing your scripts and executables. It will be explained later on how you can reserve your own compute-node (by opening an interactive session) to test your software. But for the sake of acquiring a good understanding of what is happening, you are pardoned for this example since these jobs require very little computing power.

The job script contains a description of the job by specifying the command that need to be executed on the compute node:

— fibo.pbs —

```
1 #!/bin/bash -l
2 cd $PBS_O_WORKDIR
3 ./fibo.pl
```

So, jobs are submitted as scripts (bash, Perl, Python, etc.), which specify the parameters related

to the jobs such as expected runtime (walltime), e-mail notification, etc. These parameters can also be specified on the command line.

This job script that can now be submitted to the cluster's job system for execution, using the qsub (Queue SUBmit) command:

```
$ qsub fibo.pbs
433253.leibniz
```

The qsub command returns a job identifier on the HPC cluster. The important part is the number (e.g., "433253"); this is a unique identifier for the job and can be used to monitor and manage your job.

Remark: the modules that were loaded when you submitted the job will *not* be loaded when the job is started. You should always specify the `module load` statements that are required for your job in the job script itself.

Your job is now waiting in the queue for a free workernode to start on.

Go and drink some coffee ... but not too long. If you get impatient you can start reading the next section for more information on how to monitor jobs in the queue.

After your job was started, and ended, check the contents of the directory:

```
$ ls -l
total 768
-rw-r--r-- 1 vsc20167 vsc20167   44 Feb 28 13:33 fibo.pbs
-rw------- 1 vsc20167 vsc20167    0 Feb 28 13:33 fibo.pbs.e433253
-rw------- 1 vsc20167 vsc20167 1010 Feb 28 13:33 fibo.pbs.o433253
-rwxrwxr-x 1 vsc20167 vsc20167  302 Feb 28 13:32 fibo.pl
```

Explore the contents of the 2 new files:

```
$ more fibo.pbs.o433253
$ more fibo.pbs.e433253
```

These files are used to store the standard output and error that would otherwise be shown in the terminal window. By default, they have the same name as that of the PBS script, i.e., "fibo.pbs" as base name, followed by the extension ".o" (output) and ".e" (error), respectively, and the job number ('433253' for this example). The error file will be empty, at least if all went well. If not, it may contain valuable information to determine and remedy the problem that prevented a successful run. The standard output file will contain the results of your calculation (here, the output of the Perl script)

4.4 Monitoring and managing your job(s)

Using the job ID that `qsub` returned, there are various ways to monitor the status of your job. In the following commands, replace `12345` with the job ID `qsub` returned.

```
$ qstat 12345
```

To show an estimated start time for your job (note that this may be very inaccurate, the margin of error on this figure can be bigger then 100% due to a sample in a population of 1.) This

command is not available on all systems.

```
$ showstart 12345
```

This is only a very rough estimate. Jobs may launch sooner than estimated if other jobs end faster than estimated, but may also be delayed if other higher-priority jobs enter the system.

To show the status, but also the resources required by the job, with error messages that may prevent your job from starting:

```
$ checkjob 12345
```

To show on which compute nodes your job is running, at least, when it is running:

```
$ qstat -n 12345
```

To remove a job from the queue so that it will not run, or to stop a job that is already running.

```
$ qdel 12345
```

When you have submitted several jobs (or you just forgot about the job ID), you can retrieve the status of all your jobs that are submitted and are not yet finished using:

```
$ qstat
leibniz :
Job ID      Name    User      Time Use S Queue
----------- ------- --------- -------- - -----
433253.... mpi     vsc20167 0        Q short
```

Here:

Job ID the job's unique identifier

Name the name of the job

User the user that owns the job

Time Use the elapsed walltime for the job

Queue the queue the job is in

The state S can be any of the following:

State	Meaning
Q	The job is **queued** and is waiting to start.
R	The job is currently **running**.
E	The job is currently **exiting** after having run.
C	The job is **completed** after having run.
H	The job has a user or system **hold** on it and will not be eligible to run until the hold is removed.

User hold means that the user can remove the hold. System hold means that the system or an administrator has put the job on hold, very likely because something is wrong with it. Check with your helpdesk to see why this is the case.

4.5 Examining the queue

As we learned above, Moab is the software application that actually decides when to run your job and what resources your job will run on.

You can look at the queue by using the PBS **qstat** command or the Moab **showq** command. By default, **qstat** will display the queue ordered by **JobID**, whereas **showq** will display jobs grouped by their state ("running", "idle", or "hold") then ordered by priority. Therefore, **showq** is often more useful. Note however that at some VSC-sites, these commands show only your jobs or may be even disabled to not reveal what other users are doing.

The **showq** command displays information about active ("running"), eligible ("idle"), blocked ("hold"), and/or recently completed jobs. To get a summary:

```
$ showq -s
active jobs: 163
eligible jobs: 133
blocked jobs: 243
Total jobs:  539
```

And to get the full detail of all the jobs, which are in the system:

```
$ showq
active jobs------------------------
JOBID     USERNAME  STATE PROCS REMAINING          STARTTIME
428024    vsc20167  Running   8   2:57:32  Mon Sep  2 14:55:05
...
153 active jobs 1307 of 3360 processors in use by local jobs (38.90%)
153 of 168 nodes active      (91.07%)

eligible jobs----------------------
JOBID     USERNAME  STATE PROCS   WCLIMIT            QUEUETIME
442604    vsc20167   Idle  48  7:00:00:00  Sun Sep 22 16:39:13
442605    vsc20167   Idle  48  7:00:00:00  Sun Sep 22 16:46:22
...

135 eligible jobs

blocked jobs-----------------------
JOBID   USERNAME     STATE PROCS WCLIMIT            QUEUETIME
441237  vsc20167      Idle   8 3:00:00:00 Thu Sep 19 15:53:10
442536  vsc20167  UserHold  40 3:00:00:00 Sun Sep 22 00:14:22
...
252 blocked jobs
Total jobs:  540
```

There are 3 categories, the **active**, **eligible** and **blocked** jobs.

Active jobs are jobs that are running or starting and that consume computer resources. The amount of time remaining (w.r.t. walltime, sorted to earliest completion time) and the start time are displayed. This will give you an idea about the foreseen completion time. These jobs could be in a number of states:

Started attempting to start, performing pre-start tasks

Running currently executing the user application

Suspended has been suspended by scheduler or admin (still in place on the allocated resources, not executing)

Cancelling has been cancelled, in process of cleaning up

Eligible jobs are jobs that are waiting in the queues and are considered eligible for both scheduling and backfilling. They are all in the idle job state and do not violate any fairness policies or do not have any job holds in place. The requested walltime is displayed, and the list is ordered by job priority.

Blocked jobs are jobs that are ineligible to be run or queued. These jobs could be in a number of states for the following reasons:

Idle when the job violates a fairness policy

Userhold or systemhold when it is user or administrative hold

Batchhold when the requested resources are not available or the resource manager has repeatedly failed to start the job

Deferred when a temporary hold when the job has been unable to start after a specified number of attempts

Notqueued when scheduling daemon is unavailable

4.6 Specifying job requirements

Without giving more information about your job upon submitting it with **qsub**, default values will be assumed that are almost never appropriate for real jobs.

It is important to estimate the resources you need to successfully run your program, such as the amount of time the job will require, the amount of memory it needs, the number of CPUs it will run on, etc. This may take some work, but it is necessary to ensure your jobs will run properly.

4.6.1 Generic resource requirements

The **qsub** command takes several options to specify the requirements, of which we list the most commonly used ones below.

```
$ qsub -l walltime=2:30:00
```

For the simplest cases, only the amount of maximum estimated execution time (called "walltime") is really important. Here, the job requests 2 hours, 30 minutes. As soon as the job exceeds the requested walltime, it will be "killed" (terminated) by the job scheduler. There is no harm if you *slightly* overestimate the maximum execution time. If you omit this option, the queue manager will not complain but use a default value (one hour on most clusters).

If you want to run some final steps (for example to copy files back) before the walltime kills your main process, you have to kill the main command yourself before the walltime runs out and then copy the file back. See section 15.3 for how to do this.

```
$ qsub -l mem=4gb
```

The job requests 4 GB of RAM memory. As soon as the job tries to use more memory, it will be "killed" (terminated) by the job scheduler. There is no harm if you *slightly* overestimate the requested memory.

```
$ qsub -l nodes=5:ppn=2
```

The job requests 5 compute nodes with two cores on each node (ppn stands for "processors per node", where "processors" here actually means "CPU cores").

```
$ qsub -l nodes=1:westmere
```

The job requests just one node, but it should have an Intel Westmere processor. A list with site-specific properties can be found in the next section or in the User Portal ("VSC hardware" section)[1] of the VSC website.

These options can either be specified on the command line, e.g.

```
$ qsub -l nodes=1:ppn=1,mem=2gb fibo.pbs
```

or in the job script itself using the #PBS-directive, so "fibo.pbs" could be modified to:

```
1 #!/bin/bash -l
2 #PBS -l nodes=1:ppn=1
3 #PBS -l mem=2gb
4 cd $PBS_O_WORKDIR
5 ./fibo.pl
```

Note that the resources requested on the command line will override those specified in the PBS file.

4.6.2 Node-specific properties

The following table contains some node-specific properties that can be used to make sure the job will run on nodes with a specific CPU or interconnect. Note that these properties may vary over the different VSC sites.

Property	Explanation
ivybridge	only use Intel processors from the Ivy Bridge family (26xx-v2, hopper-only)
broadwell	only use Intel processors from the Broadwell family (26xx-v4, leibniz-only)
mem128	only use nodes with 128GB of RAM (leibniz)
mem256	only use nodes with 256GB of RAM (hopper and leibniz)
tesla, gpu	only use nodes with the NVIDUA P100 GPU (leibniz)

Since both hopper and leibniz are homogeneous with respect to processor architecture, the CPU architecture properties are not really needed and only defined for compatibility with other VSC clusters.

[1] URL: https://vscdocumentation.readthedocs.io/en/latest/hardware.html

To get a list of all properties defined for all nodes, enter

```
$ pbsnodes
```

This list will also contain properties referring to, e.g., network components, rack number, etc.

4.7 Job output and error files

At some point your job finishes, so you may no longer see the job ID in the list of jobs when you run *qstat* (since it will only be listed for a few minutes after completion with state "C"). After your job finishes, you should see the standard output and error of your job in two files, located by default in the directory where you issued the *qsub* command.

When you navigate to that directory and list its contents, you should see them:

```
$ ls -l
total 1024
-rw-r--r-- 1 vsc20167  609 Sep 11 10:54 fibo.pl
-rw-r--r-- 1 vsc20167   68 Sep 11 10:53 fibo.pbs
-rw------- 1 vsc20167   52 Sep 11 11:03 fibo.pbs.e433253
-rw------- 1 vsc20167 1307 Sep 11 11:03 fibo.pbs.o433253
```

In our case, our job has created both output ('fibo.pbs.**o**433253') and error files ('fibo.pbs.**e**433253') containing info written to *stdout* and *stderr* respectively.

Inspect the generated output and error files:

```
$ cat fibo.pbs.o433253
...
$ cat fibo.pbs.e433253
...
```

4.8 E-mail notifications

4.8.1 Upon job failure

Whenever a job fails, an e-mail will be sent to the e-mail address that's connected to your VSC account. This is the e-mail address that is linked to the university account, which was used during the registration process.

You can force a job to fail by specifying an unrealistic wall-time for the previous example. Lets give the "*fibo.pbs*" job just one second to complete:

```
$ qsub -l walltime=0:00:01 fibo.pbs
```

Now, lets hope that the UAntwerpen-HPC did not manage to run the job within one second, and you will get an e-mail informing you about this error.

```
PBS Job Id: 433253.leibniz
Job Name:   fibo.pbs
Exec host:  r1c02cn3.leibniz.antwerpen.vsc /0
Aborted by PBS Server
Job exceeded some resource limit (walltime, mem, etc.). Job was aborted.
See Administrator for help
```

4.8.2 Generate your own e-mail notifications

You can instruct the UAntwerpen-HPC to send an e-mail to your e-mail address whenever a job **b**egins, **e**nds and/or **a**borts, by adding the following lines to the job script `fibo.pbs`:

```
1 #PBS -m b
2 #PBS -m e
3 #PBS -m a
```

or

```
1 #PBS -m abe
```

These options can also be specified on the command line. Try it and see what happens:

```
$ qsub -m abe fibo.pbs
```

The system will use the e-mail address that is connected to your VSC account. You can also specify an alternate e-mail address with the `-M` option:

```
$ qsub -m b -M john.smith@example.com fibo.pbs
```

will send an e-mail to john.smith@example.com when the job begins.

4.9 Running a job after another job

If you submit two jobs expecting that should be run one after another (for example because the first generates a file the second needs), there might be a problem as they might both be run at the same time.

So the following example might go wrong:

```
$ qsub job1.sh
$ qsub job2.sh
```

You can make jobs that depend on other jobs. This can be useful for breaking up large jobs into smaller jobs that can be run in a pipeline. The following example will submit 2 jobs, but the second job (`job2.sh`) will be held (`H` status in `qstat`) until the first job successfully completes. If the first job fails, the second will be cancelled.

```
$ FIRST_ID=$(qsub job1.sh)
$ qsub -W depend=afterok:$FIRST_ID job2.sh
```

`afterok` means "After OK", or in other words, after the first job successfully completed.

It's also possible to use `afternotok` ("After not OK") to run the second job only if the first job exited with errors. A third option is to use `afterany` ("After any"), to run the second job after the first job (regardless of success or failure).

Chapter 5

Running interactive jobs

5.1 Introduction

Interactive jobs are jobs which give you an interactive session on one of the compute nodes. Importantly, accessing the compute nodes this way means that the job control system guarantees the resources that you have asked for.

Interactive PBS jobs are similar to non-interactive PBS jobs in that they are submitted to PBS via the command **qsub**. Where an interactive job differs is that it does not require a job script, the required PBS directives can be specified on the command line.

Interactive jobs can be useful to debug certain job scripts or programs, but should not be the main use of the UAntwerpen-HPC. Waiting for user input takes a very long time in the life of a CPU and does not make efficient usage of the computing resources.

The syntax for *qsub* for submitting an interactive PBS job is:

```
$ qsub -I <...  pbs directives ...>
```

5.2 Interactive jobs, without X support

Tip: Find the code in "~/examples/Running-interactive-jobs"

First of all, in order to know on which computer you're working, enter:

```
$ hostname -f
ln2.leibniz.uantwerpen.vsc
```

This means that you're now working on the login node `ln2.leibniz.uantwerpen.vsc` of the UAntwerpen-HPC cluster.

The most basic way to start an interactive job is the following:

```
$ qsub -I
qsub: waiting for job 433253.leibniz to start
qsub: job 433253.leibniz ready
```

There are two things of note here.

1. The "*qsub*" command (with the interactive -I flag) waits until a node is assigned to your interactive session, connects to the compute node and shows you the terminal prompt on that node.

2. You'll see that your directory structure of your home directory has remained the same. Your home directory is actually located on a shared storage system. This means that the exact same directory is available on all login nodes and all compute nodes on all clusters.

In order to know on which compute-node you're working, enter again:

```
$ hostname -f
r1c02cn3.leibniz.antwerpen.vsc
```

Note that we are now working on the compute-node called "*r1c02cn3.leibniz.antwerpen.vsc*". This is the compute node, which was assigned to us by the scheduler after issuing the "*qsub -I*" command.

This computer name looks strange, but bears some logic in it. It provides the system administrators with information where to find the computer in the computer room.

The computer "*r1c2cn03*" stands for:

1. "r5" is rack #5.

2. "c3" is enclosure/chassis #3.

3. "cn08" is compute node #08.

With this naming convention, the system administrator can easily find the physical computers when they need to execute some maintenance activities.

Now, go to the directory of our second interactive example and run the program "primes.py". This program will ask you for an upper limit (> 1) and will print all the primes between 1 and your upper limit:

```
$ cd ~/examples/Running-interactive-jobs
$ ./primes.py
This program calculates all primes between 1 and your upper limit.
Enter your upper limit (>1): 50
Start Time:  2013-09-11 15:49:06
[Prime#1] = 1
[Prime#2] = 2
[Prime#3] = 3
[Prime#4] = 5
[Prime#5] = 7
[Prime#6] = 11
[Prime#7] = 13
[Prime#8] = 17
[Prime#9] = 19
[Prime#10] = 23
[Prime#11] = 29
[Prime#12] = 31
[Prime#13] = 37
[Prime#14] = 41
[Prime#15] = 43
[Prime#16] = 47
End Time:  2013-09-11 15:49:06
Duration:  0 seconds.
```

You can exit the interactive session with:

```
$ exit
```

Note that you can now use this allocated node for 1 hour. After this hour you will be automatically disconnected. You can change this "usage time" by explicitly specifying a "walltime", i.e., the time that you want to work on this node. (Think of walltime as the time elapsed when watching the clock on the wall.)

You can work for 3 hours by:

```
$ qsub -I -l walltime=03:00:00
```

If the walltime of the job is exceeded, the (interactive) job will be killed and your connection to the compute node will be closed. So do make sure to provide adequate walltime and that you save your data before your (wall)time is up (exceeded)! When you do not specify a walltime, you get a default walltime of 1 hour.

5.3 Interactive jobs, with graphical support

5.3.1 Software Installation

To display graphical applications from a Linux computer (such as the VSC clusters) on your machine, you need to install an X Window server on your local computer.

The X Window system (commonly known as **X11**, based on its current major version being 11, or shortened to simply **X**) is the system-level software infrastructure for the windowing GUI on Linux, BSD and other UNIX-like operating systems. It was designed to handle both local displays, as well as displays sent across a network. More formally, it is a computer software

system and network protocol that provides a basis for graphical user interfaces (GUIs) and rich input device capability for networked computers.

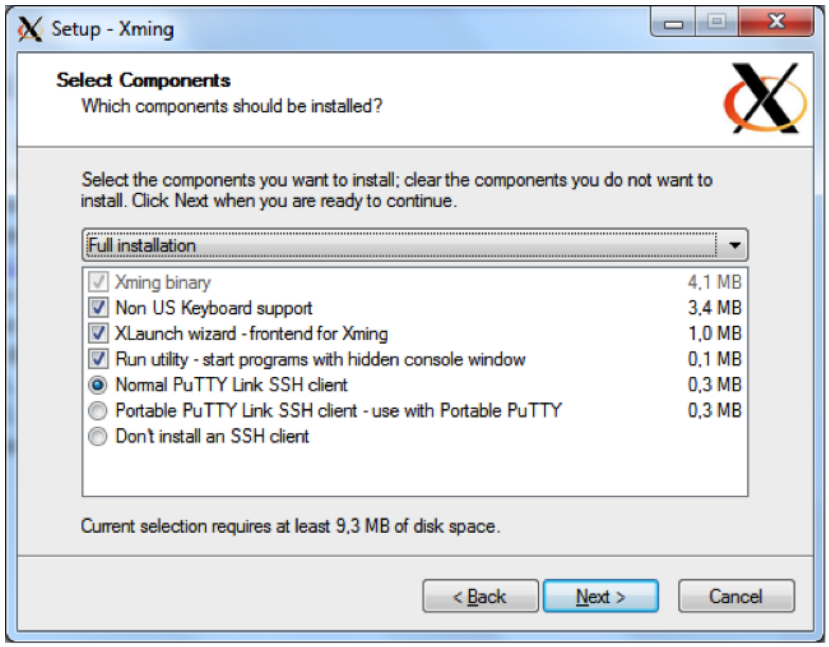
Install Xming The first task is to install the Xming software.

1. Download the Xming installer from the following address: `http://www.straightrunning.com/XmingNotes/`. Either download Xming from the **Public Domain Releases** (free) or from the **Website Releases** (after a donation) on the website.

2. Run the Xming setup program on your Windows desktop.

3. Keep the proposed default folders for the Xming installation.

4. When selecting the components that need to be installed, make sure to select "*XLaunch wizard*" and "*Normal PuTTY Link SSH client*".



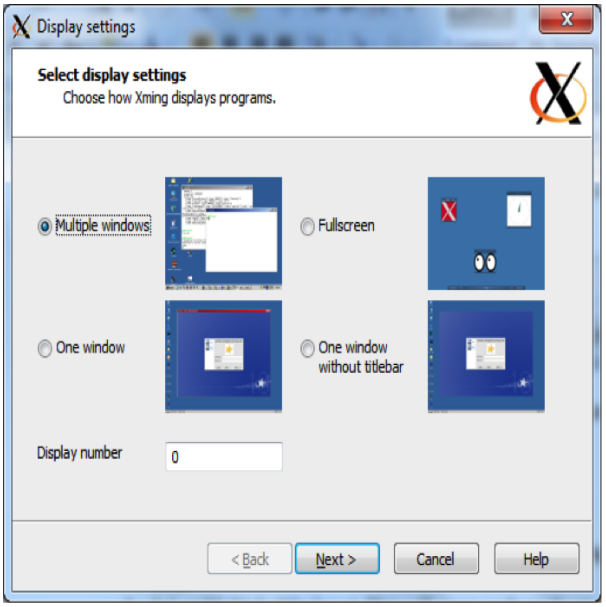
5. We suggest to create a Desktop icon for Xming and XLaunch.

6. And Install.

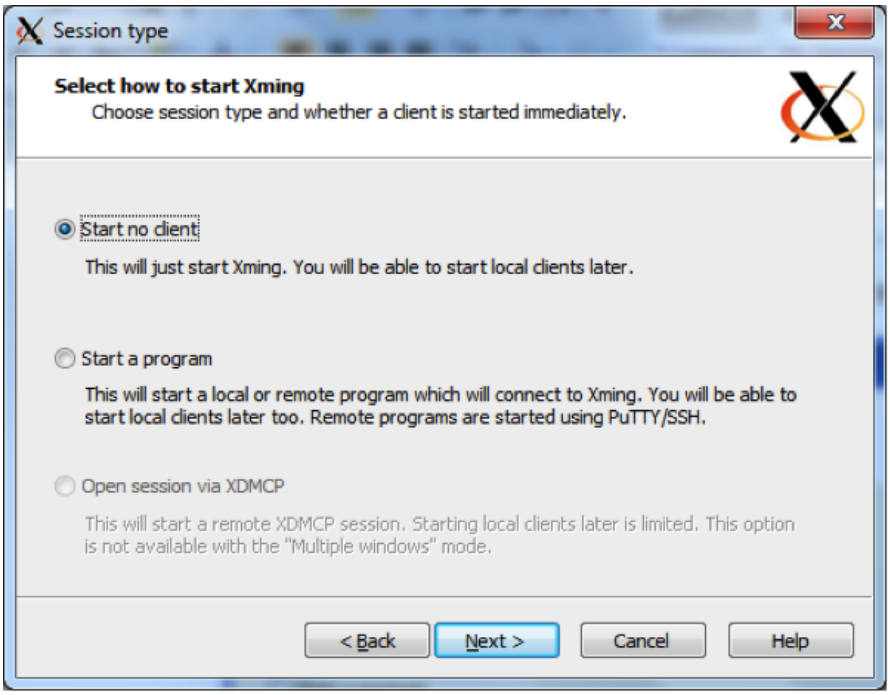
And now we can run Xming:

1. Select XLaunch from the Start Menu or by double-clicking the Desktop icon.

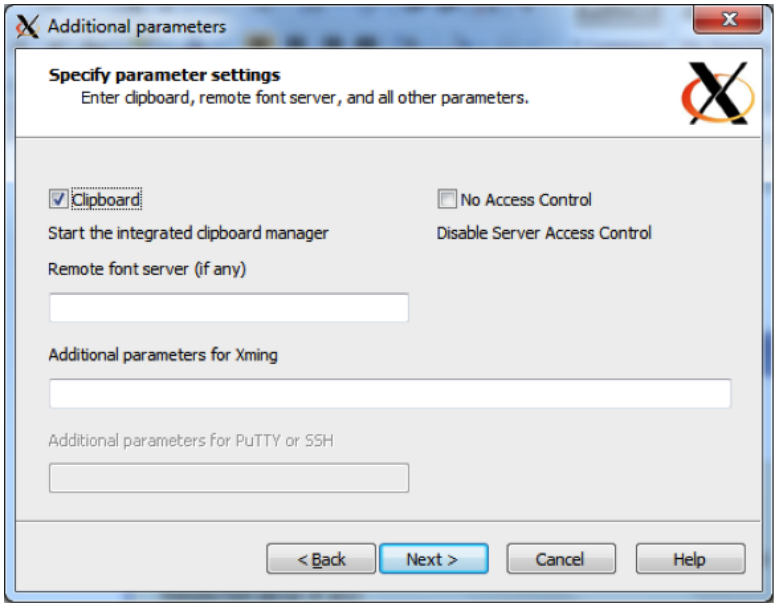
2. Select Multiple Windows. This will open each application in a separate window.



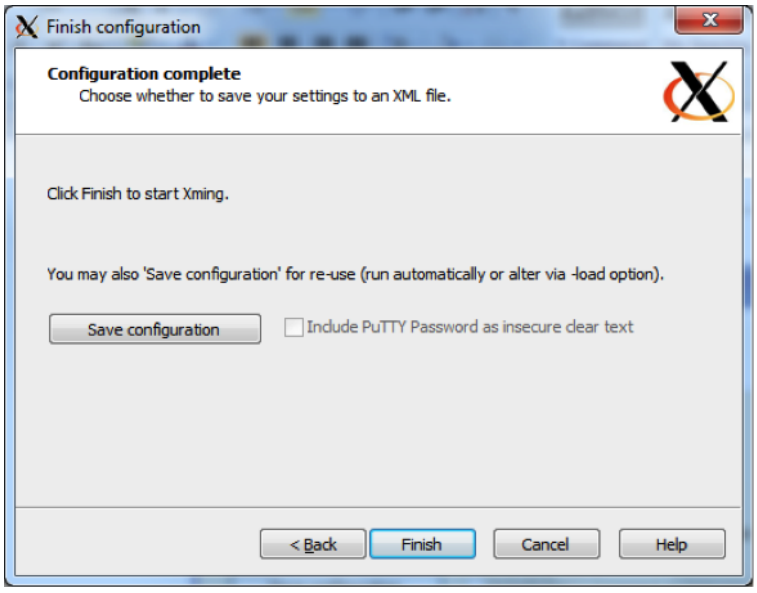
3. Select Start no client to make XLaunch wait for other programs (such as PuTTY).



4. Select Clipboard to share the clipboard.



5. Finally Save configuration into a file. You can keep the default filename and save it in your Xming installation directory.



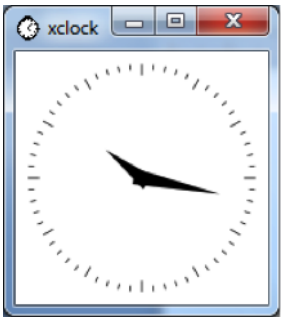
6. Now Xming is running in the background ...
   and you can launch a graphical application in your PuTTY terminal.

7. Open a PuTTY terminal and connect to the HPC.

8. In order to test the X-server, run "*xclock*". "*xclock*" is the standard GUI clock for the X Window System.

```
$ xclock
```

You should see the XWindow clock application appearing on your Windows machine. The "*xclock*" application runs on the login-node of the UAntwerpen-HPC, but is displayed on your Windows machine.



You can close your clock and connect further to a compute node with again your X-forwarding enabled:

```
$ qsub -I -X
qsub: waiting for job 433253.leibniz to start
qsub: job 433253.leibniz ready
$ hostname -f
r1c02cn3.leibniz.antwerpen.vsc
$ xclock
```

and you should see your clock again.

SSH Tunnel In order to work in client/server mode, it is often required to establish an SSH tunnel between your Windows desktop machine and the compute node your job is running on. PuTTY must have been installed on your computer, and you should be able to connect via SSH to the HPC cluster's login node.

Because of one or more firewalls between your desktop and the HPC clusters, it is generally impossible to communicate directly with a process on the cluster from your desktop except when the network managers have given you explicit permission (which for security reasons is not often done). One way to work around this limitation is SSH tunnelling.

There are several cases where this is useful:

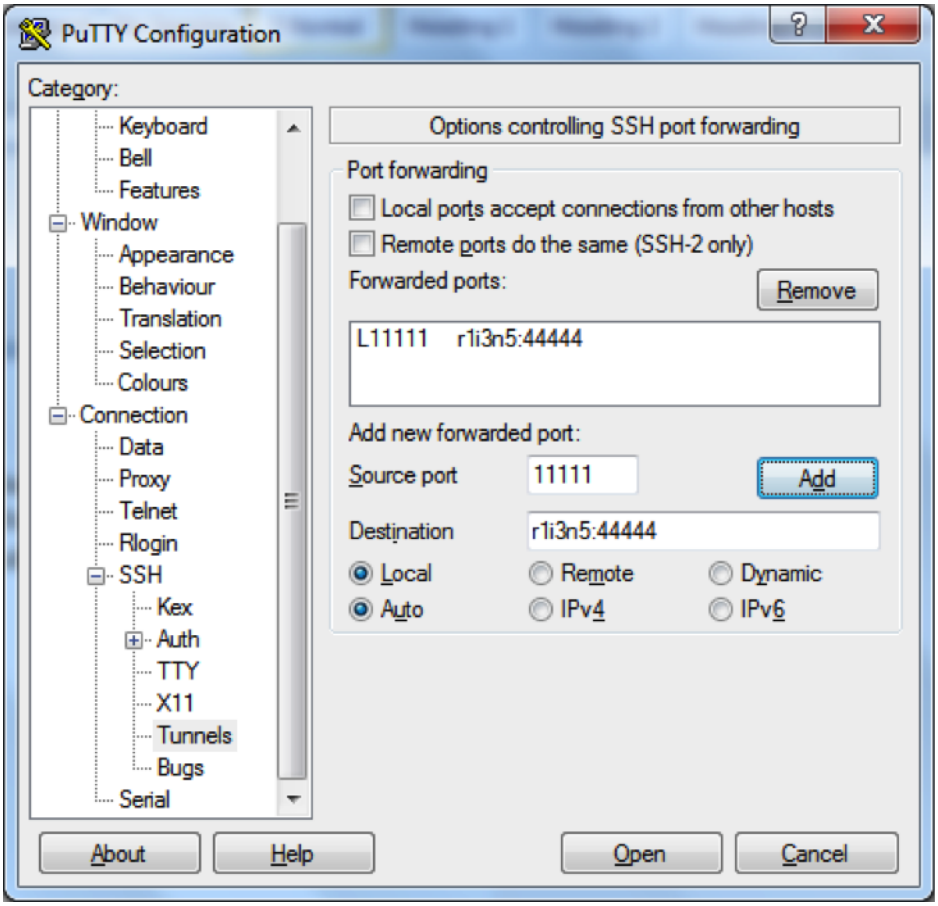
1. Running graphical applications on the cluster: The graphical program cannot directly communicate with the X Window server on your local system. In this case, the tunnelling is easy to set up as PuTTY will do it for you if you select the right options on the X11 settings page as explained on the page about text-mode access using PuTTY.

2. Running a server application on the cluster that a client on the desktop connects to. One example of this scenario is ParaView in remote visualisation mode, with the interactive client on the desktop and the data processing and image rendering on the cluster. This scenario is explained on this page.

3. Running clients on the cluster and a server on your desktop. In this case, the source port is a port on the cluster and the destination port is on the desktop.

Procedure: A tunnel from a local client to a specific computer node on the cluster

1. Log in on the login node via PuTTY.
2. Start the server job, note the compute node's name the job is running on (e.g., r1c02cn3.leibniz.antwerpen.vsc) as well as the port the server is listening on (e.g., "54321").
3. Set up the tunnel:
   (a) Close your current PuTTY session.
   (b) In the "*Category*" pane, expand Connection » SSH, and select Tunnels as show below:



   (c) In the Source port field, enter the local port to use (e.g., *5555*).
   (d) In the Destination field, enter *<hostname>:<server-port>* (e.g., r1c02cn3.leibniz.antwerpen.vsc:54321 as in the example above, these are the details you noted in the second step).
   (e) Click the Add button.
   (f) Click the Open button

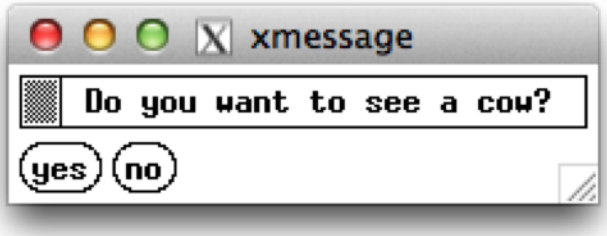
The tunnel is now ready to use.

5.3.2 Run simple example

We have developed a little interactive program that shows the communication in 2 directions. It will send information to your local screen, but also asks you to click a button.

```
$ cd ~/examples/Running-interactive-jobs
$ ./message.py
```

You should see the following message appearing.



Click any button and see what happens.

```
-----------------------
< Enjoy the day! Mooh >
-----------------------
     ^__^
     (oo)\_______
     (__)\       )\/\
         ||----w |
         ||     ||
```

5.3.3 Run your interactive application

In this last example, we will show you that you can just work on this compute node, just as if you were working locally on your desktop. We will run the Fibonacci example of the previous chapter again, but now in full interactive mode in MATLAB.

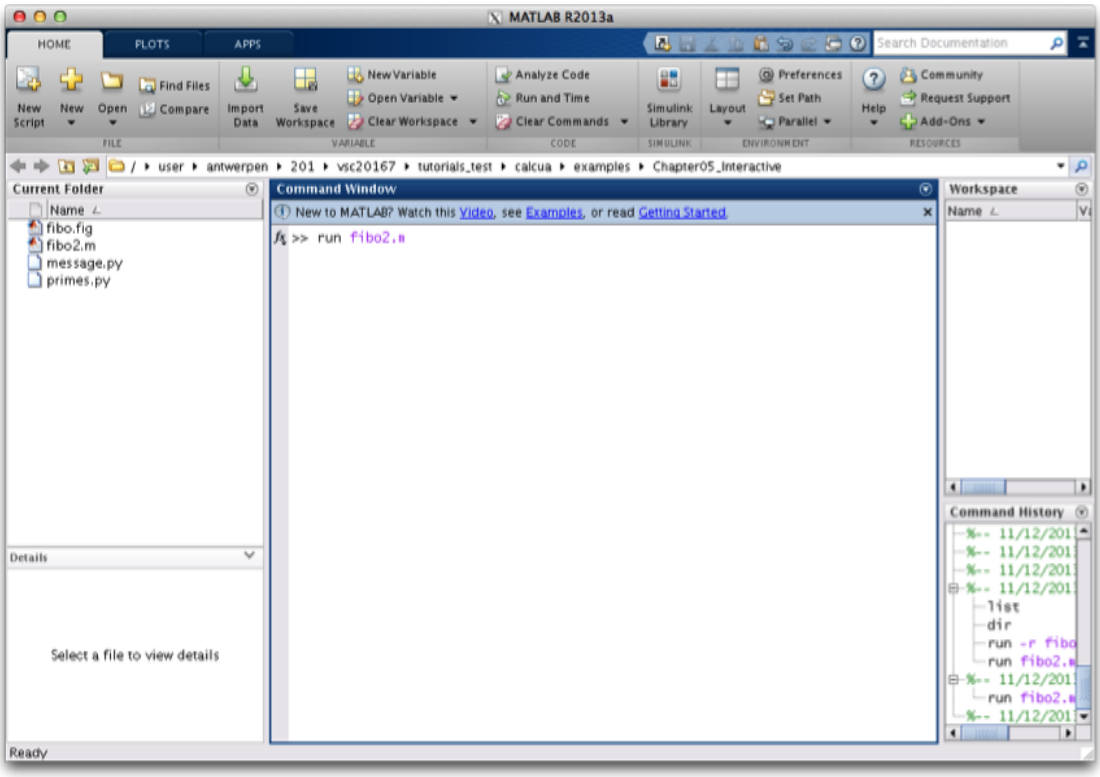
```
$ module load MATLAB
```

And start the MATLAB interactive environment:

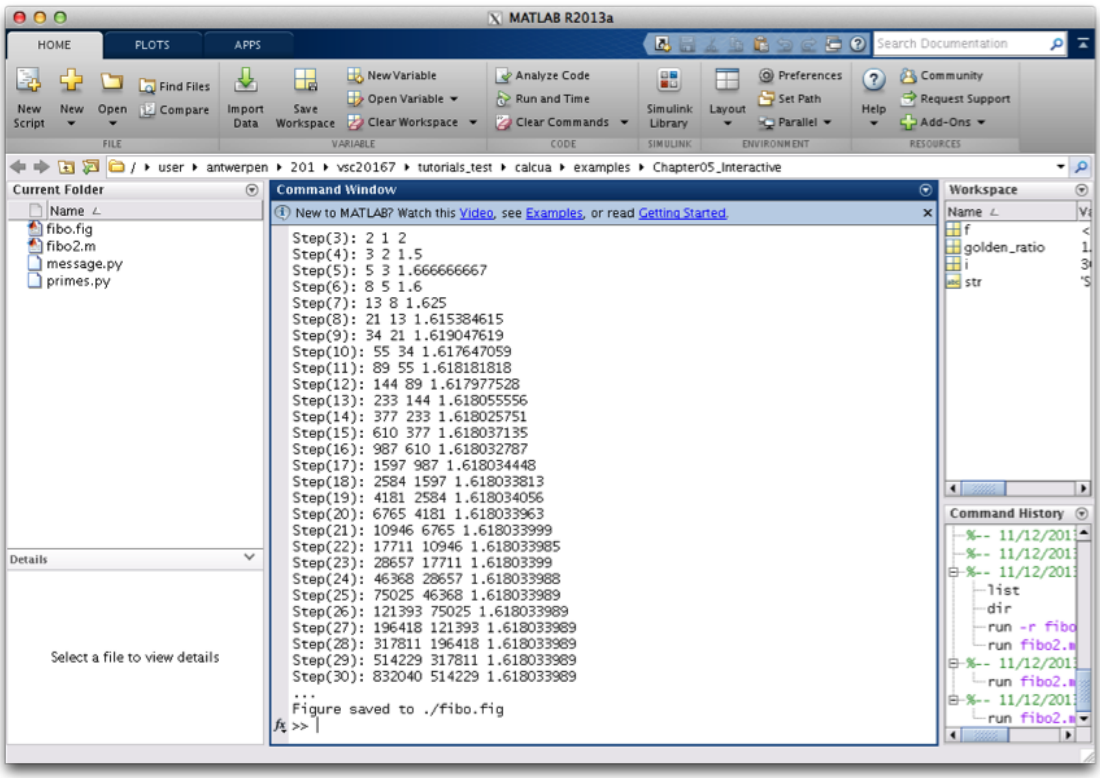
```
$ matlab
```

And start the fibo2.m program in the command window:

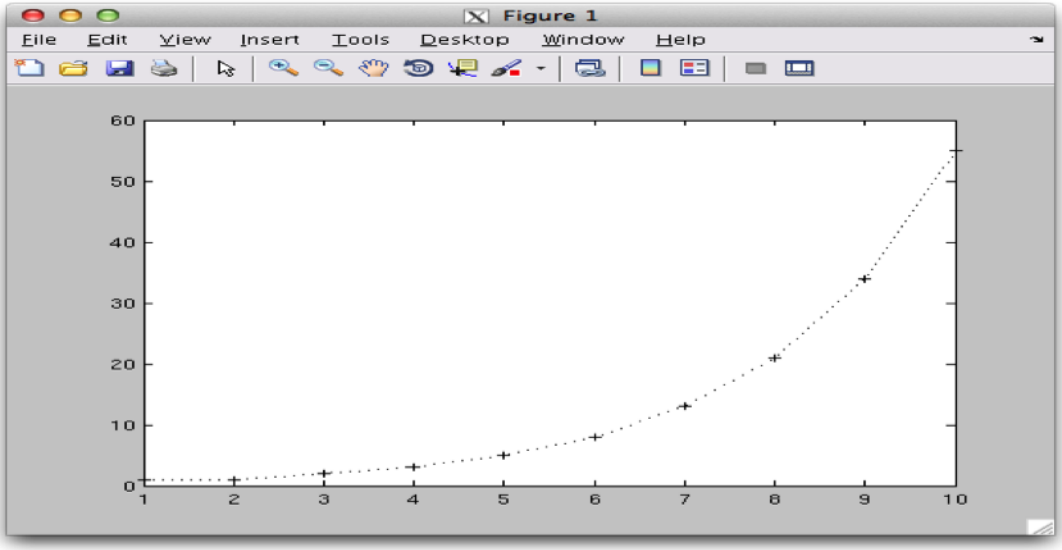
```
fx >> run fibo2.m
```



And see the displayed calculations, ...



as well as the nice "plot" appearing:



You can work in this MATLAB GUI, and finally terminate the application by entering "**exit**" in the command window again.

```
fx >> exit
```

Chapter 6

Running jobs with input/output data

You have now learned how to start a batch job and how to start an interactive session. The next question is how to deal with input and output files, where your standard output and error messages will go to and where that you can collect your results.

6.1 The current directory and output and error files

6.1.1 Default file names

First go to the directory:

```
$ cd ~/examples/Running-jobs-with-input-output-data
```

List and check the contents with:

```
$ ls -l
total 2304
-rwxrwxr-x 1 vsc20167  682 Sep 13 11:34 file1.py
-rw-rw-r-- 1 vsc20167  212 Sep 13 11:54 file1a.pbs
-rw-rw-r-- 1 vsc20167  994 Sep 13 11:53 file1b.pbs
-rw-rw-r-- 1 vsc20167  994 Sep 13 11:53 file1c.pbs
-rw-r--r-- 1 vsc20167 1393 Sep 13 10:41 file2.pbs
-rwxrwxr-x 1 vsc20167 2393 Sep 13 10:40 file2.py
-rw-r--r-- 1 vsc20167 1393 Sep 13 10:41 file3.pbs
-rwxrwxr-x 1 vsc20167 2393 Sep 13 10:40 file3.py
```

Now, let us inspect the contents of the first executable (which is just a Python script with execute permission).

— file1.py —

```
#!/usr/bin/env python
#
# VSC        : Flemish Supercomputing Centre
# Tutorial   : Introduction to HPC
# Description: Writing to the current directory, stdout and stderr
#
import sys

# Step #1: write to a local file in your current directory
local_f = open("Hello.txt", 'w+')
local_f.write("Hello World!\n")
local_f.write("I am writing in the file:<Hello.txt>.\n")
local_f.write("in the current directory.\n")
local_f.write("Cheers!\n")
local_f.close()

# Step #2: Write to stdout
sys.stdout.write("Hello World!\n")
sys.stdout.write("I am writing to <stdout>.\n")
sys.stdout.write("Cheers!\n")

# Step #3: Write to stderr
sys.stderr.write("Hello World!\n")
sys.stderr.write("This is NO ERROR or WARNING.\n")
sys.stderr.write("I am just writing to <stderr>.\n")
sys.stderr.write("Cheers!\n")
```

The code of the Python script, is self explanatory:

1. In step 1, we write something to the file `hello.txt` in the current directory.
2. In step 2, we write some text to stdout.
3. In step 3, we write to stderr.

Check the contents of the first job script:

— file1a.pbs —

```
#!/bin/bash

#PBS -l walltime=00:05:00

# go to the (current) working directory (optional, if this is the
# directory where you submitted the job)
cd $PBS_O_WORKDIR

# the program itself
echo Start Job
date
./file1.py
echo End Job
```

You'll see that there are NO specific PBS directives for the placement of the output files. All output files are just written to the standard paths.

Submit it:

```
$ qsub file1a.pbs
```

After the job has finished, inspect the local directory again, i.e., the directory where you executed the *qsub* command:

```
$ ls -l
total 3072
-rw-rw-r-- 1 vsc20167   90 Sep 13 13:13 Hello.txt
-rwxrwxr-x 1 vsc20167  693 Sep 13 13:03 file1.py*
-rw-rw-r-- 1 vsc20167  229 Sep 13 13:01 file1a.pbs
-rw------- 1 vsc20167   91 Sep 13 13:13 file1a.pbs.e433253
-rw------- 1 vsc20167  105 Sep 13 13:13 file1a.pbs.o433253
-rw-rw-r-- 1 vsc20167  143 Sep 13 13:07 file1b.pbs
-rw-rw-r-- 1 vsc20167  177 Sep 13 13:06 file1c.pbs
-rw-r--r-- 1 vsc20167 1393 Sep 13 10:41 file2.pbs
-rwxrwxr-x 1 vsc20167 2393 Sep 13 10:40 file2.py*
-rw-r--r-- 1 vsc20167 1393 Sep 13 10:41 file3.pbs
-rwxrwxr-x 1 vsc20167 2393 Sep 13 10:40 file3.py*
```

Some observations:

1. The file `Hello.txt` was created in the current directory.

2. The file `file1a.pbs.o433253` contains all the text that was written to the standard output stream ("stdout").

3. The file `file1a.pbs.e433253` contains all the text that was written to the standard error stream ("stderr").

Inspect their contents . . . and remove the files

```
$ cat Hello.txt
$ cat file1a.pbs.o433253
$ cat file1a.pbs.e433253
$ rm Hello.txt file1a.pbs.o433253 file1a.pbs.e433253
```

Tip: Type `cat H` and press the Tab button (looks like ⇥), and it will **expand** into `cat Hello.txt`.

6.1.2 Filenames using the name of the job

Check the contents of the job script and execute it.

— file1b.pbs —

```
#!/bin/bash

#   Specify the "name" of the job
#PBS -N my_serial_job

cd $PBS_O_WORKDIR
echo Start Job
date
./file1.py
echo End Job
```

```
$ qsub file1b.pbs
```

Inspect the contents again ... and remove the generated files:

```
$ ls
Hello.txt  file1a.pbs  file1c.pbs  file2.pbs  file3.pbs  my_serial_job.e433253
file1.py*  file1b.pbs  file2.py*   file3.py*  my_serial_job.o433253
$ rm Hello.txt my_serial_job.*
```

Here, the option "`-N`" was used to explicitly assign a name to the job. This overwrote the JOBNAME variable, and resulted in a different name for the *stdout* and *stderr* files. This name is also shown in the second column of the "`qstat`" command. If no name is provided, it defaults to the name of the job script.

6.1.3 User-defined file names

You can also specify the name of *stdout* and *stderr* files explicitly by adding two lines in the job script, as in our third example:

— file1c.pbs —

```
#!/bin/bash

# redirect standard output (-o) and error (-e)
#PBS -o stdout.$PBS_JOBID
#PBS -e stderr.$PBS_JOBID

cd $PBS_O_WORKDIR
echo Start Job
date
./file1.py
echo End Job
```

```
$ qsub file1c.pbs
$ ls
```

6.2 Where to store your data on the UAntwerpen-HPC

The UAntwerpen-HPC cluster offers their users several locations to store their data. Most of the data will reside on the shared storage system, but all compute nodes also have their own (small) local disk.

6.2.1 Pre-defined user directories

Three different pre-defined user directories are available, where each directory has been created for different purposes. The best place to store your data depends on the purpose, but also the size and type of usage of the data.

The following locations are available:

Variable	Description
Long-term storage slow filesystem, intended for smaller files	
$VSC_HOME	For your configuration files and other small files, see §6.2.2. The default directory is `/user/antwerpen/xxx/vsc20167`. The same file system is accessible from all sites, i.e., you'll see the same contents in $VSC_HOME on all sites.
$VSC_DATA	A bigger "workspace", for **datasets**, results, logfiles, etc. see §6.2.3. The default directory is `/data/antwerpen/xxx/vsc20167`. The same file system is accessible from all sites.
Fast temporary storage	
$VSC_SCRATCH_NODE	For **temporary** or transient data on the local compute node, where fast access is important; see §6.2.4. This space is available per node. The default directory is `/tmp`. On different nodes, you'll see different content.
$VSC_SCRATCH	For **temporary** or transient data that has to be accessible from all nodes of a cluster (including the login nodes) The default directory is `/scratch/antwerpen/xxx/vsc20167`. This directory is cluster- or site-specific: On different sites, and sometimes on different clusters on the same site, you'll get a different directory with different content.
$VSC_SCRATCH_SITE	Currently the same as $VSC_SCRATCH, but could be used for a scratch space shared across all clusters at a site in the future. See §6.2.4.
$VSC_SCRATCH_GLOBAL	Currently the same as $VSC_SCRATCH, but could be used for a scratch space shared across all clusters of the VSC in the future. See §6.2.4.

Since these directories are not necessarily mounted on the same locations over all sites, you should always (try to) use the environment variables that have been created.

We elaborate more on the specific function of these locations in the following sections.

6.2.2 Your home directory ($VSC_HOME)

Your home directory is where you arrive by default when you login to the cluster. Your shell refers to it as "~" (tilde), and its absolute path is also stored in the environment variable $VSC_HOME. Your home directory is shared across all clusters of the VSC.

The data stored here should be relatively small (e.g., no files or directories larger than a few megabytes), and preferably should only contain configuration files. Note that various kinds of configuration files are also stored here, e.g., by MATLAB, Eclipse, ...

The operating system also creates a few files and folders here to manage your account. Examples are:

File or Directory	Description
.ssh/	This directory contains some files necessary for you to login to the cluster and to submit jobs on the cluster. Do not remove them, and do not alter anything if you don't know what you are doing!
.bash_profile	When you login (type username and password) remotely via ssh, .bash_profile is executed to configure your shell before the initial command prompt.
.bashrc	This script is executed every time you start a session on the cluster: when you login to the cluster and when a job starts.
.bash_history	This file contains the commands you typed at your shell prompt, in case you need them again.

Furthermore, we have initially created some files/directories there (tutorial, docs, examples, examples.pbs) that accompany this manual and allow you to easily execute the provided examples.

6.2.3 Your data directory ($VSC_DATA)

In this directory you can store all other data that you need for longer terms (such as the results of previous jobs, ...). It is a good place for, e.g., storing big files like genome data.

The environment variable pointing to this directory is $VSC_DATA. This volume is shared across all clusters of the VSC. There are however no guarantees about the speed you will achieve on this volume. For guaranteed fast performance and very heavy I/O, you should use the scratch space instead.

6.2.4 Your scratch space ($VSC_SCRATCH)

To enable quick writing from your job, a few extra file systems are available on the compute nodes. These extra file systems are called scratch folders, and can be used for storage of temporary and/or transient data (temporary results, anything you just need during your job, or your batch of jobs).

You should remove any data from these systems after your processing them has finished. There are no guarantees about the time your data will be stored on this system, and we plan to clean these automatically on a regular base. The maximum allowed age of files on these scratch file systems depends on the type of scratch, and can be anywhere between a day and a few weeks. We

don't guarantee that these policies remain forever, and may change them if this seems necessary for the healthy operation of the cluster.

Each type of scratch has its own use:

Node scratch ($VSC_SCRATCH_NODE). Every node has its own scratch space, which is completely separated from the other nodes. On some clusters, it will be on a local disk in the node, while on other clusters it will be emulated through another file server. In many cases, it will be significantly slower than the cluster scratch as it typically consists of just a single disk. Some **drawbacks** are that the storage can only be accessed on that particular node and that the capacity is often very limited (e.g., 100 GB). The performance will depend a lot on the particular implementation in the cluster. In many cases, it will be significantly slower than the cluster scratch as it typically consists of just a single disk. However, if that disk is local to the node (as on most clusters), the performance will not depend on what others are doing on the cluster.

Cluster scratch ($VSC_SCRATCH). To allow a job running on multiple nodes (or multiple jobs running on separate nodes) to share data as files, every node of the cluster (including the login nodes) has access to this shared scratch directory. Just like the home and data directories, every user has its own scratch directory. Because this scratch is also available from the login nodes, you could manually copy results to your data directory after your job has ended. Also, this type of scratch is usually implemented by running tens or hundreds of disks in parallel on a powerful file server with fast connection to all the cluster nodes and therefore is often the fastest file system available on a cluster.
You may not get the same file system on different clusters, i.e., you may see different content on different clusters at the same institute. At the time of writing, the cluster scratch space is shared between both clusters at the University of Antwerp. This may change again in the future when storage gets updated.

Site scratch ($VSC_SCRATCH_SITE). At the time of writing, the site scratch is just the same volume as the cluster scratch, and thus contains the same data. In the future it may point to a different scratch file system that is available across all clusters at a particular site, which is in fact the case for the cluster scratch on some sites.

Global scratch ($VSC_SCRATCH_GLOBAL). At the time of writing, the global scratch is just the same volume as the cluster scratch, and thus contains the same data. In the future it may point to a scratch file system that is available across all clusters of the VSC, but at the moment of writing there are no plans to provide this.

6.2.5 Pre-defined quotas

Quota is enabled on these directories, which means that the amount of data you can store there is limited. This holds for both the total size of all files as well as the total number of files that can be stored. The system works with a soft quota and a hard quota. You can temporarily exceed the soft quota, but you can never exceed the hard quota. The user will get warnings as soon as he exceeds the soft quota.

The amount of data (called "*Block Limits*") that is currently in use by the user ("*KB*"), the soft limits ("*quota*") and the hard limits ("*limit*") for all 3 file-systems are always displayed when a user connects to the UAntwerpen-HPC.

With regards to the *file limits*, the number of files in use ("*files*"), its soft limit ("*quota*") and its hard limit ("*limit*") for the 3 file-systems are also displayed.

```
---------------------------------------------------------
Your quota is:

                  Block Limits
   Filesystem         KB      quota      limit    grace
   home           177920    3145728    3461120     none
   data         17707776   26214400   28835840     none
   scratch        371520   26214400   28835840     none

                File Limits
   Filesystem      files      quota      limit    grace
   home              671      20000      25000     none
   data           103079     100000     150000  expired
   scratch          2214     100000     150000     none

---------------------------------------------------------
```

Make sure to regularly check these numbers at log-in!

The rules are:

1. You will only receive a warning when you have reached the soft limit of either quota.
2. You *will* start losing data and get I/O errors when you reach the hard limit. In this case, data loss will occur since nothing can be written anymore (this holds both for new files as well as for existing files), until you free up some space by removing some files. Also note that you *will not* be warned when data loss occurs, so keep an eye open for the general quota warnings!
3. The same holds for running jobs that need to write files: when you reach your hard quota, jobs will crash.

We do realise that quota are often observed as a nuisance by users, especially if you're running low on it. However, it is an essential feature of a shared infrastructure. Quota ensure that a single user cannot accidentally take a cluster down (and break other user's jobs) by filling up the available disk space. And they help to guarantee a fair use of all available resources for all users. Quota also help to ensure that each folder is used for its intended purpose.

6.3 Writing Output files

Tip: Find the code of the exercises in "`~/examples/Running-jobs-with-input-output-data`"

In the next exercise, you will generate a file in the $VSC_SCRATCH directory. In order to generate some CPU- and disk-I/O load, we will

1. take a random integer between 1 and 2000 and calculate all primes up to that limit;
2. repeat this action 30.000 times;

3. write the output to the "`primes_1.txt`" output file in the $VSC_SCRATCH-directory.

Check the Python and the PBS file, and submit the job: Remember that this is already a more serious (disk-I/O and computational intensive) job, which takes approximately 3 minutes on the UAntwerpen-HPC.

```
$ cat file2.py
$ cat file2.pbs
$ qsub file2.pbs
$ qstat
$ ls -l
$ echo $VSC_SCRATCH
$ ls -l $VSC_SCRATCH
$ more $VSC_SCRATCH/primes_1.txt
```

6.4 Reading Input files

Tip: Find the code of the exercise "`file3.py`" in

"`~/examples/Running-jobs-with-input-output-data`".

In this exercise, you will

1. Generate the file "`primes_1.txt`" again as in the previous exercise;
2. open the this file;
3. read it line by line;
4. calculate the average of primes in the line;
5. count the number of primes found per line;
6. write it to the "`primes_2.txt`" output file in the $VSC_SCRATCH-directory.

Check the Python and the PBS file, and submit the job:

```
$ cat file3.py
$ cat file3.pbs
$ qsub file3.pbs
$ qstat
$ ls -l
$ more $VSC_SCRATCH/primes_2.txt
...
```

6.5 How much disk space do I get?

6.5.1 Quota

The available disk space on the UAntwerpen-HPC is limited. The actual disk capacity, shared by all users, can be found on the "Available hardware" page on the website. (https://

vscdocumentation.readthedocs.io/en/latest/hardware.html) As explained in subsection 6.2.5, this implies that there are also limits to:

- the amount of disk space; and
- the number of files

that can be made available to each individual UAntwerpen-HPC user.

The quota of disk space and number of files for each UAntwerpen-HPC user is:

Volume	Max. disk space	Max. # Files
HOME	3 GB	20000
DATA	25 GB	100000
SCRATCH	25 GB	100000

Tip: The first action to take when you have exceeded your quota is to clean up your directories. You could start by removing intermediate, temporary or log files. Keeping your environment clean will never do any harm.

Tip: Users can request for additional quota, which can be granted in duly justified cases. Please contact the UAntwerpen-HPC staff.

6.5.2 Check your quota

The "`show_quota`" command has been developed to show you the status of your quota in a readable format:

```
$ show_quota
VSC_DATA:    used 81MB (0%)  quota 25600MB
VSC_HOME:    used 33MB (1%)  quota 3072MB
VSC_SCRATCH:   used 28MB (0%)  quota 25600MB
VSC_SCRATCH_GLOBAL: used 28MB (0%)  quota 25600MB
VSC_SCRATCH_SITE:   used 28MB (0%)  quota 25600MB
```

or on the UAntwerp clusters

```
$ module load scripts
$ show_quota.py
VSC_DATA:    used 81MB (0%)  quota 25600MB
VSC_HOME:    used 33MB (1%)  quota 3072MB
VSC_SCRATCH:   used 28MB (0%)  quota 25600MB
VSC_SCRATCH_GLOBAL: used 28MB (0%)  quota 25600MB
VSC_SCRATCH_SITE:   used 28MB (0%)  quota 25600MB
```

With this command, you can follow up the consumption of your total disk quota easily, as it is expressed in percentages. Depending of on which cluster you are running the script, it may not be able to show the quota on all your folders. E.g., when running on the tier-1 system Muk, the script will not be able to show the quota on $VSC_HOME or $VSC_DATA if your account is a KU Leuven, UAntwerpen or VUB account.

Once your quota is (nearly) exhausted, you will want to know which directories are responsible for the consumption of your disk space. You can check the size of all subdirectories in the current directory with the "`du`" (**Disk Usage**) command:

```
$ du
256 ./ex01-matlab/log
1536 ./ex01-matlab
768 ./ex04-python
512 ./ex02-python
768 ./ex03-python
5632
```

This shows you first the aggregated size of all subdirectories, and finally the total size of the current directory "." (this includes files stored in the current directory).

If you also want this size to be "human readable" (and not always the total number of kilobytes), you add the parameter "-h":

```
$ du -h
256K ./ex01-matlab/log
1.5M ./ex01-matlab
768K ./ex04-python
512K ./ex02-python
768K ./ex03-python
5.5M .
```

If the number of lower level subdirectories starts to grow too big, you may not want to see the information at that depth; you could just ask for a summary of the current directory:

```
$ du -s
5632 .
$ du -s -h
5.5M .
```

If you want to see the size of any file or top-level subdirectory in the current directory, you could use the following command:

```
$ du -s -h *
1.5M ex01-matlab
512K ex02-python
768K ex03-python
768K ex04-python
256K example.sh
1.5M intro-HPC.pdf
```

Finally, if you don't want to know the size of the data in your current directory, but in some other directory (e.g., your data directory), you just pass this directory as a parameter. The command below will show the disk use in your home directory, even if you are currently in a different directory:

```
$ du -h $VSC_HOME/*
22M /user/antwerpen/201/vsc20167/dataset01
36M /user/antwerpen/201/vsc20167/dataset02
22M /user/antwerpen/201/vsc20167/dataset03
3.5M /user/antwerpen/201/vsc20167/primes.txt
```

We also want to mention the `tree` command, as it also provides an easy manner to see which files consumed your available quotas. *Tree* is a recursive directory-listing program that produces a depth indented listing of files.

Try:

```
$ tree -s -d
```

However, we urge you to only use the `du` and `tree` commands when you really need them as they can put a heavy strain on the file system and thus slow down file operations on the cluster for all other users.

6.6 Groups

Groups are a way to manage who can access what data. A user can belong to multiple groups at a time. **Groups can be created and managed without any interaction from the system administrators**.

Please note that changes are not instantaneous: it may take about an hour for the changes to propagate throughout the entire HPC infrastructure.

To change the group of a directory and it's underlying directories and files, you can use:

```
$ chgrp -R groupname directory
```

6.6.1 Joining an existing group

1. Get the group name you want to belong to.
2. Go to https://account.vscentrum.be/django/group/new and fill in the section named "Join group". You will be asked to fill in the group name and a message for the moderator of the group, where you identify yourself. This should look something like Figure 6.6.1.
3. After clicking the submit button, a message will be sent to the moderator of the group, who will either approve or deny the request. You will be a member of the group shortly after the group moderator approves your request.

6.6.2 Creating a new group

1. Go to https://account.vscentrum.be/django/group/new and scroll down to the section "Request new group". This should look something like Figure 6.6.2.
2. Fill out the group name. This cannot contain spaces.
3. Put a description of your group in the "Info" field.
4. You will now be a member and moderator of your newly created group.

6.6.3 Managing a group

Group moderators can go to https://account.vscentrum.be/django/group/edit to manage their group (see Figure 6.6.3). Moderators can invite and remove members. They can also promote other members to moderator and remove other moderators.

Figure 6.1: Joining a group.

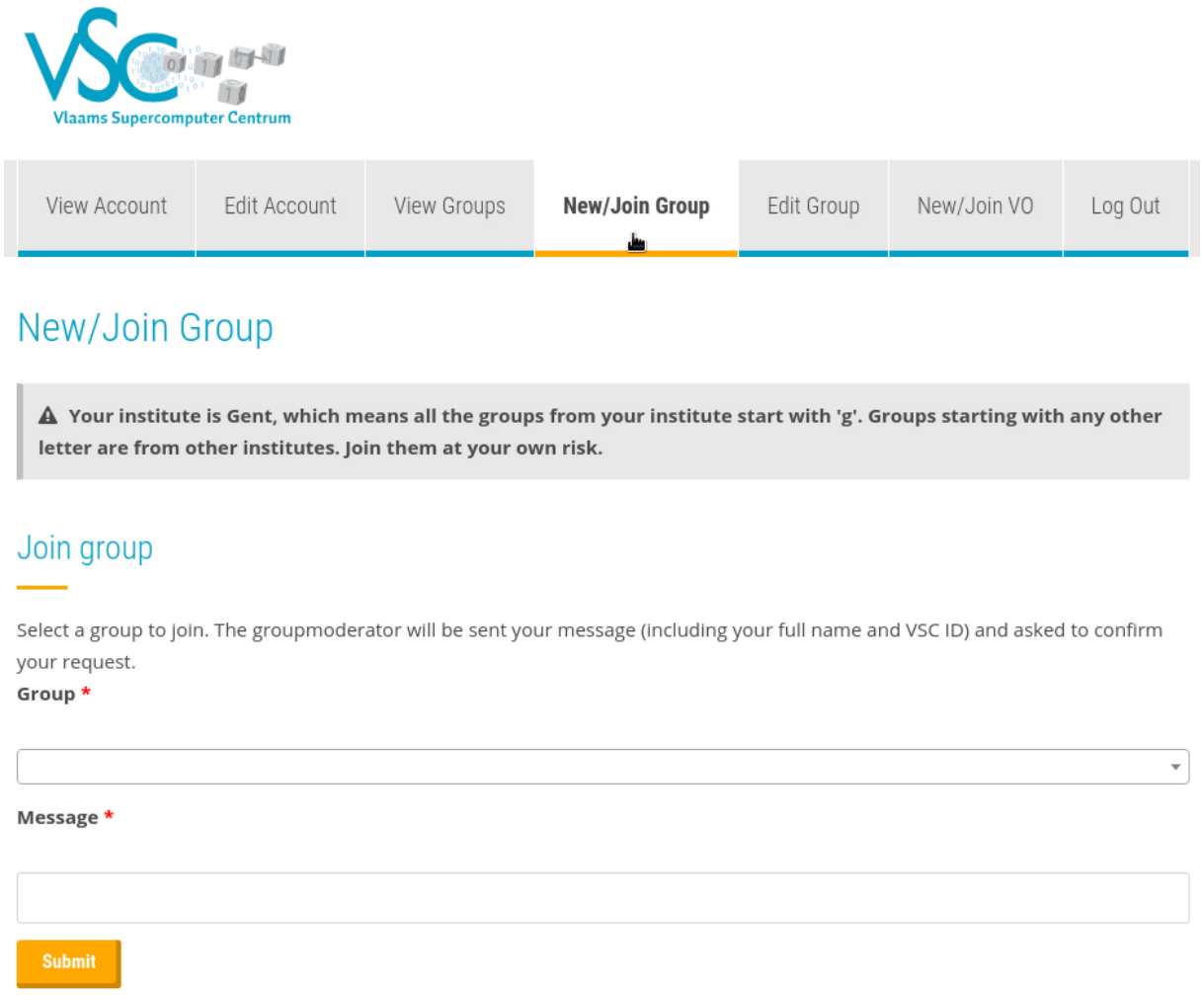
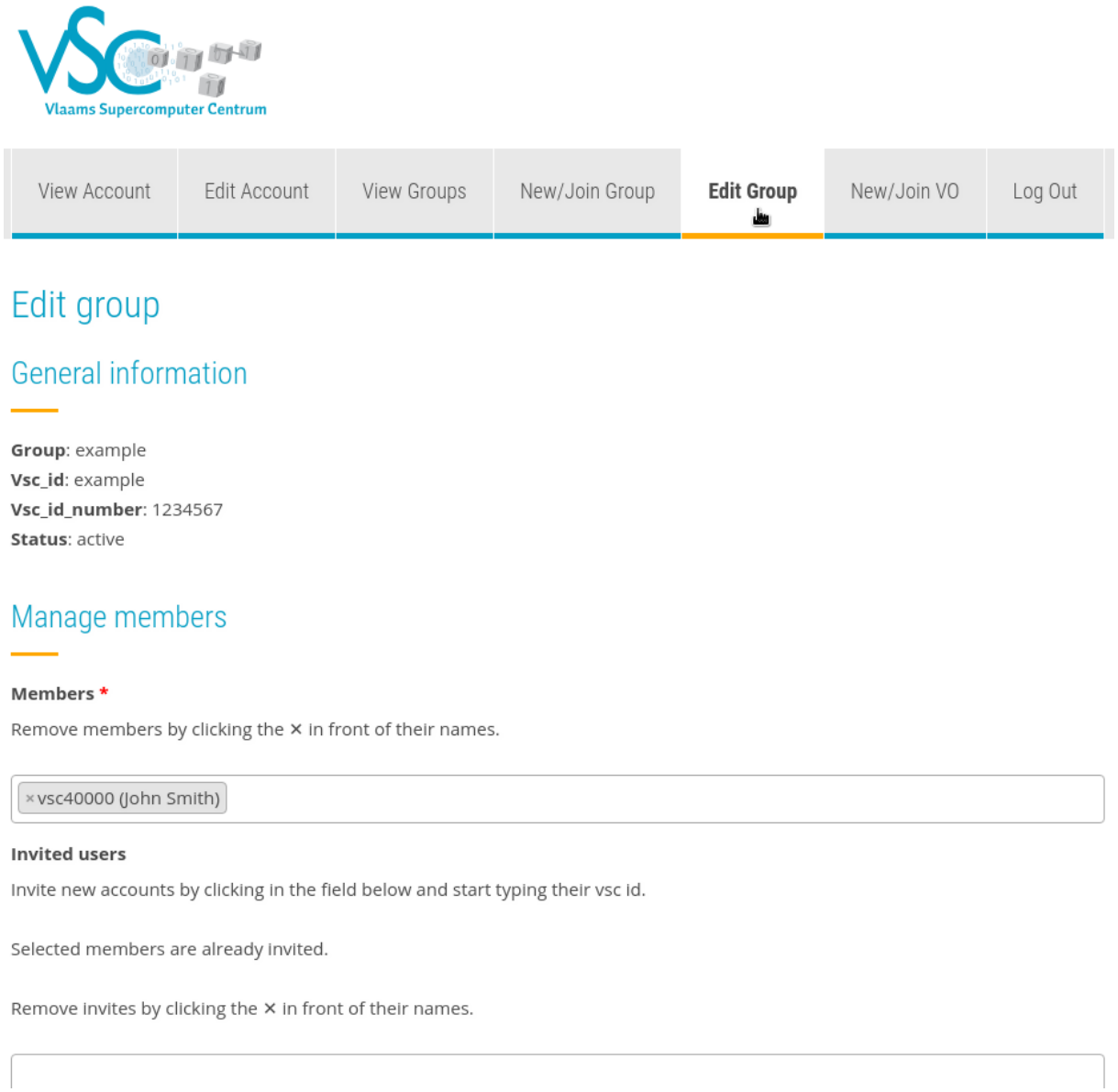


Figure 6.2: Creating a new group.

Figure 6.3: Creating a new group.



6.6.4 Inspecting groups

You can get details about the current state of groups on the HPC infrastructure with the following command (`example` is the name of the group we want to inspect):

```
$ getent group example
example:*:1234567:vsc40001,vsc40002,vsc40003
```

We can see that the VSC id number is 1234567 and that there are three members in the group: `vsc40001`, `vsc40002` and `vsc40003`.

Chapter 7

Multi core jobs/Parallel Computing

7.1 Why Parallel Programming?

There are two important motivations to engage in parallel programming.

1. Firstly, the need to decrease the time to solution: distributing your code over C cores holds the promise of speeding up execution times by a factor C. All modern computers (and probably even your smartphone) are equipped with multi-core processors capable of parallel processing.

2. The second reason is problem size: distributing your code over N nodes increases the available memory by a factor N, and thus holds the promise of being able to tackle problems which are N times bigger.

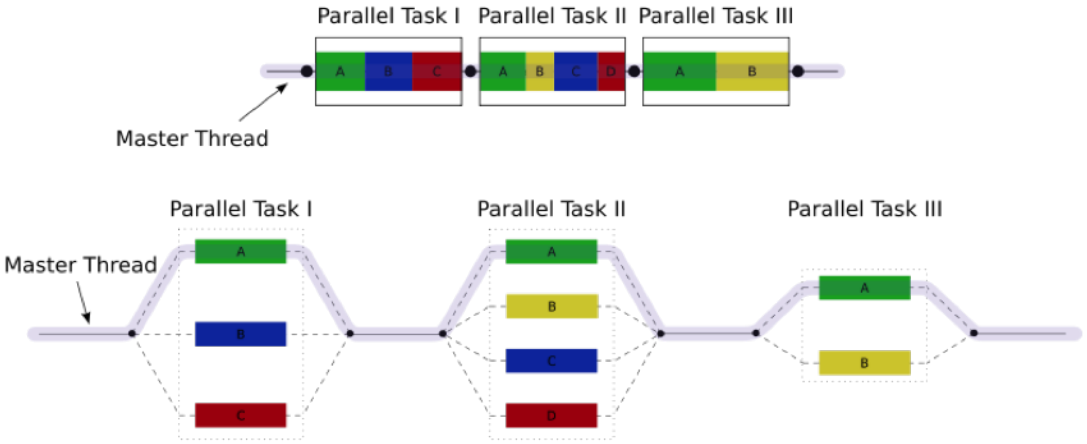
On a desktop computer, this enables a user to run multiple programs and the operating system simultaneously. For scientific computing, this means you have the ability in principle of splitting up your computations into groups and running each group on its own core.

There are multiple different ways to achieve parallel programming. The table below gives a (non-exhaustive) overview of problem independent approaches to parallel programming. In addition there are many problem specific libraries that incorporate parallel capabilities. The next three sections explore some common approaches: (raw) threads, OpenMP and MPI.

Parallel programming approaches		
Tool	**Available language bindings**	**Limitations**
Raw threads pthreads, boost:: threading, ...	Threading libraries are available for all common programming languages	Threads are limited to shared memory systems. They are more often used on single node systems rather than for UAntwerpen-HPC. Thread management is hard.
OpenMP	Fortran/C/C++	Limited to shared memory systems, but large shared memory systems for HPC are not uncommon (e.g., SGI UV). Loops and task can be parallelised by simple insertion of compiler directives. Under the hood threads are used. Hybrid approaches exist which use OpenMP to parallelise the work load on each node and MPI (see below) for communication between nodes.
Lightweight threads with clever scheduling, Intel TBB, Intel Cilk Plus	C/C++	Limited to shared memory systems, but may be combined with MPI. Thread management is taken care of by a very clever scheduler enabling the programmer to focus on parallelisation itself. Hybrid approaches exist which use TBB and/or Cilk Plus to parallelise the work load on each node and MPI (see below) for communication between nodes.
MPI	Fortran/C/C++, Python	Applies to both distributed and shared memory systems. Cooperation between different nodes or cores is managed by explicit calls to library routines handling communication routines.
Global Arrays library	C/C++, Python	Mimics a global address space on distributed memory systems, by distributing arrays over many nodes and one sided communication. This library is used a lot for chemical structure calculation codes and was used in one of the first applications that broke the PetaFlop barrier.
Scoop	Python	Applies to both shared and distributed memory system. Not extremely advanced, but may present a quick road to parallelisation of Python code.

7.2 Parallel Computing with threads

Multi-threading is a widespread programming and execution model that allows multiple threads to exist within the context of a single process. These threads share the process' resources, but are able to execute independently. The threaded programming model provides developers with a useful abstraction of concurrent execution. Multi-threading can also be applied to a single process to enable parallel execution on a multiprocessing system.



This advantage of a multithreaded program allows it to operate faster on computer systems that have multiple CPUs or across a cluster of machines — because the threads of the program naturally lend themselves to truly concurrent execution. In such a case, the programmer needs to be careful to avoid race conditions, and other non-intuitive behaviours. In order for data to be correctly manipulated, threads will often need to synchronise in time in order to process the data in the correct order. Threads may also require mutually exclusive operations (often implemented using semaphores) in order to prevent common data from being simultaneously modified, or read while in the process of being modified. Careless use of such primitives can lead to deadlocks.

Threads are a way that a program can spawn concurrent units of processing that can then be delegated by the operating system to multiple processing cores. Clearly the advantage of a multithreaded program (one that uses multiple threads that are assigned to multiple processing cores) is that you can achieve big speedups, as all cores of your CPU (and all CPUs if you have more than one) are used at the same time.

Here is a simple example program that spawns 5 threads, where each one runs a simple function that only prints "Hello from thread".

Go to the example directory:

```
$ cd ~/examples/Multi-core-jobs-Parallel-Computing
```

Study the example first:

— T_hello.c —

```c
/*
 * VSC        : Flemish Supercomputing Centre
 * Tutorial   : Introduction to HPC
 * Description: Showcase of working with threads
 */
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NTHREADS 5

void *myFun(void *x)
{
  int tid;
  tid = *((int *) x);
  printf("Hello from thread %d!\n", tid);
  return NULL;
}

int main(int argc, char *argv[])
{
  pthread_t threads[NTHREADS];
  int thread_args[NTHREADS];
  int rc, i;

  /* spawn the threads */
  for (i=0; i<NTHREADS; ++i)
    {
      thread_args[i] = i;
      printf("spawning thread %d\n", i);
      rc = pthread_create(&threads[i], NULL, myFun, (void *) &thread_args[i]);
    }

  /* wait for threads to finish */
  for (i=0; i<NTHREADS; ++i) {
    rc = pthread_join(threads[i], NULL);
  }

  return 1;
}
```

And compile it (whilst including the thread library) and run and test it on the login-node:

```
$ module load GCC
$ gcc -o T_hello T_hello.c -lpthread
$ ./T_hello
spawning thread 0
spawning thread 1
spawning thread 2
Hello from thread 0!
Hello from thread 1!
Hello from thread 2!
spawning thread 3
spawning thread 4
Hello from thread 3!
Hello from thread 4!
```

Now, run it on the cluster and check the output:

```
$ qsub T_hello.pbs
433253.leibniz
$ more T_hello.pbs.o433253
spawning thread 0
spawning thread 1
spawning thread 2
Hello from thread 0!
Hello from thread 1!
Hello from thread 2!
spawning thread 3
spawning thread 4
Hello from thread 3!
Hello from thread 4!
```

Tip: If you plan engaging in parallel programming using threads, this book may prove useful: *Professional Multicore Programming: Design and Implementation for C++ Developers. Cameron Hughes and Tracey Hughes. Wrox 2008.*

7.3 Parallel Computing with OpenMP

OpenMP is an API that implements a multi-threaded, shared memory form of parallelism. It uses a set of compiler directives (statements that you add to your code and that are recognised by your Fortran/C/C++ compiler if OpenMP is enabled or otherwise ignored) that are incorporated at compile-time to generate a multi-threaded version of your code. You can think of Pthreads (above) as doing multi-threaded programming "by hand", and OpenMP as a slightly more automated, higher-level API to make your program multithreaded. OpenMP takes care of many of the low-level details that you would normally have to implement yourself, if you were using Pthreads from the ground up.

An important advantage of OpenMP is that, because it uses compiler directives, the original serial version stays intact, and minimal changes (in the form of compiler directives) are necessary to turn a working serial code into a working parallel code.

Here is the general code structure of an OpenMP program:

```
1  #include <omp.h>
2  main ()  {
3  int var1, var2, var3;
4  // Serial code
5  // Beginning of parallel section. Fork a team of threads.
6  // Specify variable scoping
7
8  #pragma omp parallel private(var1, var2) shared(var3)
9    {
10   // Parallel section executed by all threads
11   // All threads join master thread and disband
12   }
13 // Resume serial code
14 }
```

7.3.1 Private versus Shared variables

By using the private() and shared() clauses, you can specify variables within the parallel region as being **shared**, i.e., visible and accessible by all threads simultaneously, or **private**, i.e., private to each thread, meaning each thread will have its own local copy. In the code example below for parallelising a for loop, you can see that we specify the thread_id and nloops variables as private.

7.3.2 Parallelising for loops with OpenMP

Parallelising for loops is really simple (see code below). By default, loop iteration counters in OpenMP loop constructs (in this case the i variable) in the for loop are set to private variables.

— omp1.c —

```
1  /*
2   * VSC        : Flemish Supercomputing Centre
3   * Tutorial   : Introduction to HPC
4   * Description: Showcase program for OMP loops
5   */
6  /* OpenMP_loop.c  */
7  #include <stdio.h>
8  #include <omp.h>
9
10 int main(int argc, char **argv)
11 {
12   int i, thread_id, nloops;
13
14 #pragma omp parallel private(thread_id, nloops)
15   {
16     nloops = 0;
17
18 #pragma omp for
19     for (i=0; i<1000; ++i)
20     {
21       ++nloops;
22     }
23     thread_id = omp_get_thread_num();
24     printf("Thread %d performed %d iterations of the loop.\n", thread_id, nloops );
25   }
26
27   return 0;
28 }
```

And compile it (whilst including the "*openmp*" library) and run and test it on the login-node:

```
$ module load GCC
$ gcc -fopenmp -o omp1 omp1.c
$ ./omp1
Thread 6 performed 125 iterations of the loop.
Thread 7 performed 125 iterations of the loop.
Thread 5 performed 125 iterations of the loop.
Thread 4 performed 125 iterations of the loop.
Thread 0 performed 125 iterations of the loop.
Thread 2 performed 125 iterations of the loop.
Thread 3 performed 125 iterations of the loop.
Thread 1 performed 125 iterations of the loop.
```

Now run it in the cluster and check the result again.

```
$ qsub omp1.pbs
$ cat omp1.pbs.o*
Thread 1 performed 125 iterations of the loop.
Thread 4 performed 125 iterations of the loop.
Thread 3 performed 125 iterations of the loop.
Thread 0 performed 125 iterations of the loop.
Thread 5 performed 125 iterations of the loop.
Thread 7 performed 125 iterations of the loop.
Thread 2 performed 125 iterations of the loop.
Thread 6 performed 125 iterations of the loop.
```

7.3.3 Critical Code

Using OpenMP you can specify something called a "critical" section of code. This is code that is performed by all threads, but is only performed **one thread at a time** (i.e., in serial). This provides a convenient way of letting you do things like updating a global variable with local results from each thread, and you don't have to worry about things like other threads writing to that global variable at the same time (a collision).

— omp2.c —

```c
/*
 * VSC        : Flemish Supercomputing Centre
 * Tutorial   : Introduction to HPC
 * Description: OpenMP Test Program
 */
#include <stdio.h>
#include <omp.h>

int main(int argc, char *argv[])
{
  int i, thread_id;
  int glob_nloops, priv_nloops;
  glob_nloops = 0;

  // parallelize this chunk of code
  #pragma omp parallel private(priv_nloops, thread_id)
  {
    priv_nloops = 0;
    thread_id = omp_get_thread_num();

    // parallelize this for loop
    #pragma omp for
    for (i=0; i<100000; ++i)
    {
      ++priv_nloops;
    }

    // make this a "critical" code section
    #pragma omp critical
    {
      printf("Thread %d is adding its iterations (%d) to sum (%d), ", thread_id,
    priv_nloops, glob_nloops);
      glob_nloops += priv_nloops;
      printf("total is now %d.\n", glob_nloops);
    }
  }
  printf("Total # loop iterations is %d\n", glob_nloops);
  return 0;
}
```

And compile it (whilst including the "*openmp*" library) and run and test it on the login-node:

```
$ module load GCC
$ gcc -fopenmp -o omp2 omp2.c
$ ./omp2
Thread 3 is adding its iterations (12500) to sum (0), total is now 12500.
Thread 7 is adding its iterations (12500) to sum (12500), total is now 25000.
Thread 5 is adding its iterations (12500) to sum (25000), total is now 37500.
Thread 6 is adding its iterations (12500) to sum (37500), total is now 50000.
Thread 2 is adding its iterations (12500) to sum (50000), total is now 62500.
Thread 4 is adding its iterations (12500) to sum (62500), total is now 75000.
Thread 1 is adding its iterations (12500) to sum (75000), total is now 87500.
Thread 0 is adding its iterations (12500) to sum (87500), total is now 100000.
Total # loop iterations is 100000
```

Now run it in the cluster and check the result again.

```
$ qsub omp2.pbs
$ cat omp2.pbs.o*
Thread 2 is adding its iterations (12500) to sum (0), total is now 12500.
Thread 0 is adding its iterations (12500) to sum (12500), total is now 25000.
Thread 1 is adding its iterations (12500) to sum (25000), total is now 37500.
Thread 4 is adding its iterations (12500) to sum (37500), total is now 50000.
Thread 7 is adding its iterations (12500) to sum (50000), total is now 62500.
Thread 3 is adding its iterations (12500) to sum (62500), total is now 75000.
Thread 5 is adding its iterations (12500) to sum (75000), total is now 87500.
Thread 6 is adding its iterations (12500) to sum (87500), total is now 100000.
Total # loop iterations is 100000
```

7.3.4 Reduction

Reduction refers to the process of combining the results of several sub-calculations into a final result. This is a very common paradigm (and indeed the so-called "map-reduce" framework used by Google and others is very popular). Indeed we used this paradigm in the code example above, where we used the "critical code" directive to accomplish this. The map-reduce paradigm is so common that OpenMP has a specific directive that allows you to more easily implement this.

— omp3.c —

```c
/*
 * VSC        : Flemish Supercomputing Centre
 * Tutorial   : Introduction to HPC
 * Description: OpenMP Test Program
 */
#include <stdio.h>
#include <omp.h>

int main(int argc, char *argv[])
{
  int i, thread_id;
  int glob_nloops, priv_nloops;
  glob_nloops = 0;

  // parallelize this chunk of code
  #pragma omp parallel private(priv_nloops, thread_id) reduction(+:glob_nloops)
  {
    priv_nloops = 0;
    thread_id = omp_get_thread_num();

    // parallelize this for loop
    #pragma omp for
    for (i=0; i<100000; ++i)
    {
      ++priv_nloops;
    }
    glob_nloops += priv_nloops;
  }
  printf("Total # loop iterations is %d\n", glob_nloops);
  return 0;
}
```

And compile it (whilst including the "*openmp*" library) and run and test it on the login-node:

```
$ module load GCC
$ gcc -fopenmp -o omp3 omp3.c
$ ./omp3
Total # loop iterations is 100000
```

Now run it in the cluster and check the result again.

```
$ qsub omp3.pbs
$ cat omp3.pbs.o*
Total # loop iterations is 100000
```

7.3.5 Other OpenMP directives

There are a host of other directives you can issue using OpenMP.

Some other clauses of interest are:

1. barrier: each thread will wait until all threads have reached this point in the code, before

proceeding

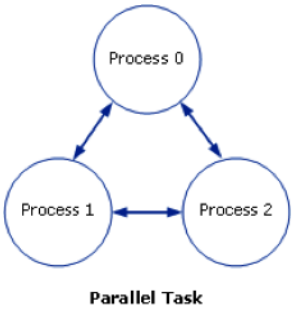
2. nowait: threads will not wait until everybody is finished
3. schedule(type, chunk) allows you to specify how tasks are spawned out to threads in a for loop. There are three types of scheduling you can specify
4. if: allows you to parallelise only if a certain condition is met
5. ... and a host of others

Tip: If you plan engaging in parallel programming using OpenMP, this book may prove useful: *Using OpenMP - Portable Shared Memory Parallel Programming.* By Barbara Chapman Gabriele Jost and Ruud van der Pas Scientific and Engineering Computation. 2005.

7.4 Parallel Computing with MPI

The Message Passing Interface (MPI) is a standard defining core syntax and semantics of library routines that can be used to implement parallel programming in C (and in other languages as well). There are several implementations of MPI such as Open MPI, Intel MPI, M(VA)PICH and LAM/MPI.

In the context of this tutorial, you can think of MPI, in terms of its complexity, scope and control, as sitting in between programming with Pthreads, and using a high-level API such as OpenMP. For a Message Passing Interface (MPI) application, a parallel task usually consists of a single executable running concurrently on multiple processors, with communication between the processes. This is shown in the following diagram:



The process numbers 0, 1 and 2 represent the process rank and have greater or less significance depending on the processing paradigm. At the minimum, Process 0 handles the input/output and determines what other processes are running.

The MPI interface allows you to manage allocation, communication, and synchronisation of a set of processes that are mapped onto multiple nodes, where each node can be a core within a single CPU, or CPUs within a single machine, or even across multiple machines (as long as they are networked together).

One context where MPI shines in particular is the ability to easily take advantage not just of multiple cores on a single machine, but to run programs on clusters of several machines. Even if

you don't have a dedicated cluster, you could still write a program using MPI that could run your program in parallel, across any collection of computers, as long as they are networked together.

Here is a "Hello World" program in MPI written in C. In this example, we send a "Hello" message to each processor, manipulate it trivially, return the results to the main process, and print the messages.

Study the MPI-programme and the PBS-file:

— mpi_hello.c —

```c
/*
 * VSC        : Flemish Supercomputing Centre
 * Tutorial   : Introduction to HPC
 * Description: "Hello World" MPI Test Program
 */
#include <stdio.h>
#include <mpi.h>

 #include <mpi.h>
 #include <stdio.h>
 #include <string.h>

 #define BUFSIZE 128
 #define TAG 0

 int main(int argc, char *argv[])
 {
   char idstr[32];
   char buff[BUFSIZE];
   int numprocs;
   int myid;
   int i;
   MPI_Status stat;
   /* MPI programs start with MPI_Init; all 'N' processes exist thereafter */
   MPI_Init(&argc,&argv);
   /* find out how big the SPMD world is */
   MPI_Comm_size(MPI_COMM_WORLD,&numprocs);
   /* and this processes' rank is */
   MPI_Comm_rank(MPI_COMM_WORLD,&myid);

   /* At this point, all programs are running equivalently, the rank
      distinguishes the roles of the programs in the SPMD model, with
      rank 0 often used specially... */
   if(myid == 0)
   {
     printf("%d: We have %d processors\n", myid, numprocs);
     for(i=1;i<numprocs;i++)
     {
       sprintf(buff, "Hello %d! ", i);
       MPI_Send(buff, BUFSIZE, MPI_CHAR, i, TAG, MPI_COMM_WORLD);
     }
     for(i=1;i<numprocs;i++)
     {
       MPI_Recv(buff, BUFSIZE, MPI_CHAR, i, TAG, MPI_COMM_WORLD, &stat);
       printf("%d: %s\n", myid, buff);
     }
   }
   else
   {
     /* receive from rank 0: */
     MPI_Recv(buff, BUFSIZE, MPI_CHAR, 0, TAG, MPI_COMM_WORLD, &stat);
     sprintf(idstr, "Processor %d ", myid);
     strncat(buff, idstr, BUFSIZE-1);
     strncat(buff, "reporting for duty", BUFSIZE-1);
     /* send to rank 0: */
     MPI_Send(buff, BUFSIZE, MPI_CHAR, 0, TAG, MPI_COMM_WORLD);
   }

   /* MPI programs end with MPI Finalize; this is a weak synchronization point */
   MPI_Finalize();
   return 0;
 }
```

— mpi_hello.pbs —

```
#!/bin/bash

#PBS -N mpihello
#PBS -l walltime=00:05:00

  # assume a 40 core job
#PBS -l nodes=2:ppn=20

  # make sure we are in the right directory in case writing files
cd $PBS_O_WORKDIR

  # load the environment

module load intel

mpirun ./mpi_hello
```

and compile it:

```
$ module load intel
$ mpiicc -o mpi_hello mpi_hello.c
```

mpiicc is a wrapper of the Intel C++ compiler icc to compile MPI programs (see the chapter on compilation for details).

Run the parallel program:

```
$ qsub mpi_hello.pbs
$ ls -l
total 1024
-rwxrwxr-x 1 vsc20167 8746 Sep 16 14:19 mpi_hello*
-rw-r--r-- 1 vsc20167 1626 Sep 16 14:18 mpi_hello.c
-rw------- 1 vsc20167    0 Sep 16 14:22 mpi_hello.o433253
-rw------- 1 vsc20167  697 Sep 16 14:22 mpi_hello.o433253
-rw-r--r-- 1 vsc20167  304 Sep 16 14:22 mpi_hello.pbs
$ cat mpi_hello.o433253
0: We have 16 processors
0: Hello 1! Processor 1 reporting for duty
0: Hello 2! Processor 2 reporting for duty
0: Hello 3! Processor 3 reporting for duty
0: Hello 4! Processor 4 reporting for duty
0: Hello 5! Processor 5 reporting for duty
0: Hello 6! Processor 6 reporting for duty
0: Hello 7! Processor 7 reporting for duty
0: Hello 8! Processor 8 reporting for duty
0: Hello 9! Processor 9 reporting for duty
0: Hello 10! Processor 10 reporting for duty
0: Hello 11! Processor 11 reporting for duty
0: Hello 12! Processor 12 reporting for duty
0: Hello 13! Processor 13 reporting for duty
0: Hello 14! Processor 14 reporting for duty
0: Hello 15! Processor 15 reporting for duty
```

The runtime environment for the MPI implementation used (often called mpirun or mpiexec)

spawns multiple copies of the program, with the total number of copies determining the number of process *ranks* in MPI_COMM_WORLD, which is an opaque descriptor for communication between the set of processes. A single process, multiple data (SPMD = Single Program, Multiple Data) programming model is thereby facilitated, but not required; many MPI implementations allow multiple, different, executables to be started in the same MPI job. Each process has its own rank, the total number of processes in the world, and the ability to communicate between them either with point-to-point (send/receive) communication, or by collective communication among the group. It is enough for MPI to provide an SPMD-style program with MPI_COMM_WORLD, its own rank, and the size of the world to allow algorithms to decide what to do. In more realistic situations, I/O is more carefully managed than in this example. MPI does not guarantee how POSIX I/O would actually work on a given system, but it commonly does work, at least from rank 0.

MPI uses the notion of process rather than processor. Program copies are *mapped* to processors by the MPI runtime. In that sense, the parallel machine can map to 1 physical processor, or N where N is the total number of processors available, or something in between. For maximum parallel speedup, more physical processors are used. This example adjusts its behaviour to the size of the world N, so it also seeks to scale to the runtime configuration without compilation for each size variation, although runtime decisions might vary depending on that absolute amount of concurrency available.

Tip: If you plan engaging in parallel programming using MPI, this book may prove useful: *Parallel Programming with MPI. Peter Pacheo. Morgan Kaufmann. 1996.*

Chapter 8

Troubleshooting

8.1 Walltime issues

If you get from your job output an error message similar to this:

```
=>> PBS: job killed: walltime <value in seconds> exceeded limit <value in seconds>
```

This occurs when your job did not complete within the requested walltime. See section 11.1 for more information about how to request the walltime. It is recommended to use *checkpointing* if the job requires **72 hours** of walltime or more to be executed.

8.2 Out of quota issues

Sometimes a job hangs at some point or it stops writing in the disk. These errors are usually related to the quota usage. You may have reached your quota limit at some storage endpoint. You should move (or remove) the data to a different storage endpoint (or request more quota) to be able to write to the disk and then resubmit the jobs.

8.3 Issues connecting to login node

If you are confused about the SSH public/private key pair concept, maybe the key/lock analogy in subsection 2.1.1 can help.

If you have errors that look like:

```
vsc20167@login.hpc.uantwerpen.be: Permission denied
```

or you are experiencing problems with connecting, here is a list of things to do that should help:

1. Keep in mind that it an take up to an hour for your VSC account to become active after it has been *approved*; until then, logging in to your VSC account will not work.
2. Make sure you are connecting from an IP address that is allowed to access the VSC login nodes, see section 3.1 for more information.

3. Please double/triple check your VSC login ID. It should look something like *vsc20167*: the letters `vsc`, followed by exactly 5 digits. Make sure it's the same one as the one on https://account.vscentrum.be/.

4. You previously connected to the UAntwerpen-HPC from another machine, but now have another machine? Please follow the procedure for adding additional keys in section 2.2.3. You may need to wait for 15-20 minutes until the SSH public key(s) you added become active.

5. Make sure you are using the private key (not the public key) when trying to connect: If you followed the manual, the private key filename should end in `.ppk` (not in `.pub`).

6. If you have multiple private keys on your machine, please make sure you are using the one that corresponds to (one of) the public key(s) you added on https://account.vscentrum.be/.

7. Please do not use someone else's private keys. You must never share your private key, they're called *private* for a good reason.

If you are using PuTTY and get this error message:

```
server unexpectedly closed network connection
```

it is possible that the PuTTY version you are using is too old and doesn't support some required (security-related) features.

Make sure you are using the latest PuTTY version if you are encountering problems connecting (see subsection 2.1.2). If that doesn't help, please contact hpc@uantwerpen.be.

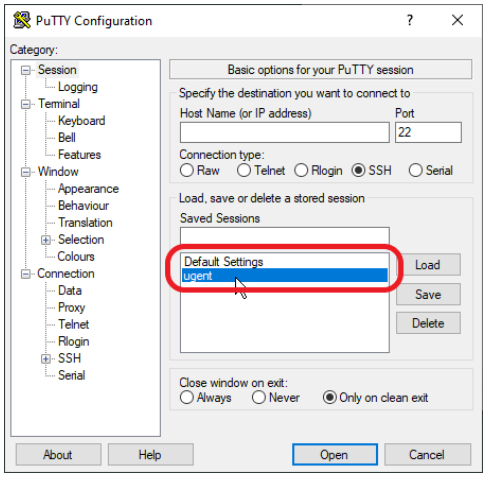
If you've tried all applicable items above and it doesn't solve your problem, please contact hpc@uantwerpen.be and include the following information:

Please create a log file of your SSH session by following the steps in this article and include it in the email.

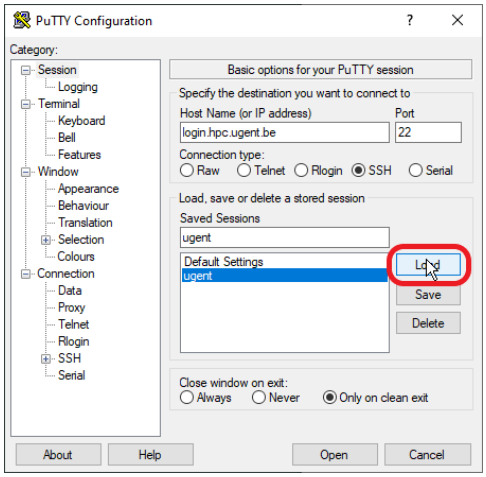
8.3.1 Change PuTTY private key for a saved configuration

1. Open PuTTY

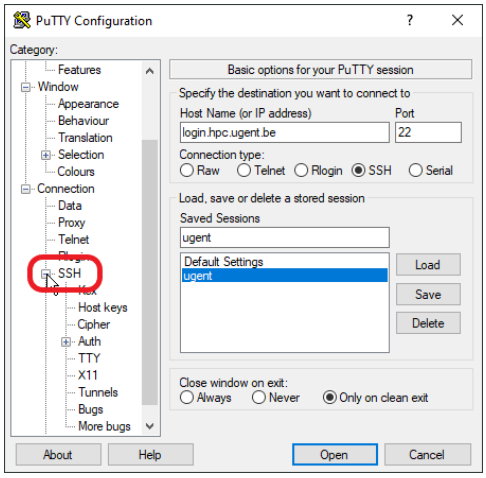
2. Single click on the saved configuration



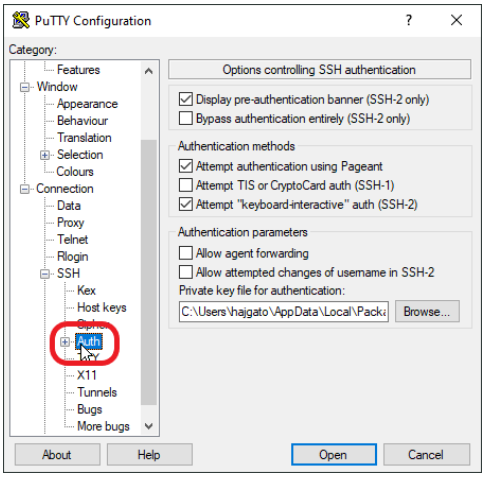
3. Then click Load button



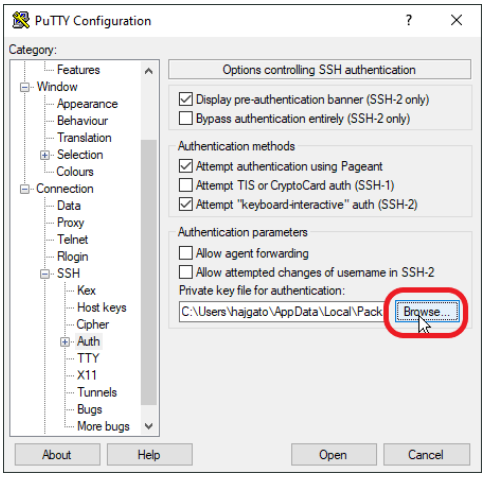
4. Expand SSH category (on the left panel) clicking on the "+" next to SSH



5. Click on Auth under the SSH category

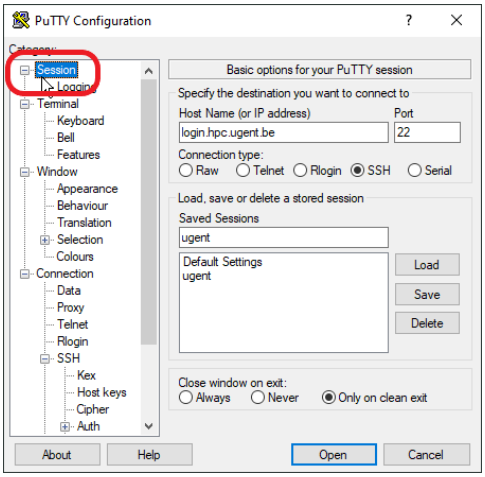


6. On the right panel, click Browse button

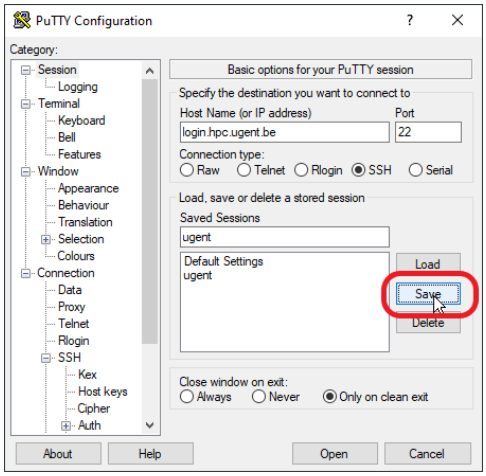


7. Then search your private key on your computer (with the extension ".ppk")

8. Go back to the top of category, and click Session



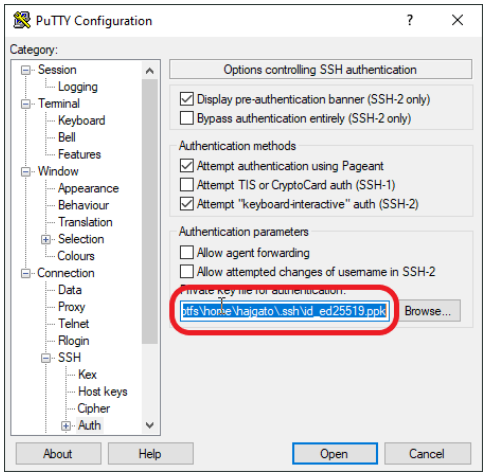
9. On the right panel, click on Save button



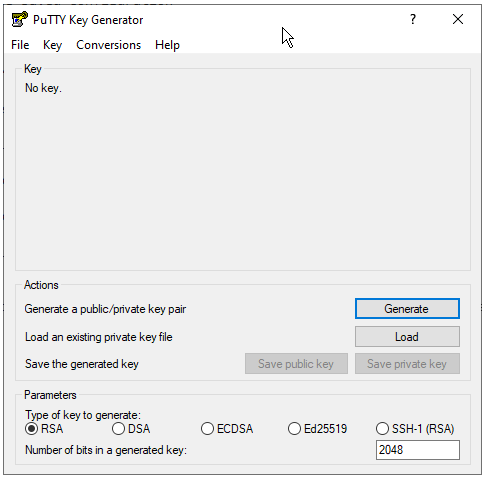
8.3.2 Check whether your private key in PuTTY matches the public key on the accountpage

Follow the instructions in subsection 8.3.1 util item item 5, then:

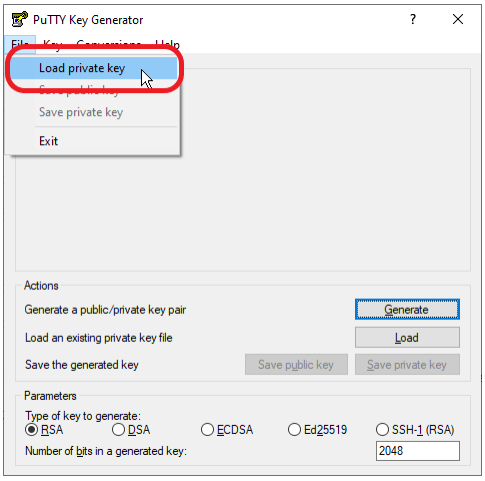
6. Single click on the textbox containig the path to your private key, then select all text (push Ctrl + a), then copy the location of the private key (push Ctrl + c)



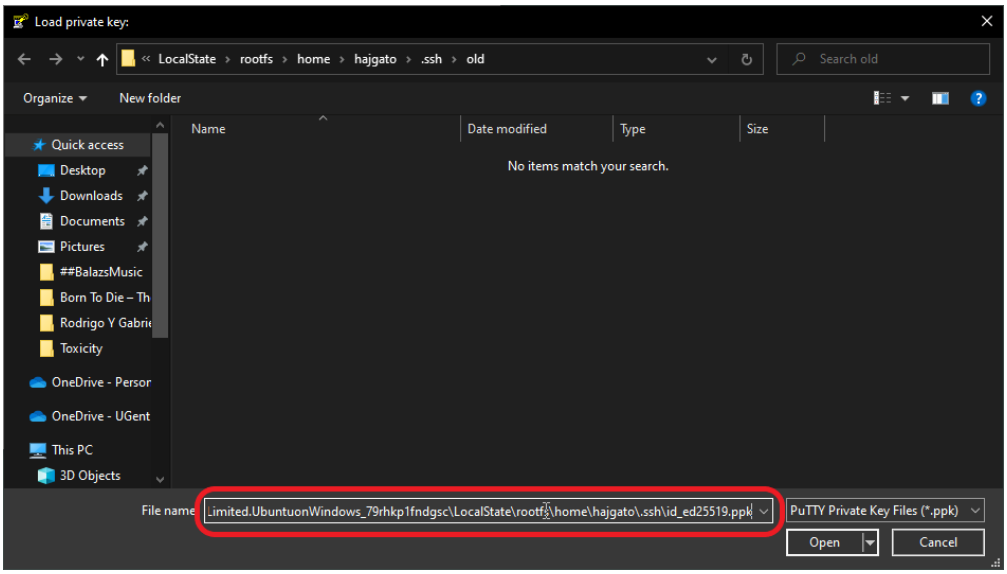
7. Open PuTTYgen



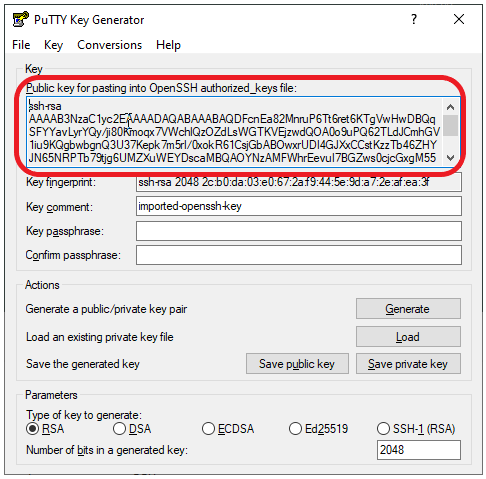
8. Enter menu item "File" and select "Load Private key"



9. On the "Load private key" popup, click in the textbox next to "File name:", then paste the location of your private key (push Ctrl + v), then click Open



10. Make sure that your Public key from the "Public key for pasting into OpenSSH authorized_keys file" textbox is in your "Public keys" section on the accountpage https://account.vscentrum.be. (Scroll down to the bottom of "View Account" tab, you will find there the "Public keys" section)



8.4 Security warning about invalid host key

If you get a warning that looks like the one below, it is possible that someone is trying to intercept the connection between you and the system you are connecting to. Another possibility is that the host key of the system you are connecting to has changed.

You will need to verify that the fingerprint shown in the dialog matches one of the following fingerprints:

- `ssh-rsa 2048 5a:40:9d:2a:f4:b7:6c:87:0d:87:30:07:9d:ea:80:11`
- `sha-rsa 2048 SHA256:W8HRHTZpPd2GIhoNU2xj2oUKhcFr2bjIzZsKzY+1PCA`
- `ssh-ecdsa 256 f9:4f:19:9b:fb:40:c5:6c:6f:b9:64:2e:33:0a:8d:26`
- `ssh-ecdsa 256 SHA256:DlsP+jFrTqSdr9VquUpDj17Uy99zFdFN/LxVhaQQzbo`
- `ssh-ed25519 255 df:0c:61:b9:26:51:0f:b4:ca:43:ac:f6:ee:d2:a1:29`
- `ssh-ed25519 255 SHA256:+vlrkJui34B4iumxGVHd447K3W8wzgE1n1h2/Ic0WlE`

Do not click "Yes" until you verified the fingerprint. Do not press "No" in any case..

If it the fingerprint matches, click "Yes".

If it doesn't (like in the example) or you are in doubt, take a screenshot, press "Cancel" and contact hpc@uantwerpen.be.

Note: it is possible that the `ssh-ed25519` fingerprint starts with "`ssh-ed25519 255`" rather than "`ssh-ed25519 256`" (or vice versa), depending on the PuTTY version you are using. It is safe to ignore this `255` versus `256` difference, but the part after should be **identical**.

8.5 DOS/Windows text format

If you get errors like:

```
$ qsub fibo.pbs
qsub:  script is written in DOS/Windows text format
```

It's probably because you transferred the files from a Windows computer. Please go to the section about `dos2unix` in chapter 5 of the intro to Linux to fix this error.

8.6 Warning message when first connecting to new host

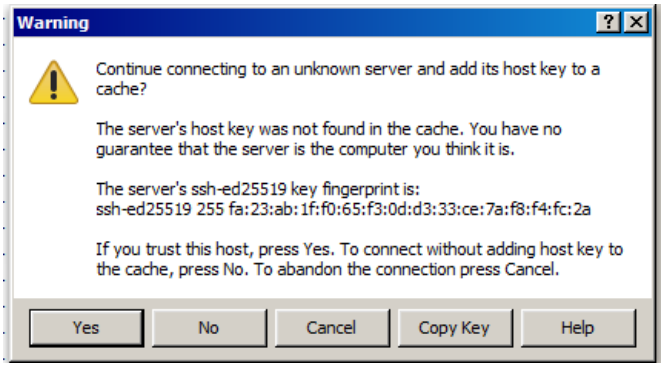
The first time you make a connection to the login node, a Security Alert will appear and you will be asked to verify the authenticity of the login node.

Make sure the fingerprint in the alert matches one of the following:

- `ssh-rsa 2048 5a:40:9d:2a:f4:b7:6c:87:0d:87:30:07:9d:ea:80:11`
- `sha-rsa 2048 SHA256:W8HRHTZpPd2GIhoNU2xj2oUKhcFr2bjIzZsKzY+1PCA`
- `ssh-ecdsa 256 f9:4f:19:9b:fb:40:c5:6c:6f:b9:64:2e:33:0a:8d:26`
- `ssh-ecdsa 256 SHA256:DlsP+jFrTqSdr9VquUpDj17Uy99zFdFN/LxVhaQQzbo`
- `ssh-ed25519 255 df:0c:61:b9:26:51:0f:b4:ca:43:ac:f6:ee:d2:a1:29`
- `ssh-ed25519 255 SHA256:+vlrkJui34B4iumxGVHd447K3W8wzgE1n1h2/Ic0WlE`

If it does, press ***Yes***, if it doesn't, please contact hpc@uantwerpen.be.

Note: it is possible that the `ssh-ed25519` fingerprint starts with "`ssh-ed25519 255`" rather than "`ssh-ed25519 256`" (or vice versa), depending on the PuTTY version you are using. It is safe to ignore this `255` versus `256` difference, but the part after should be **identical**.



8.7 Memory limits

To avoid jobs allocating too much memory, there are memory limits in place by default. It is possible to specify higher memory limits if your jobs require this.

8.7.1 How will I know if memory limits are the cause of my problem?

If your program fails with a memory-related issue, there is a good chance it failed because of the memory limits and you should increase the memory limits for your job.

Examples of these error messages are: `malloc failed`, `Out of memory`, `Could not allocate memory` or in Java: `Could not reserve enough space for object heap` . Your program can also run into a `Segmentation fault` (or `segfault`) or crash due to bus errors.

You can check the amount of virtual memory (in Kb) that is available to you via the `ulimit -v` command *in your job script*.

8.7.2 How do I specify the amount of memory I need?

See subsection 4.6.1 to set memory and other requirements, see section 11.2 to finetune the amount of memory you request.

Chapter 9

HPC Policies

The cluster has been setup to support the ongoing research in all the domains at the University of Antwerp. As such, it should be considered as valuable research equipment. Users shall not abuse the system for other purposes. By registering, users accept this implicit user agreement.

In order to shared resources in a fair way, a number of site policies have been implemented:

1. Single user node policy
2. Priority based scheduling
3. Fairshare mechanism

9.1 Single user node policy

Each user will have exclusive access to the nodes that are assigned to him. This means that no jobs from other users will be executed on the nodes during your computations. Of course, it is very likely that multiple jobs from the same user are being sent to the same node at the same time, in order to maximise the utilisation of the remaining available resources.

The motivation behind this "one node, one user" policy is:

1. Parallel jobs can suffer badly if they don't have exclusive access to the full node.
2. Sometimes the wrong job gets killed if a user uses more memory than requested and a node runs out of memory. With this policy, a user cannot get affected by the malicious software from other users.
3. A user may accidentally spawn more processes/threads and use more cores than expected (depends on the OS).

The scheduler will (try to) fill up a node with several jobs from the same user. If you don't have enough work for a single node, you need a good PC and not a supercomputer.

9.2 Priority based scheduling

The scheduler associates a priority number to each job:

1. the highest priority job will (usually) be the next one to run;
2. jobs with a negative priority will (temporarily) be blocked.

Currently, the priority calculation is based on:

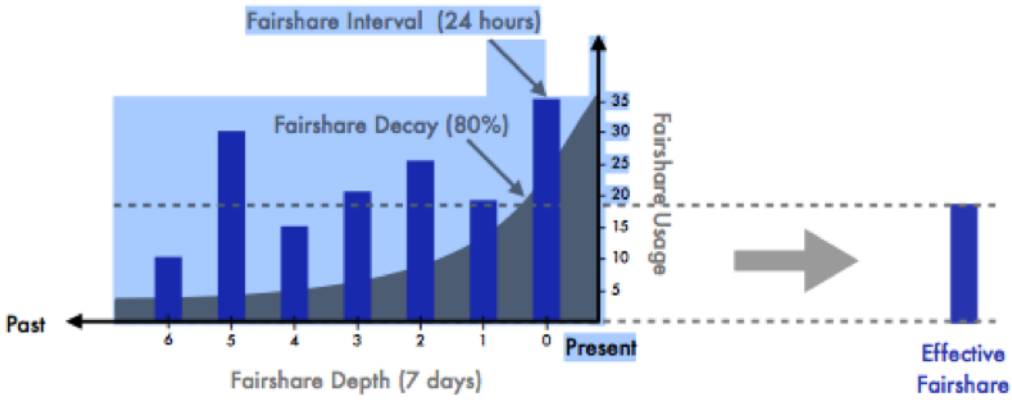
1. the time a job is queued: the priority of a job is increased in accordance to the time (in minutes) it is waiting in the queue to get started;
2. a "*Fairshare*" window of 7 days with a target of 12%: the priority of jobs of a user who has used too many resources over the current "*Fairshare*" window is lowered.

The Priority system can of course be adapted in the future, but this will be communicated.

9.3 Fairshare mechanism

A "*Fairshare*" system has been setup to allow all users to get their fair share of the UAntwerpen-HPC utilization. The mechanism incorporates historical resource utilization over the last week. A users' "Fairshare" index is used to adapt its job priority in the queue. It only affects its job's priority relative to other available jobs. When there are no other competing jobs, the job will immediately run.

As such, the "*Fairshare*" mechanism has nothing to do with the allocation of computing time.



Chapter 10

Frequently Asked Questions

10.1 When will my job start?

You can use the `showstart` command. For more information, see section 4.4.

10.2 Can I share my account with someone else?

NO. You are not allowed to share your VSC account with anyone else, it is strictly personal. See https://pintra.uantwerpen.be/bbcswebdav/xid-23610_1

10.3 Can I share my data with other UAntwerpen-HPC users?

Yes, you can use the `chmod` or `setfacl` commands to change permissions of files so other users can access the data. For example, the following command will enable a user named "otheruser" to read the file named `dataset.txt`. See

```
$ setfacl -m u:otheruser:r dataset.txt
$ ls -l dataset.txt
-rwxr-x---+ 2 vsc20167 mygroup      40 Apr 12 15:00 dataset.txt
```

For more information about `chmod` or `setfacl`, see the section on chmod in chapter 3 of the Linux intro manual.

10.4 Can I use multiple different SSH key pairs to connect to my VSC account?

Yes, and this is recommended when working from different computers. Please see subsection 2.2.3 on how to do this.

10.5 I want to use software that is not available on the clusters yet

Please send an e-mail to hpc@uantwerpen.be that includes:

- What software you want to install and the required version
- Detailed installation instructions
- The purpose for which you want to install the software

Part II

Advanced Guide

Chapter 11

Fine-tuning Job Specifications

As UAntwerpen-HPC system administrators, we often observe that the UAntwerpen-HPC resources are not optimally (or wisely) used. For example, we regularly notice that several cores on a computing node are not utilised, due to the fact that one sequential program uses only one core on the node. Or users run I/O intensive applications on nodes with "slow" network connections.

Users often tend to run their jobs without specifying specific PBS Job parameters. As such, their job will automatically use the default parameters, which are not necessarily (or rarely) the optimal ones. This can slow down the run time of your application, but also block UAntwerpen-HPC resources for other users.

Specifying the "optimal" Job Parameters requires some knowledge of your application (e.g., how many parallel threads does my application uses, is there a lot of inter-process communication, how much memory does my application need) and also some knowledge about the UAntwerpen-HPC infrastructure (e.g., what kind of multi-core processors are available, which nodes have InfiniBand).

There are plenty of monitoring tools on Linux available to the user, which are useful to analyse your individual application. The UAntwerpen-HPC environment as a whole often requires different techniques, metrics and time goals, which are not discussed here. We will focus on tools that can help to optimise your Job Specifications.

Determining the optimal computer resource specifications can be broken down into different parts. The first is actually determining which metrics are needed and then collecting that data from the hosts. Some of the most commonly tracked metrics are CPU usage, memory consumption, network bandwidth, and disk I/O stats. These provide different indications of how well a system is performing, and may indicate where there are potential problems or performance bottlenecks. Once the data have actually been acquired, the second task is analysing the data and adapting your PBS Job Specifications.

Another different task is to monitor the behaviour of an application at run time and detect anomalies or unexpected behaviour. Linux provides a large number of utilities to monitor the performance of its components.

This chapter shows you how to measure:

1. Walltime

2. Memory usage
3. CPU usage
4. Disk (storage) needs
5. Network bottlenecks

First, we allocate a compute node and move to our relevant directory:

```
$ qsub -I
$ cd ~/examples/Fine-tuning-Job-Specifications
```

11.1 Specifying Walltime

One of the most important and also easiest parameters to measure is the duration of your program. This information is needed to specify the *walltime*.

The *time* utility **executes** and **times** your application. You can just add the time command in front of your normal command line, including your command line options. After your executable has finished, **time** writes the total time elapsed, the time consumed by system overhead, and the time used to execute your executable to the standard error stream. The calculated times are reported in seconds.

Test the time command:

```
$ time sleep 75
real 1m15.005s
user 0m0.001s
sys 0m0.002s
```

It is a good practice to correctly estimate and specify the run time (duration) of an application. Of course, a margin of 10% to 20% can be taken to be on the safe side.

It is also wise to check the walltime on different compute nodes or to select the "slowest" compute node for your walltime tests. Your estimate should appropriate in case your application will run on the "slowest" (oldest) compute nodes.

The walltime can be specified in a job scripts as:

```
#PBS -l walltime=3:00:00:00
```

or on the command line

```
$ qsub -l walltime=3:00:00:00
```

It is recommended to always specify the walltime for a job.

11.2 Specifying memory requirements

In many situations, it is useful to monitor the amount of memory an application is using. You need this information to determine the characteristics of the required compute node, where that

application should run on. Estimating the amount of memory an application will use during execution is often non-trivial, especially when one uses third-party software.

The "eat_mem" application in the HPC examples directory just consumes and then releases memory, for the purpose of this test. It has one parameter, the amount of gigabytes of memory which needs to be allocated.

First compile the program on your machine and then test it for 1 GB:

```
$ gcc -o eat_mem eat_mem.c
$ ./eat_mem 1
Consuming 1 gigabyte of memory.
```

11.2.1 Available Memory on the machine

The first point is to be aware of the available free memory in your computer. The "*free*" command displays the total amount of free and used physical and swap memory in the system, as well as the buffers used by the kernel. We also use the options "-m" to see the results expressed in Mega-Bytes and the "-t" option to get totals.

```
$ free -m -t
                total   used   free  shared  buffers  cached
Mem:            16049   4772  11277       0      107     161
-/+ buffers/cache:      4503  11546
Swap:           16002   4185  11816
Total:          32052   8957  23094
```

Important is to note the total amount of memory available in the machine (i.e., 16 GB in this example) and the amount of used and free memory (i.e., 4.7 GB is used and another 11.2 GB is free here).

It is not a good practice to use swap-space for your computational applications. A lot of "swapping" can increase the execution time of your application tremendously.

11.2.2 Checking the memory consumption

The "Monitor" tool monitors applications in terms of memory and CPU usage, as well as the size of temporary files. Note that currently only single node jobs are supported, MPI support may be added in a future release.

To start using monitor, first load the appropriate module. Then we study the "eat_mem.c" program and compile it:

```
$ module load monitor
$ cat eat_mem.c
$ gcc -o eat_mem eat_mem.c
```

Starting a program to monitor is very straightforward; you just add the "monitor" command before the regular command line.

```
$ monitor ./eat_mem 3
time (s) size (kb) %mem %cpu
Consuming 3 gigabyte of memory.
5  252900 1.4 0.6
10  498592 2.9 0.3
15  743256 4.4 0.3
20  988948 5.9 0.3
25  1233612 7.4 0.3
30  1479304 8.9 0.2
35  1723968 10.4 0.2
40  1969660 11.9 0.2
45  2214324 13.4 0.2
50  2460016 14.9 0.2
55  2704680 16.4 0.2
60  2950372 17.9 0.2
65  3167280 19.2 0.2
70  3167280 19.2 0.2
75  9264  0 0.5
80  9264  0 0.4
```

Whereby:

1. The first column shows you the elapsed time in seconds. By default, all values will be displayed every 5 seconds.

2. The second column shows you the used memory in kb. We note that the memory slowly increases up to just over 3 GB (3GB is 3,145,728 KB), and is released again.

3. The third column shows the memory utilisation, expressed in percentages of the full available memory. At full memory consumption, 19.2% of the memory was being used by our application. With the *"free"* command, we have previously seen that we had a node of 16 GB in this example. 3 GB is indeed more or less 19.2% of the full available memory.

4. The fourth column shows you the CPU utilisation, expressed in percentages of a full CPU load. As there are no computations done in our exercise, the value remains very low (i.e., 0.2%).

Monitor will write the CPU usage and memory consumption of simulation to standard error.

This is the rate at which monitor samples the program's metrics. Since monitor's output may interfere with that of the program to monitor, it is often convenient to use a log file. The latter can be specified as follows:

```
$ monitor -l test1.log eat_mem 2
Consuming 2 gigabyte of memory.
$ cat test1.log
```

For long running programs, it may be convenient to limit the output to, e.g., the last minute of the programs execution. Since monitor provides metrics every 5 seconds, this implies we want to limit the output to the last 12 values to cover a minute:

```
$ monitor -l test2.log -n 12 eat_mem 4
Consuming 4 gigabyte of memory.
```

Note that this option is only available when monitor writes its metrics to a log file, not when standard error is used.

The interval at which monitor will show the metrics can be modified by specifying delta, the sample rate:

```
$ monitor -d 1 ./eat_mem
Consuming 3 gigabyte of memory.
```

Monitor will now print the program's metrics every second. Note that the minimum delta value is 1 second.

Alternative options to monitor the memory consumption are the "*top*" or the "*htop*" command.

top provides an ongoing look at processor activity in real time. It displays a listing of the most CPU-intensive tasks on the system, and can provide an interactive interface for manipulating processes. It can sort the tasks by memory usage, CPU usage and run time.

htop is similar to top, but shows the CPU-utilisation for all the CPUs in the machine and allows to scroll the list vertically and horizontally to see all processes and their full command lines.

```
$ top
$ htop
```

11.2.3 Setting the memory parameter

Once you gathered a good idea of the overall memory consumption of your application, you can define it in your job script. It is wise to foresee a margin of about 10%.

Sequential or single-node applications:

The maximum amount of physical memory used by the job can be specified in a job script as:

```
#PBS -l mem=4gb
```

or on the command line

```
$ qsub -l mem=4gb
```

This setting is ignored if the number of nodes is not 1.

Parallel or multi-node applications:

When you are running a parallel application over multiple cores, you can also specify the memory requirements per processor (pmem). This directive specifies the maximum amount of physical memory used by any process in the job.

For example, if the job would run four processes and each would use up to 2 GB (gigabytes) of memory, then the memory directive would read:

```
#PBS -l pmem=2gb
```

or on the command line

```
$ qsub -l pmem=2gb
```

(and of course this would need to be combined with a CPU cores directive such as nodes=1:ppn=4). In this example, you request 8 GB of memory in total on the node.

11.3 Specifying processors requirements

Users are encouraged to fully utilise all the available cores on a certain compute node. Once the required numbers of cores and nodes are decently specified, it is also good practice to monitor the CPU utilisation on these cores and to make sure that all the assigned nodes are working at full load.

11.3.1 Number of processors

The number of core and nodes that a user shall request fully depends on the architecture of the application. Developers design their applications with a strategy for parallelisation in mind. The application can be designed for a certain fixed number or for a configurable number of nodes and cores. It is wise to target a specific set of compute nodes (e.g., Westmere, Harpertown) for your computing work and then to configure your software to nicely fill up all processors on these compute nodes.

The */proc/cpuinfo* stores info about your CPU architecture like number of CPUs, threads, cores, information about CPU caches, CPU family, model and much more. So, if you want to detect how many cores are available on a specific machine:

```
$ less /proc/cpuinfo
processor       : 0
vendor_id       : GenuineIntel
cpu family      : 6
model           : 23
model name      : Intel(R) Xeon(R) CPU  E5420  @ 2.50GHz
stepping        : 10
cpu MHz         : 2500.088
cache size      : 6144 KB
...
```

Or if you want to see it in a more readable format, execute:

```
$ grep processor /proc/cpuinfo
processor : 0
processor : 1
processor : 2
processor : 3
processor : 4
processor : 5
processor : 6
processor : 7
```

Remark: Unless you want information of the login nodes, you'll have to issue these commands on one of the workernodes. This is most easily achieved in an interactive job, see the chapter on Running interactive jobs.

In order to specify the number of nodes and the number of processors per node in your job script, use:

```
#PBS -l nodes=N:ppn=M
```

or with equivalent parameters on the command line

```
$ qsub -l nodes=N:ppn=M
```

This specifies the number of nodes (nodes=N) and the number of processors per node (ppn=M) that the job should use. PBS treats a processor core as a processor, so a system with eight cores per compute node can have ppn=8 as its maximum ppn request.

Note that unless a job has some inherent parallelism of its own through something like MPI or OpenMP, requesting more than a single processor on a single node is usually wasteful and can impact the job start time.

11.3.2 Monitoring the CPU-utilisation

The previously used "monitor" tool also shows the overall CPU-load. The "eat_cpu" program performs a multiplication of 2 randomly filled a 1500×1500 matrices and is just written to consumes a lot of "*cpu*".

```
$ module load monitor
$ cat eat_cpu.c
$ gcc -o eat_cpu eat_cpu.c
```

And then start to monitor the *eat_ cpu* program:

```
$ monitor -d 1 ./eat_cpu
time   (s) size (kb) %mem %cpu
1   52852   0.3 100
2   52852   0.3 100
3   52852   0.3 100
4   52852   0.3 100
5   52852   0.3  99
6   52852   0.3 100
7   52852   0.3 100
8   52852   0.3 100
```

We notice that it the program keeps its CPU nicely busy at 100%.

Some processes spawn one or more sub-processes. In that case, the metrics shown by monitor are aggregated over the process and all of its sub-processes (recursively). The reported CPU usage is the sum of all these processes, and can thus exceed 100%.

Some (well, since this is a UAntwerpen-HPC Cluster, we hope most) programs use more than one core to perform their computations. Hence, it should not come as a surprise that the CPU usage is reported as larger than 100%. When programs of this type are running on a computer with n cores, the CPU usage can go up to $n \times 100\%$.

This could also be monitored with the ***htop*** command:

```
$ htop
```

```
  1  [|||   11.0%]   5  [||     3.0%]     9  [||     3.0%]   13 [       0.0%]
  2  [||||||100.0%]   6  [       0.0%]     10 [       0.0%]   14 [       0.0%]
  3  [||     4.9%]   7  [||     9.1%]     11 [       0.0%]   15 [       0.0%]
  4  [||     1.8%]   8  [       0.0%]     12 [       0.0%]   16 [       0.0%]
  Mem[|||||||||||||||||59211/64512MB]     Tasks: 323, 932 thr; 2 running
  Swp[||||||||||||      7943/20479MB]     Load average: 1.48 1.46 1.27
                                          Uptime: 211 days(!), 22:12:58

  PID USER      PRI  NI  VIRT   RES   SHR S CPU% MEM%   TIME+  Command
22350 vsc00000   20   0 1729M 1071M   704 R 98.0  1.7 27:15.59 bwa index
 7703 root        0 -20 10.1G 1289M 70156 S 11.0  2.0 36h10:11 /usr/lpp/mmfs/bin
27905 vsc00000   20   0  123M  2800  1556 R  7.0  0.0  0:17.51 htop
```

The advantage of htop is that it shows you the cpu utilisation for all processors as well as the details per application. A nice exercise is to start 4 instances of the "cpu_eat" program in 4 different terminals, and inspect the cpu utilisation per processor with monitor and htop.

If ***htop*** reports that your program is taking 75% CPU on a certain processor, it means that 75% of the samples taken by top found your process active on the CPU. The rest of the time your application was in a wait. (It is important to remember that a CPU is a discrete state machine. It really can be at only 100%, executing an instruction, or at 0%, waiting for something to do. There is no such thing as using 45% of a CPU. The CPU percentage is a function of time.) However, it is likely that your application's rest periods include waiting to be dispatched on a CPU and not on external devices. That part of the wait percentage is then very relevant to understanding your overall CPU usage pattern.

11.3.3 Fine-tuning your executable and/or job script

It is good practice to perform a number of run time stress tests, and to check the CPU utilisation of your nodes. We (and all other users of the UAntwerpen-HPC) would appreciate that you use the maximum of the CPU resources that are assigned to you and make sure that there are no CPUs in your node who are not utilised without reasons.

But how can you maximise?

1. Configure your software. (e.g., to exactly use the available amount of processors in a node)
2. Develop your parallel program in a smart way.
3. Demand a specific type of compute node (e.g., Harpertown, Westmere), which have a specific number of cores.
4. Correct your request for CPUs in your job script.

11.4 The system load

On top of the CPU utilisation, it is also important to check the **system load**. The system **load** is a measure of the amount of computational work that a computer system performs.

The system load is the number of applications running or waiting to run on the compute node. In a system with for example four CPUs, a load average of 3.61 would indicate that there were, on average, 3.61 processes ready to run, and each one could be scheduled into a CPU.

The load averages differ from CPU percentage in two significant ways:

1. "*load averages*" measure the trend of processes waiting to be run (and not only an instantaneous snapshot, as does CPU percentage); and
2. "*load averages*" include all demand for all resources, e.g., CPU and also I/O and network (and not only how much was active at the time of measurement).

11.4.1 Optimal load

What is the "*optimal load*" rule of thumb?

The load averages tell us whether our physical CPUs are over- or under-utilised. The **point of perfect utilisation**, meaning that the CPUs are always busy and, yet, no process ever waits for one, is **the average matching the number of CPUs**. Your load should not exceed the number of cores available. E.g., if there are four CPUs on a machine and the reported one-minute load average is 4.00, the machine has been utilising its processors perfectly for the last 60 seconds. The "100% utilisation" mark is 1.0 on a single-core system, 2.0 on a dual-core, 4.0 on a quad-core, etc. The optimal load shall be between 0.7 and 1.0 per processor.

In general, the intuitive idea of load averages is the higher they rise above the number of processors, the more processes are waiting and doing nothing, and the lower they fall below the number of processors, the more untapped CPU capacity there is.

Load averages do include any processes or threads waiting on I/O, networking, databases or anything else not demanding the CPU. This means that the optimal *number of applications* running on a system at the same time, might be more than one per processor.

The "**optimal number of applications**" running on one machine at the same time depends on the type of the applications that you are running.

1. When you are running **computational intensive applications**, one application per processor will generate the optimal load.
2. For **I/O intensive applications** (e.g., applications which perform a lot of disk-I/O), a higher number of applications can generate the optimal load. While some applications are reading or writing data on disks, the processors can serve other applications.

The optimal number of applications on a machine could be empirically calculated by performing a number of stress tests, whilst checking the highest throughput. There is however no manner in the UAntwerpen-HPC at the moment to specify the maximum number of applications that shall run per core dynamically. The UAntwerpen-HPC scheduler will not launch more than one process per core.

The manner how the cores are spread out over CPUs does not matter for what regards the load. Two quad-cores perform similar to four dual-cores, and again perform similar to eight single-cores. It's all eight cores for these purposes.

11.4.2 Monitoring the load

The **load average** represents the average system load over a period of time. It conventionally appears in the form of three numbers, which represent the system load during the last **one**-, **five**-, and **fifteen**-minute periods.

The **uptime** command will show us the average load

```
$ uptime
10:14:05 up 86 days, 12:01, 11 users, load average: 0.60, 0.41, 0.41
```

Now, start a few instances of the "*eat_ cpu*" program in the background, and check the effect on the load again:

```
$ ./eat_cpu&
$ ./eat_cpu&
$ ./eat_cpu&
$ uptime
10:14:42 up 86 days, 12:02, 11 users, load average: 2.60, 0.93, 0.58
```

You can also read it in the **htop** command.

11.4.3 Fine-tuning your executable and/or job script

It is good practice to perform a number of run time stress tests, and to check the system load of your nodes. We (and all other users of the UAntwerpen-HPC) would appreciate that you use the maximum of the CPU resources that are assigned to you and make sure that there are no CPUs in your node who are not utilised without reasons.

But how can you maximise?

1. Profile your software to improve its performance.
2. Configure your software (e.g., to exactly use the available amount of processors in a node).
3. Develop your parallel program in a smart way, so that it fully utilises the available processors.
4. Demand a specific type of compute node (e.g., Harpertown, Westmere), which have a specific number of cores.
5. Correct your request for CPUs in your job script.

And then check again.

11.5 Checking File sizes & Disk I/O

11.5.1 Monitoring File sizes during execution

Some programs generate intermediate or output files, the size of which may also be a useful metric.

Remember that your available disk space on the UAntwerpen-HPC online storage is limited, and that you have environment variables which point to these directories available (i.e., *$VSC_DATA*, *$VSC_SCRATCH* and *$VSC_DATA*). On top of those, you can also access some temporary storage (i.e., the /tmp directory) on the compute node, which is defined by the *$VSC_SCRATCH_NODE* environment variable.

We first load the monitor modules, study the "eat_disk.c" program and compile it:

```
$ module load monitor
$ cat eat_disk.c
$ gcc -o eat_disk eat_disk.c
```

The *monitor* tool provides an option (-f) to display the size of one or more files:

```
$ monitor -f $VSC_SCRATCH/test.txt ./eat_disk
time (s) size (kb) %mem %cpu
5  1276  0 38.6 168820736
10  1276  0 24.8 238026752
15  1276  0 22.8 318767104
20  1276  0 25 456130560
25  1276  0 26.9 614465536
30  1276  0 27.7 760217600
...
```

Here, the size of the file "*test.txt*" in directory $VSC_SCRATCH will be monitored. Files can be specified by absolute as well as relative path, and multiple files are separated by ",".

It is important to be aware of the sizes of the file that will be generated, as the available disk space for each user is limited. We refer to section 6.5 on "Quotas" to check your quota and tools to find which files consumed the "quota".

Several actions can be taken, to avoid storage problems:

1. Be aware of all the files that are generated by your program. Also check out the hidden files.
2. Check your quota consumption regularly.
3. Clean up your files regularly.
4. First work (i.e., read and write) with your big files in the local /tmp directory. Once finished, you can move your files once to the VSC_DATA directories.
5. Make sure your programs clean up their temporary files after execution.
6. Move your output results to your own computer regularly.
7. Anyone can request more disk space to the UAntwerpen-HPC staff, but you will have to duly justify your request.

11.6 Specifying network requirements

Users can examine their network activities with the htop command. When your processors are 100% busy, but you see a lot of red bars and only limited green bars in the htop screen, it is

mostly an indication that they loose a lot of time with inter-process communication.

Whenever your application utilises a lot of inter-process communication (as is the case in most parallel programs), we strongly recommend to request nodes with an "InfiniBand" network. The InfiniBand is a specialised high bandwidth, low latency network that enables large parallel jobs to run as efficiently as possible.

The parameter to add in your job script would be:

```
#PBS -l ib
```

If for some other reasons, a user is fine with the gigabit Ethernet network, he can specify:

```
#PBS -l gbe
```

11.7 Some more tips on the Monitor tool

11.7.1 Command Lines arguments

Many programs, e.g., MATLAB, take command line options. To make sure these do not interfere with those of monitor and vice versa, the program can for instance be started in the following way:

```
$ monitor -delta 60 - matlab -nojvm -nodisplay computation.m
```

The use of `--` will ensure that monitor does not get confused by MATLAB's `-nojvm` and `-nodisplay` options.

11.7.2 Exit Code

Monitor will propagate the exit code of the program it is watching. Suppose the latter ends normally, then monitor's exit code will be 0. On the other hand, when the program terminates abnormally with a non-zero exit code, e.g., 3, then this will be monitor's exit code as well.

When monitor terminates in an abnormal state, for instance if it can't create the log file, its exit code will be 65. If this interferes with an exit code of the program to be monitored, it can be modified by setting the environment variable MONITOR_EXIT_ERROR to a more suitable value.

11.7.3 Monitoring a running process

It is also possible to "attach" monitor to a program or process that is already running. One simply determines the relevant process ID using the ps command, e.g., 18749, and starts monitor:

```
$ monitor -p 18749
```

Note that this feature can be (ab)used to monitor specific sub-processes.

Chapter 12

Multi-job submission

A frequent occurring characteristic of scientific computation is their focus on data intensive processing. A typical example is the iterative evaluation of a program over different input parameter values, often referred to as a "*parameter sweep*". A **Parameter Sweep** runs a job a specified number of times, as if we sweep the parameter values through a user defined range.

Users then often want to submit a large numbers of jobs based on the same job script but with (i) slightly different parameters settings or with (ii) different input files.

These parameter values can have many forms, we can think about a range (e.g., from 1 to 100), or the parameters can be stored line by line in a comma-separated file. The users want to run their job once for each instance of the parameter values.

One option could be to launch a lot of separate individual small jobs (one for each parameter) on the cluster, but this is not a good idea. The cluster scheduler isn't meant to deal with tons of small jobs. Those huge amounts of small jobs will create a lot of overhead, and can slow down the whole cluster. It would be better to bundle those jobs in larger sets. In TORQUE, an experimental feature known as "*job arrays*" existed to allow the creation of multiple jobs with one *qsub* command, but is was not supported by Moab, the current scheduler.

The "**Worker framework**" has been developed to address this issue.

It can handle many small jobs determined by:

parameter variations i.e., many small jobs determined by a specific parameter set which is stored in a .csv (comma separated value) input file.

job arrays i.e., each individual job got a unique numeric identifier.

Both use cases often have a common root: the user wants to run a program with a large number of parameter settings, and the program does not allow for aggregation, i.e., it has to be run once for each instance of the parameter values.

However, the Worker Framework's scope is wider: it can be used for any scenario that can be reduced to a **MapReduce** approach.[1]

[1] MapReduce: 'Map' refers to the map pattern in which every item in a collection is mapped onto a new value by applying a given function, while "reduce" refers to the reduction pattern which condenses or reduces a collection of previously computed results to a single value.

12.1 The worker Framework: Parameter Sweeps

First go to the right directory:

```
$ cd ~/examples/Multi-job-submission/par_sweep
```

Suppose the program the user wishes to run the "*weather*" program, which takes three parameters: a temperature, a pressure and a volume. A typical call of the program looks like:

```
$ ./weather -t 20 -p 1.05 -v 4.3
T: 20  P: 1.05  V: 4.3
```

For the purpose of this exercise, the weather program is just a simple bash script, which prints the 3 variables to the standard output and waits a bit:

weather– par_sweep/weather —

```
1 #!/bin/bash
2 # Here you could do your calculations
3 echo "T: $2  P: $4  V: $6"
4 sleep 100
```

A job script that would run this as a job for the first parameters (p01) would then look like:

weather_p01.pbs– par_sweep/weather_p01.pbs —

```
1 #!/bin/bash
2
3 #PBS -l nodes=1:ppn=8
4 #PBS -l walltime=01:00:00
5
6 cd $PBS_O_WORKDIR
7 ./weather -t 20 -p 1.05 -v 4.3
```

When submitting this job, the calculation is performed or this particular instance of the parameters, i.e., temperature = 20, pressure = 1.05, and volume = 4.3.

To submit the job, the user would use:

```
$ qsub weather_p01.pbs
```

However, the user wants to run this program for many parameter instances, e.g., he wants to run the program on 100 instances of temperature, pressure and volume. The 100 parameter instances can be stored in a comma separated value file (.csv) that can be generated using a spreadsheet program such as Microsoft Excel or RDBMS or just by hand using any text editor (do **not** use a word processor such as Microsoft Word). The first few lines of the file "*data.csv*" would look like:

```
$ more data.csv
temperature, pressure, volume
293, 1.0e5, 107
294, 1.0e5, 106
295, 1.0e5, 105
296, 1.0e5, 104
297, 1.0e5, 103
...
```

It has to contain the names of the variables on the first line, followed by 100 parameter instances in the current example.

In order to make our PBS generic, the PBS file can be modified as follows:

weather.pbs– par_sweep/weather.pbs —

```
1  #!/bin/bash
2
3  #PBS -l nodes=1:ppn=8
4  #PBS -l walltime=04:00:00
5
6  cd $PBS_O_WORKDIR
7  ./weather -t $temperature -p $pressure -v $volume
8
9  # # This script is submitted to the cluster with the following 2 commands:
10 # module load worker/1.6.8-intel-2018a
11 # wsub -data data.csv -batch weather.pbs
```

Note that:

1. the parameter values 20, 1.05, 4.3 have been replaced by variables $temperature, $pressure and $volume respectively, which were being specified on the first line of the "*data.csv*" file;
2. the number of processors per node has been increased to 8 (i.e., ppn=1 is replaced by ppn=8);
3. the walltime has been increased to 4 hours (i.e., walltime=00:15:00 is replaced by walltime=04:00:00).

The walltime is calculated as follows: one calculation takes 15 minutes, so 100 calculations take 1500 minutes on one CPU. However, this job will use 8 CPUs, so the 100 calculations will be done in 1500/8 = 187.5 minutes, i.e., 4 hours to be on the safe side.

The job can now be submitted as follows (to check which `worker` module to use, see subsection 4.1.7):

```
$ module load worker/1.6.8-intel-2018a
$ wsub -batch weather.pbs -data data.csv
total number of work items: 41
433253.leibniz
```

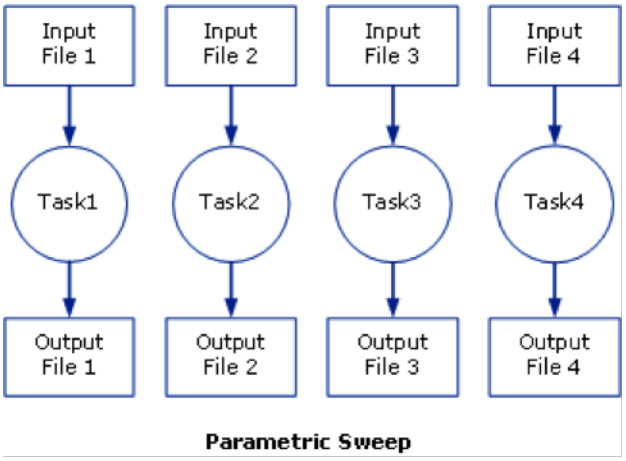
Note that the PBS file is the value of the -batch option. The weather program will now be run for all 100 parameter instances – 8 concurrently – until all computations are done. A computation for such a parameter instance is called a work item in Worker parlance.

12.2 The Worker framework: Job arrays

First go to the right directory:

```
$ cd ~/examples/Multi-job-submission/job_array
```

As a simple example, assume you have a serial program called *myprog* that you want to run on various input files *input[1-100]*.



The following bash script would submit these jobs all one by one:

```
1 #!/bin/bash
2 for i in `seq 1 100`; do
3    qsub -o output $i -i input $i myprog.pbs
4 done
```

This, as said before, could be disturbing for the job scheduler.

Alternatively, TORQUE provides a feature known as *job arrays* which allows the creation of multiple, similar jobs with only **one qsub** command. This feature introduced a new job naming convention that allows users either to reference the entire set of jobs as a unit or to reference one particular job from the set.

Under TORQUE, the *-t range* option is used with qsub to specify a job array, where *range* is a range of numbers (e.g., *1-100* or *2,4-5,7*).

The details are

1. a job is submitted for each *number* in the range;
2. individuals jobs are referenced as *jobid-number*, and the entire array can be referenced as *jobid* for easy killing etc.; and
3. each jobs has *PBS_ARRAYID* set to its *number* which allows the script/program to specialise for that job

The job could have been submitted using:

```
$ qsub -t 1-100 my_prog.pbs
```

The effect was that rather than 1 job, the user would actually submit 100 jobs to the queue system. This was a popular feature of TORQUE, but as this technique puts quite a burden on the scheduler, it is not supported by Moab (the current job scheduler).

To support those users who used the feature and since it offers a convenient workflow, the "worker framework" implements the idea of "job arrays" in its own way.

A typical job script for use with job arrays would look like this:

job_array.pbs– job_array/job_array.pbs —

```
1 #!/bin/bash -l
2 #PBS -l nodes=1:ppn=1
3 #PBS -l walltime=00:15:00
4 cd $PBS_O_WORKDIR
5 INPUT_FILE="input_${PBS_ARRAYID}.dat"
6 OUTPUT_FILE="output_${PBS_ARRAYID}.dat"
7 my_prog -input ${INPUT_FILE}  -output ${OUTPUT_FILE}
```

In our specific example, we have prefabricated 100 input files in the "./input" subdirectory. Each of those files contains a number of parameters for the "test_set" program, which will perform some tests with those parameters.

Input for the program is stored in files with names such as input_1.dat, input_2.dat, ..., input_100.dat in the ./input subdirectory.

```
$ ls ./input
...
$ more ./input/input_99.dat
This is input file \#99
Parameter #1 = 99
Parameter #2 = 25.67
Parameter #3 = Batch
Parameter #4 = 0x562867
```

For the sole purpose of this exercise, we have provided a short "test_set" program, which reads the "input" files and just copies them into a corresponding output file. We even add a few lines to each output file. The corresponding output computed by our "*test_set*" program will be written to the "*./output*" directory in output_1.dat, output_2.dat, ..., output_100.dat. files.

test_set– job_array/test_set —

```
#!/bin/bash

# Check if the output Directory exists
if [ ! -d "./output" ] ; then
  mkdir ./output
fi

#   Here you could do your calculations...
echo "This is Job_array #" $1
echo "Input File : " $3
echo "Output File: " $5
cat ./input/$3 | sed -e "s/input/output/g" | grep -v "Parameter" > ./output/$5
echo "Calculations done, no results" >> ./output/$5
```

Using the "worker framework", a feature akin to job arrays can be used with minimal modifications to the job script:

test_set.pbs– job_array/test_set.pbs —

```
#!/bin/bash -l
#PBS -l nodes=1:ppn=8
#PBS -l walltime=04:00:00
cd $PBS_O_WORKDIR
INPUT_FILE="input_${PBS_ARRAYID}.dat"
OUTPUT_FILE="output_${PBS_ARRAYID}.dat"
./test_set ${PBS_ARRAYID} -input ${INPUT_FILE}  -output ${OUTPUT_FILE}
```

Note that

1. the number of CPUs is increased to 8 (ppn=1 is replaced by ppn=8); and

2. the walltime has been modified (walltime=00:15:00 is replaced by walltime=04:00:00).

The job is now submitted as follows:

```
$ module load worker/1.6.8-intel-2018a
$ wsub -t 1-100 -batch test_set.pbs
total number of work items: 100
433253.leibniz
```

The "*test_set*" program will now be run for all 100 input files – 8 concurrently – until all computations are done. Again, a computation for an individual input file, or, equivalently, an array id, is called a work item in Worker speak.

Note that in contrast to TORQUE job arrays, a worker job array only submits a single job.

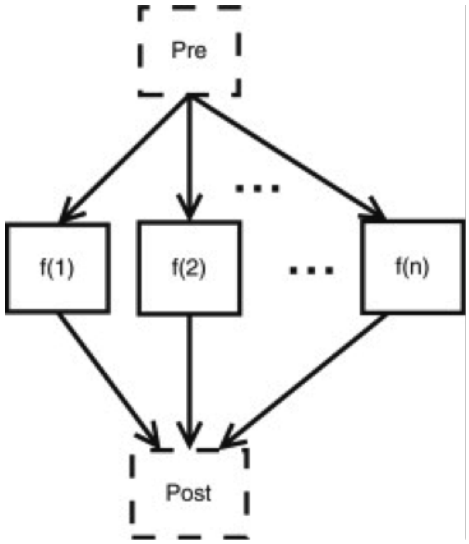
```
$ qstat
Job id          Name          User      Time   Use S Queue
--------------- ------------- --------- ---- ----- - -----
433253.leibniz  test_set.pbs  vsc20167          0 Q

And you can now check the generated output files:
$ more ./output/output_99.dat
This is output file #99
Calculations done, no results
```

12.3 MapReduce: prologues and epilogue

Often, an embarrassingly parallel computation can be abstracted to three simple steps:

1. a preparation phase in which the data is split up into smaller, more manageable chunks;
2. on these chunks, the same algorithm is applied independently (these are the work items); and
3. the results of the computations on those chunks are aggregated into, e.g., a statistical description of some sort.



The Worker framework directly supports this scenario by using a prologue (pre-processing) and an epilogue (post-processing). The former is executed just once before work is started on the work items, the latter is executed just once after the work on all work items has finished. Technically, the master, i.e., the process that is responsible for dispatching work and logging progress, executes the prologue and epilogue.

```
$ cd ~/examples/Multi-job-submission/map_reduce
```

The script "pre.sh" prepares the data by creating 100 different input-files, and the script "post.sh" aggregates (concatenates) the data.

First study the scripts:

pre.sh– map_reduce/pre.sh —

```
#!/bin/bash

# Check if the input Directory exists
if [ ! -d "./input" ] ; then
  mkdir ./input
fi

# Just generate all dummy input files
for i in {1..100};
do
  echo "This is input file #$i" >  ./input/input_$i.dat
  echo "Parameter #1 = $i" >>  ./input/input_$i.dat
  echo "Parameter #2 = 25.67" >>  ./input/input_$i.dat
  echo "Parameter #3 = Batch" >>  ./input/input_$i.dat
  echo "Parameter #4 = 0x562867" >>  ./input/input_$i.dat
done
```

post.sh– map_reduce/post.sh —

```
#!/bin/bash

# Check if the input Directory exists
if [ ! -d "./output" ] ; then
  echo "The output directory does not exist!"
  exit
fi

# Just concatenate all output files
touch all_output.txt
for i in {1..100};
do
  cat ./output/output_$i.dat >> all_output.txt
done
```

Then one can submit a MapReduce style job as follows:

```
$ wsub -prolog pre.sh -batch test_set.pbs -epilog post.sh -t 1-100
total number of work items: 100
433253.leibniz
$ cat all_output.txt
...
$ rm -r -f ./output/
```

Note that the time taken for executing the prologue and the epilogue should be added to the job's total walltime.

12.4 Some more on the Worker Framework

12.4.1 Using Worker efficiently

The "Worker Framework" is implemented using MPI, so it is not restricted to a single compute nodes, it scales well to multiple nodes. However, remember that jobs requesting a large number of nodes typically spend quite some time in the queue.

The "Worker Framework" will be effective when

1. work items, i.e., individual computations, are neither too short, nor too long (i.e., from a few minutes to a few hours); and,
2. when the number of work items is larger than the number of CPUs involved in the job (e.g., more than 30 for 8 CPUs).

12.4.2 Monitoring a worker job

Since a Worker job will typically run for several hours, it may be reassuring to monitor its progress. Worker keeps a log of its activity in the directory where the job was submitted. The log's name is derived from the job's name and the job's ID, i.e., it has the form `<jobname>.log<jobid>`. For the running example, this could be `run.pbs.log433253`, assuming the job's ID is 433253. To keep an eye on the progress, one can use:

```
$ tail -f run.pbs.log433253
```

Alternatively, `wsummarize`, a Worker command that summarises a log file, can be used:

```
$ watch -n 60 wsummarize run.pbs.log433253
```

This will summarise the log file every 60 seconds.

12.4.3 Time limits for work items

Sometimes, the execution of a work item takes long than expected, or worse, some work items get stuck in an infinite loop. This situation is unfortunate, since it implies that work items that could successfully execute are not even started. Again, the Worker framework offers a simple and yet versatile solution. If we want to limit the execution of each work item to at most 20 minutes, this can be accomplished by modifying the script of the running example.

```
1 #!/bin/bash -l
2 #PBS -l nodes=1:ppn=8
3 #PBS -l walltime=04:00:00
4 module load timedrun/1.0
5 cd $PBS_O_WORKDIR
6 timedrun -t 00:20:00 weather -t $temperature  -p $pressure  -v $volume
```

Note that it is trivial to set individual time constraints for work items by introducing a parameter, and including the values of the latter in the CSV file, along with those for the temperature, pressure and volume.

Also note that "timedrun" is in fact offered in a module of its own, so it can be used outside the Worker framework as well.

12.4.4 Resuming a Worker job

Unfortunately, it is not always easy to estimate the walltime for a job, and consequently, sometimes the latter is underestimated. When using the Worker framework, this implies that not all work items will have been processed. Worker makes it very easy to resume such a job without having to figure out which work items did complete successfully, and which remain to be computed. Suppose the job that did not complete all its work items had ID "445948".

```
$ wresume -jobid 433253
```

This will submit a new job that will start to work on the work items that were not done yet. Note that it is possible to change almost all job parameters when resuming, specifically the requested resources such as the number of cores and the walltime.

```
$ wresume -l walltime=1:30:00 -jobid 433253
```

Work items may fail to complete successfully for a variety of reasons, e.g., a data file that is missing, a (minor) programming error, etc. Upon resuming a job, the work items that failed are considered to be done, so resuming a job will only execute work items that did not terminate either successfully, or reporting a failure. It is also possible to retry work items that failed (preferably after the glitch why they failed was fixed).

```
$ wresume -jobid 433253 -retry
```

By default, a job's prologue is not executed when it is resumed, while its epilogue is. "wresume" has options to modify this default behaviour.

12.4.5 Further information

This how-to introduces only Worker's basic features. The wsub command has some usage information that is printed when the -help option is specified:

```
$ wsub -help
### usage: wsub  -batch <batch-file>          \
#                [-data <data-files>]         \
#                [-prolog <prolog-file>]      \
#                [-epilog <epilog-file>]      \
#                [-log <log-file>]            \
#                [-mpiverbose]                \
#                [-dryrun] [-verbose]         \
#                [-quiet] [-help]             \
#                [-t <array-req>]             \
#                [<pbs-qsub-options>]
#
#   -batch <batch-file>   : batch file template, containing variables to be
#                           replaced with data from the data file(s) or the
#                           PBS array request option
#   -data <data-files>    : comma-separated list of data files (default CSV
#                           files) used to provide the data for the work
#                           items
#   -prolog <prolog-file> : prolog script to be executed before any of the
#                           work items are executed
#   -epilog <epilog-file> : epilog script to be executed after all the work
#                           items are executed
#   -mpiverbose           : pass verbose flag to the underlying MPI program
#   -verbose              : feedback information is written to standard error
#   -dryrun               : run without actually submitting the job, useful
#   -quiet                : don't show information
#   -help                 : print this help message
#   -t <array-req>        : qsub's PBS array request options, e.g., 1-10
#   <pbs-qsub-options>    : options passed on to the queue submission
#                           command
```

Chapter 13

Compiling and testing your software on the HPC

All nodes in the UAntwerpen-HPC cluster are running the "CentOS Linux release 7.8.2003 (Core)" Operating system, which is a specific version of RedHat Enterprise Linux. This means that all the software programs (executable) that the end-user wants to run on the UAntwerpen-HPC first must be compiled for CentOS Linux release 7.8.2003 (Core). It also means that you first have to install all the required external software packages on the UAntwerpen-HPC.

Most commonly used compilers are already pre-installed on the UAntwerpen-HPC and can be used straight away. Also many popular external software packages, which are regularly used in the scientific community, are also pre-installed.

13.1 Check the pre-installed software on the UAntwerpen-HPC

In order to check all the available modules and their version numbers, which are pre-installed on the UAntwerpen-HPC enter:

```
$ module av 2>&1 | more
------------- /apps/antwerpen/modules/hopper/2015a/all ------------
ABINIT/7.10.2-intel-2015a
ADF/2014.05
Advisor/2015_update1
Bison/3.0.4-intel-2015a
Boost/1.57.0-foss-2015a-Python-2.7.9
Boost/1.57.0-intel-2015a-Python-2.7.9
bzip2/1.0.6-foss-2015a
bzip2/1.0.6-intel-2015a
...
```

Or when you want to check whether some specific software, some compiler or some application (e.g., LAMMPS) is installed on the UAntwerpen-HPC.

```
$ module av 2>&1 | grep -i -e "LAMMPS"
LAMMPS/9Dec14-intel-2015a
LAMMPS/30Oct14-intel-2014a
LAMMPS/5Sep14-intel-2014a
```

As you are not aware of the capitals letters in the module name, we looked for a case-insensitive name with the "-i" option.

When your required application is not available on the UAntwerpen-HPC please contact any UAntwerpen-HPC member. Be aware of potential "License Costs". "Open Source" software is often preferred.

13.2 Porting your code

To **port** a software-program is to translate it from the operating system in which it was developed (e.g., Windows 7) to another operating system (e.g., RedHat Enterprise Linux on our UAntwerpen-HPC) so that it can be used there. Porting implies some degree of effort, but not nearly as much as redeveloping the program in the new environment. It all depends on how "portable" you wrote your code.

In the simplest case the file or files may simply be copied from one machine to the other. However, in many cases the software is installed on a computer in a way, which depends upon its detailed hardware, software, and setup, with device drivers for particular devices, using installed operating system and supporting software components, and using different directories.

In some cases software, usually described as "portable software" is specifically designed to run on different computers with compatible operating systems and processors without any machine-dependent installation; it is sufficient to transfer specified directories and their contents. Hardware- and software-specific information is often stored in configuration files in specified locations (e.g., the registry on machines running MS Windows).

Software, which is not portable in this sense, will have to be transferred with modifications to support the environment on the destination machine.

Whilst programming, it would be wise to stick to certain standards (e.g., ISO/ANSI/POSIX). This will ease the porting of your code to other platforms.

Porting your code to the CentOS Linux release 7.8.2003 (Core) platform is the responsibility of the end-user.

13.3 Compiling and building on the UAntwerpen-HPC

Compiling refers to the process of translating code written in some programming language, e.g., Fortran, C, or C++, to machine code. Building is similar, but includes gluing together the machine code resulting from different source files into an executable (or library). The text below guides you through some basic problems typical for small software projects. For larger projects it is more appropriate to use makefiles or even an advanced build system like CMake.

All the UAntwerpen-HPC nodes run the same version of the Operating System, i.e. CentOS Linux release 7.8.2003 (Core). So, it is sufficient to compile your program on any compute node. Once you have generated an executable with your compiler, this executable should be able to run on any other compute-node.

A typical process looks like:

1. Copy your software to the login-node of the UAntwerpen-HPC
2. Start an interactive session on a compute node;
3. Compile it;
4. Test it locally;
5. Generate your job scripts;
6. Test it on the UAntwerpen-HPC
7. Run it (in parallel);

We assume you've copied your software to the UAntwerpen-HPC The next step is to request your private compute node.

```
$ qsub -I
qsub: waiting for job 433253.leibniz to start
```

13.3.1 Compiling a sequential program in C

Go to the examples for chapter 13 and load the foss module:

```
$ cd ~/examples/Compiling-and-testing-your-software-on-the-HPC
$ module load foss
```

We now list the directory and explore the contents of the "*hello.c*" program:

```
$ ls -l
total 512
-rw-r--r-- 1 vsc20167 214 Sep 16 09:42 hello.c
-rw-r--r-- 1 vsc20167 130 Sep 16 11:39 hello.pbs*
-rw-r--r-- 1 vsc20167 359 Sep 16 13:55 mpihello.c
-rw-r--r-- 1 vsc20167 304 Sep 16 13:55 mpihello.pbs
```

— hello.c —

```
1  /*
2   * VSC        : Flemish Supercomputing Centre
3   * Tutorial   : Introduction to HPC
4   * Description: Print 500 numbers, whilst waiting 1 second in between
5   */
6  #include "stdio.h"
7  int main( int argc, char *argv[] )
8  {
9    int i;
10   for (i=0; i<500; i++)
11   {
12     printf("Hello #%d\n", i);
13     fflush(stdout);
14     sleep(1);
15   }
16 }
```

The "hello.c" program is a simple source file, written in C. It'll print 500 times "Hello #<num>", and waits one second between 2 printouts.

We first need to compile this C-file into an executable with the gcc-compiler.

First, check the command line options for *"gcc" (GNU C-Compiler)*, then we compile and list the contents of the directory again:

```
$ gcc -help
$ gcc -o hello hello.c
$ ls -l
total 512
-rwxrwxr-x 1 vsc20167 7116 Sep 16 11:43 hello*
-rw-r--r-- 1 vsc20167  214 Sep 16 09:42 hello.c
-rwxr-xr-x 1 vsc20167  130 Sep 16 11:39 hello.pbs*
```

A new file "hello" has been created. Note that this file has "execute" rights, i.e., it is an executable. More often than not, calling gcc – or any other compiler for that matter – will provide you with a list of errors and warnings referring to mistakes the programmer made, such as typos, syntax errors. You will have to correct them first in order to make the code compile. Warnings pinpoint less crucial issues that may relate to performance problems, using unsafe or obsolete language features, etc. It is good practice to remove all warnings from a compilation process, even if they seem unimportant so that a code change that produces a warning does not go unnoticed.

Let's test this program on the local compute node, which is at your disposal after the "qsub –I" command:

```
$ ./hello
Hello #0
Hello #1
Hello #2
Hello #3
Hello #4
...
```

It seems to work, now run it on the UAntwerpen-HPC

```
$ qsub hello.pbs
```

13.3.2 Compiling a parallel program in C/MPI

```
$ cd ~/examples/Compiling-and-testing-your-software-on-the-HPC
```

List the directory and explore the contents of the "*mpihello.c*" program:

```
$ ls -l
total 512
total 512
-rw-r--r-- 1 vsc20167 214 Sep 16 09:42 hello.c
-rw-r--r-- 1 vsc20167 130 Sep 16 11:39 hello.pbs*
-rw-r--r-- 1 vsc20167 359 Sep 16 13:55 mpihello.c
-rw-r--r-- 1 vsc20167 304 Sep 16 13:55 mpihello.pbs
```

— mpihello.c —

```c
/*
 * VSC        : Flemish Supercomputing Centre
 * Tutorial   : Introduction to HPC
 * Description: Example program, to compile with MPI
 */
#include <stdio.h>
#include <mpi.h>

main(int argc, char **argv)
{
  int node, i, j;
  float f;

  MPI_Init(&argc,&argv);
  MPI_Comm_rank(MPI_COMM_WORLD, &node);

  printf("Hello World from Node %d.\n", node);
  for (i=0; i<=100000; i++)
    f=i*2.718281828*i+i+i*3.141592654;

  MPI_Finalize();
}
```

The "mpi_hello.c" program is a simple source file, written in C with MPI library calls.

Then, check the command line options for *"mpicc" (GNU C-Compiler with MPI extensions)*, then we compile and list the contents of the directory again:

```
$ mpicc -help
$ mpicc -o mpihello mpihello.c
$ ls -l
```

A new file "hello" has been created. Note that this program has "execute" rights.

Let's test this program on the "login" node first:

```
$ ./mpihello
Hello World from Node 0.
```

It seems to work, now run it on the UAntwerpen-HPC.

```
$ qsub mpihello.pbs
```

13.3.3 Compiling a parallel program in Intel Parallel Studio Cluster Edition

We will now compile the same program, but using the Intel Parallel Studio Cluster Edition compilers. We stay in the examples directory for this chapter:

```
$ cd ~/examples/Compiling-and-testing-your-software-on-the-HPC
```

We will compile this C/MPI -file into an executable with the Intel Parallel Studio Cluster Edition. First, clear the modules (purge) and then load the latest "intel" module:

```
$ module purge
$ module load intel
```

Then, compile and list the contents of the directory again. The Intel equivalent of mpicc is mpiicc.

```
$ mpiicc -o mpihello mpihello.c
$ ls -l
```

Note that the old "mpihello" file has been overwritten. Let's test this program on the "login" node first:

```
$ ./mpihello
Hello World from Node 0.
```

It seems to work, now run it on the UAntwerpen-HPC.

```
$ qsub mpihello.pbs
```

Note: The Antwerp University Association (AUHA) only has a license for the Intel Parallel Studio Cluster Edition for a fixed number of users. As such, it might happen that you have to wait a few minutes before a floating license becomes available for your use.

Note: The Intel Parallel Studio Cluster Edition contains equivalent compilers for all GNU compilers. Hereafter the overview for C, C++ and Fortran compilers.

	Sequential Program		**Parallel Program (with MPI)**	
	GNU	**Intel**	**GNU**	**Intel**
C	gcc	icc	mpicc	mpiicc
C++	g++	icpc	mpicxx	mpiicpc
Fortran	gfortran	ifort	mpif90	mpiifort

Chapter 14

Program examples

Go to our examples:

```
$ cd ~/examples/Program-examples
```

Here, we just have put together a number of examples for your convenience. We did an effort to put comments inside the source files, so the source code files are (should be) self-explanatory.

1. 01_Python
2. 02_C_C++
3. 03_Matlab
4. 04_MPI_C
5. 05a_OMP_C
6. 05b_OMP_FORTRAN
7. 06_NWChem
8. 07_Wien2k
9. 08_Gaussian
10. 09_Fortran
11. 10_PQS

The above 2 OMP directories contain the following examples:

C Files	Fortran Files	Description
omp_hello.c	omp_hello.f	Hello world
omp_workshare1.c	omp_workshare1.f	Loop work-sharing
omp_workshare2.c	omp_workshare2.f	Sections work-sharing
omp_reduction.c	omp_reduction.f	Combined parallel loop reduction
omp_orphan.c	omp_orphan.f	Orphaned parallel loop reduction
omp_mm.c	omp_mm.f	Matrix multiply
omp_getEnvInfo.c	omp_getEnvInfo.f	Get and print environment information
omp_bug1.c omp_bug1fix.c omp_bug2.c omp_bug3.c omp_bug4.c omp_bug4fix omp_bug5.c omp_bug5fix.c omp_bug6.c	omp_bug1.f omp_bug1fix.f omp_bug2.f omp_bug3.f omp_bug4.f omp_bug4fix omp_bug5.f omp_bug5fix.f omp_bug6.f	Programs with bugs and their solution

Compile by any of the following commands:

C:	icc -openmp omp_hello.c -o hello pgcc -mp omp_hello.c -o hello gcc -fopenmp omp_hello.c -o hello
Fortran:	ifort -openmp omp_hello.f -o hello pgf90 -mp omp_hello.f -o hello gfortran -fopenmp omp_hello.f -o hello

Be invited to explore the examples.

Chapter 15

Job script examples

15.1 Single-core job

Here's an example of a single-core job script:

— single_core.sh —

```
1  #!/bin/bash
2  #PBS -N count_example          ## job name
3  #PBS -l nodes=1:ppn=1          ## single-node job, single core
4  #PBS -l walltime=2:00:00       ## max. 2h of wall time
5  module load Python/3.6.4-intel-2018a
6  # copy input data from location where job was submitted from
7  cp $PBS_O_WORKDIR/input.txt $TMPDIR
8  # go to temporary working directory (on local disk) & run
9  cd $TMPDIR
10 python -c "print(len(open('input.txt').read()))" > output.txt
11 # copy back output data, ensure unique filename using $PBS_JOBID
12 cp output.txt $VSC_DATA/output_${PBS_JOBID}.txt
```

1. Using `#PBS` header lines, we specify the resource requirements for the job, see Appendix B for a list of these options

2. A module for `Python 3.6` is loaded, see also section 4.1

3. We stage the data in: the file `input.txt` is copied into the "working" directory, see chapter 6

4. The main part of the script runs a small Python program that counts the number of characters in the provided input file `input.txt`

5. We stage the results out: the output file `output.txt` is copied from the "working directory" (`$TMPDIR`|) to a unique directory in `$VSC_DATA`. For a list of possible storage locations, see subsection 6.2.1.

15.2 Multi-core job

Here's an example of a multi-core job script that uses `mympirun`:

— multi_core.sh —

```
#!/bin/bash
#PBS -N mpi_hello             ## job name
#PBS -l nodes=2:ppn=all       ## 2 nodes, all cores per node
#PBS -l walltime=2:00:00      ## max. 2h of wall time
module load intel/2017b
module load vsc-mympirun      ## We don't use a version here, this is on purpose
# go to working directory, compile and run MPI hello world
cd $PBS_O_WORKDIR
mpicc mpi_hello.c -o mpi_hello
mympirun ./mpi_hello
```

An example MPI hello world program can be downloaded from https://github.com/hpcugent/vsc-mympirun/blob/master/testscripts/mpi_helloworld.c.

15.3 Running a command with a maximum time limit

If you want to run a job, but you are not sure it will finish before the job runs out of walltime and you want to copy data back before, you have to stop the main command before the walltime runs out and copy the data back.

This can be done with the `timeout` command. This command sets a limit of time a program can run for, and when this limit is exceeded, it kills the program. Here's an example job script using `timeout`:

— timeout.sh —

```
#!/bin/bash
#PBS -N timeout_example
#PBS -l nodes=1:ppn=1         ## single-node job, single core
#PBS -l walltime=2:00:00      ## max. 2h of wall time

# go to temporary working directory (on local disk)
cd $TMPDIR
# This command will take too long (1400 minutes is longer than our walltime)
# $PBS_O_WORKDIR/example_program.sh 1400 output.txt

# So we put it after a timeout command
# We have a total of 120 minutes (2 x 60) and we instruct the script to run for
# 100 minutes, but timeout after 90 minute,
# so we have 30 minutes left to copy files back. This should
#  be more than enough.
timeout -s SIGKILL 90m $PBS_O_WORKDIR/example_program.sh 100 output.txt
# copy back output data, ensure unique filename using $PBS_JOBID
cp output.txt $VSC_DATA/output_${PBS_JOBID}.txt
```

The example program used in this script is a dummy script that simply sleeps a specified amount of minutes:

— example_program.sh —

```
#!/bin/bash
# This is an example program
# It takes two arguments: a number of times to loop and a file to write to
# In total, it will run for (the number of times to loop) minutes

if [ $# -ne 2 ]; then
    echo "Usage: ./example_program amount filename" && exit 1
fi

for ((i = 0; i < $1; i++ )); do
    echo "${i} => $(date)" >> $2
    sleep 60
done
```

Chapter 16

Best Practices

16.1 General Best Practices

1. Before starting you should always check:
   - Are there any errors in the script?
   - Are the required modules loaded?
   - Is the correct executable used?
2. Check your computer requirements upfront, and request the correct resources in your batch job script.
   - Number of requested cores
   - Amount of requested memory
   - Requested network type
3. Check your jobs at runtime. You could login to the node and check the proper execution of your jobs with, e.g., `top` or `vmstat`. Alternatively you could run an interactive job (`qsub -I`).
4. Try to benchmark the software for scaling issues when using MPI or for I/O issues.
5. Use the scratch file system (`$VSC_SCRATCH_NODE`, which is mapped to the local /tmp) whenever possible. Local disk I/O is always much faster as it does not have to use the network.
6. When your job starts, it will log on to the compute node(s) and start executing the commands in the job script. It will start in your home directory `$VSC_HOME`, so going to the current directory with `cd $PBS_O_WORKDIR` is the first thing which needs to be done. You will have your default environment, so don't forget to load the software with `module load`.
7. In case your job not running, use "checkjob". It will show why your job is not yet running. Sometimes commands might timeout with an overloaded scheduler.
8. Submit your job and wait (be patient) ...

9. Submit small jobs by grouping them together. See chapter 12 for how this is done.
10. The runtime is limited by the maximum walltime of the queues. For longer walltimes, use checkpointing.
11. Requesting many processors could imply long queue times. It's advised to only request the resources you'll be able to use.
12. For all multi-node jobs, please use a cluster that has an "InfiniBand" interconnect network.
13. And above all, do not hesitate to contact the UAntwerpen-HPC staff at hpc@uantwerpen.be. We're here to help you.

Appendix A

HPC Quick Reference Guide

Remember to substitute the usernames, login nodes, file names, ... for your own.

Login	
Login	`ssh vsc20167@login.hpc.uantwerpen.be`
Where am I?	`hostname`
Copy to UAntwerpen-HPC	`scp foo.txt vsc20167@login.hpc.uantwerpen.be:`
Copy from UAntwerpen-HPC	`scp vsc20167@login.hpc.uantwerpen.be:foo.txt .`
Setup ftp session	`sftp vsc20167@login.hpc.uantwerpen.be`

Modules	
List all available modules	`module avail`
List loaded modules	`module list`
Load module	`module load example`
Unload module	`module unload example`
Unload all modules	`module purge`
Help on use of module	`module help`

Jobs	
Submit job with job script `script.pbs`	`qsub script.pbs`
Status of job with ID 12345	`qstat 12345`
Possible start time of job with ID 12345 (not available everywhere)	`showstart 12345`
Check job with ID 12345 (not available everywhere)	`checkjob 12345`
Show compute node of job with ID 12345	`qstat -n 12345`
Delete job with ID 12345	`qdel 12345`
Status of all your jobs	`qstat`
Detailed status of your jobs + a list nodes they are running on	`qstat -na`
Show all jobs on queue (not available everywhere)	`showq`
Submit Interactive job	`qsub -I`

Disk quota	
Check your disk quota	`mmlsquota`
Check disk quota nice	`show_quota.py`
Disk usage in current directory (`.`)	`du -h .`

Worker Framework	
Load worker module	`module load worker/1.6.8-intel-2018a` Don't forget to specify a version. To list available versions, use `module avail worker/`
Submit parameter sweep	`wsub -batch weather.pbs -data data.csv`
Submit job array	`wsub -t 1-100 -batch test_set.pbs`
Submit job array with prolog and epilog	`wsub -prolog pre.sh -batch test_set.pbs -epilog post.sh -t 1-100`

Appendix B

TORQUE options

B.1 TORQUE Submission Flags: common and useful directives

Below is a list of the most common and useful directives.

Option	**System type**	**Description**
-k	All	Send "stdout" and/or "stderr" to your home directory when the job runs **#PBS -k o** or **#PBS -k e** or **#PBS -koe**
-l	All	Precedes a resource request, e.g., processors, wallclock
-M	All	Send an e-mail messages to an alternative e-mail address **#PBS -M me@mymail.be**
-m	All	Send an e-mail address when a job **b**egins execution and/or **e**nds or **a**borts **#PBS -m b** or **#PBS -m be** or **#PBS -m ba**
mem	Shared Memory	Specifies the amount of memory you need for a job. **#PBS -l mem=80gb**
mpiprocs	Clusters	Number of processes per node on a cluster. This should equal number of processors on a node in most cases. **#PBS -l mpiprocs=4**
-N	All	Give your job a unique name **#PBS -N galaxies1234**
-ncpus	Shared Memory	The number of processors to use for a shared memory job. **#PBS ncpus=4**
-r	All	Control whether or not jobs should automatically re-run from the start if the system crashes or is rebooted. Users with check points might not wish this to happen. **#PBS -r n** **#PBS -r y**
select	Clusters	Number of compute nodes to use. Usually combined with the mpiprocs directive **#PBS -l select=2**

-V	All	Make sure that the environment in which the job **runs** is the same as the environment in which it was **submitted.** #**PBS -V**
Walltime	All	The maximum time a job can run before being stopped. If not used a default of a few minutes is used. Use this flag to prevent jobs that go bad running for hundreds of hours. Format is HH:MM:SS #**PBS -l walltime=12:00:00**

B.2 Environment Variables in Batch Job Scripts

TORQUE-related environment variables in batch job scripts.

```
1  # Using PBS - Environment Variables:
2  # When a batch job starts execution, a number of environment variables are
3  # predefined, which include:
4  #
5  #      Variables defined on the execution host.
6  #      Variables exported from the submission host with
7  #                -v (selected variables) and -V (all variables).
8  #      Variables defined by PBS.
9  #
10 # The following reflect the environment where the user ran qsub:
11 # PBS_O_HOST    The host where you ran the qsub command.
12 # PBS_O_LOGNAME Your user ID where you ran qsub.
13 # PBS_O_HOME    Your home directory where you ran qsub.
14 # PBS_O_WORKDIR The working directory where you ran qsub.
15 #
16 # These reflect the environment where the job is executing:
17 # PBS_ENVIRONMENT       Set to PBS_BATCH to indicate the job is a batch job,
18 #         or to PBS_INTERACTIVE to indicate the job is a PBS interactive job.
19 # PBS_O_QUEUE   The original queue you submitted to.
20 # PBS_QUEUE     The queue the job is executing from.
21 # PBS_JOBID     The job's PBS identifier.
22 # PBS_JOBNAME   The job's name.
```

IMPORTANT!! All PBS directives MUST come before the first line of executable code in your script, otherwise they will be ignored.

When a batch job is started, a number of environment variables are created that can be used in the batch job script. A few of the most commonly used variables are described here.

Variable	**Description**
PBS_ENVIRONMENT	set to PBS_BATCH to indicate that the job is a batch job; otherwise, set to PBS_INTERACTIVE to indicate that the job is a PBS interactive job.
PBS_JOBID	the job identifier assigned to the job by the batch system. This is the same number you see when you do *qstat*.
PBS_JOBNAME	the job name supplied by the user

PBS_NODEFILE	the name of the file that contains the list of the nodes assigned to the job . Useful for Parallel jobs if you want to refer the node, count the node etc.
PBS_QUEUE	the name of the queue from which the job is executed
PBS_O_HOME	value of the HOME variable in the environment in which *qsub* was executed
PBS_O_LANG	value of the LANG variable in the environment in which *qsub* was executed
PBS_O_LOGNAME	value of the LOGNAME variable in the environment in which *qsub* was executed
PBS_O_PATH	value of the PATH variable in the environment in which *qsub* was executed
PBS_O_MAIL	value of the MAIL variable in the environment in which *qsub* was executed
PBS_O_SHELL	value of the SHELL variable in the environment in which *qsub* was executed
PBS_O_TZ	value of the TZ variable in the environment in which *qsub* was executed
PBS_O_HOST	the name of the host upon which the *qsub* command is running
PBS_O_QUEUE	the name of the original queue to which the job was submitted
PBS_O_WORKDIR	the absolute path of the current working directory of the *qsub* command. This is the most useful. Use it in every job script. The first thing you do is, cd $PBS_O_WORKDIR after defining the resource list. This is because, pbs throw you to your $HOME directory.
PBS_VERSION	Version Number of TORQUE, e.g., TORQUE-2.5.1
PBS_MOMPORT	active port for mom daemon
PBS_TASKNUM	number of tasks requested
PBS_JOBCOOKIE	job cookie
PBS_SERVER	Server Running TORQUE

Appendix C

Useful Linux Commands

C.1 Basic Linux Usage

All the UAntwerpen-HPC clusters run some variant of the "RedHat Enterprise Linux" operating system. This means that, when you connect to one of them, you get a command line interface, which looks something like this:

```
vsc20167@ln01[203] $
```

When you see this, we also say you are inside a "shell". The shell will accept your commands, and execute them.

ls	Shows you a list of files in the current directory
cd	Change current working directory
rm	Remove file or directory
joe	Text editor
echo	Prints its parameters to the screen

Most commands will accept or even need parameters, which are placed after the command, separated by spaces. A simple example with the "echo" command:

```
$ echo This is a test
This is a test
```

Important here is the "$" sign in front of the first line. This should not be typed, but is a convention meaning "the rest of this line should be typed at your shell prompt". The lines not starting with the "$" sign are usually the feedback or output from the command.

More commands will be used in the rest of this text, and will be explained then if necessary. If not, you can usually get more information about a command, say the item or command "ls", by trying either of the following:

```
$ ls -help
$ man ls
$ info ls
```

(You can exit the last two "manuals" by using the "q" key.) For more exhaustive tutorials about Linux usage, please refer to the following sites: http://www.linux.org/lessons/

http://linux.about.com/od/nwb_guide/a/gdenwb06.htm

C.2 How to get started with shell scripts

In a shell script, you will put the commands you would normally type at your shell prompt in the same order. This will enable you to execute all those commands at any time by only issuing one command: starting the script.

Scripts are basically non-compiled pieces of code: they are just text files. Since they don't contain machine code, they are executed by what is called a "parser" or an "interpreter". This is another program that understands the command in the script, and converts them to machine code. There are many kinds of scripting languages, including Perl and Python.

Another very common scripting language is shell scripting. In a shell script, you will put the commands you would normally type at your shell prompt in the same order. This will enable you to execute all those commands at any time by only issuing one command: starting the script.

Typically in the following examples they'll have on each line the next command to be executed although it is possible to put multiple commands on one line. A very simple example of a script may be:

```
1 echo "Hello! This is my hostname:"
2 hostname
```

You can type both lines at your shell prompt, and the result will be the following:

```
$ echo "Hello!  This is my hostname:"
Hello! This is my hostname:
$ hostname
ln2.leibniz.uantwerpen.vsc
```

Suppose we want to call this script "foo". You open a new file for editing, and name it "foo", and edit it with your favourite editor

```
$ vi foo
```

or use the following commands:

```
$ echo "echo Hello!  This is my hostname:" > foo
$ echo hostname >> foo
```

The easiest ways to run a script is by starting the interpreter and pass the script as parameter. In case of our script, the interpreter may either be "sh" or "bash" (which are the same on the cluster). So start the script:

```
$ bash foo
Hello! This is my hostname:
ln2.leibniz.uantwerpen.vsc
```

Congratulations, you just created and started your first shell script!

A more advanced way of executing your shell scripts is by making them executable by their own, so without invoking the interpreter manually. The system can not automatically detect which

interpreter you want, so you need to tell this in some way. The easiest way is by using the so called "shebang" notation, explicitly created for this function: you put the following line on top of your shell script "#!/path/to/your/interpreter".

You can find this path with the "which" command. In our case, since we use bash as an interpreter, we get the following path:

```
$ which bash
/bin/bash
```

We edit our script and change it with this information:

```
1 #!/bin/bash
2 echo "Hello! This is my hostname:"
3 hostname
```

Note that the "shebang" must be the first line of your script! Now the operating system knows which program should be started to run the script.

Finally, we tell the operating system that this script is now executable. For this we change its file attributes:

```
$ chmod +x foo
```

Now you can start your script by simply executing it:

```
$ ./foo
Hello! This is my hostname:
ln2.leibniz.uantwerpen.vsc
```

The same technique can be used for all other scripting languages, like Perl and Python.

Most scripting languages understand that lines beginning with "#" are comments, and should be ignored. If the language you want to use does not ignore these lines, you may get strange results ...

C.3 Linux Quick reference Guide

C.3.1 Archive Commands

tar	An archiving program designed to store and extract files from an archive known as a tar file.
tar -cvf foo.tar foo/	compress the contents of foo folder to foo.tar
tar -xvf foo.tar	extract foo.tar
tar -xvzf foo.tar.gz	extract gzipped foo.tar.gz

C.3.2 Basic Commands

ls	Shows you a list of files in the current directory
cd	Change the current directory
rm	Remove file or directory
mv	Move file or directory
echo	Display a line or text
pwd	Print working directory
mkdir	Create directories
rmdir	Remove directories

C.3.3 Editor

emacs	
nano	Nano's ANOther editor, an enhanced free Pico clone
vi	A programmers text editor

C.3.4 File Commands

cat	Read one or more files and print them to standard output
cmp	Compare two files byte by byte
cp	Copy files from a source to the same or different target(s)
du	Estimate disk usage of each file and recursively for directories
find	Search for files in directory hierarchy
grep	Print lines matching a pattern
ls	List directory contents
mv	Move file to different targets
rm	Remove files
sort	Sort lines of text files
wc	Print the number of new lines, words, and bytes in files

C.3.5 Help Commands

man	Displays the manual page of a command with its name, synopsis, description, author, copyright etc.

C.3.6 Network Commands

hostname	show or set the system's host name
ifconfig	Display the current configuration of the network interface. It is also useful to get the information about IP address, subnet mask, set remote IP address, netmask etc.
ping	send ICMP ECHO_REQUEST to network hosts, you will get back ICMP packet if the host responds. This command is useful when you are in a doubt whether your computer is connected or not.

C.3.7 Other Commands

logname	Print user's login name
quota	Display disk usage and limits
which	Returns the pathnames of the files that would be executed in the current environment
whoami	Displays the login name of the current effective user

C.3.8 Process Commands

&	In order to execute a command in the background, place an ampersand (&) on the command line at the end of the command. A user job number (placed in brackets) and a system process number are displayed. A system process number is the number by which the system identifies the job whereas a user job number is the number by which the user identifies the job
at	executes commands at a specified time
bg	Places a suspended job in the background
crontab	crontab is a file which contains the schedule of entries to run at specified times
fg	A process running in the background will be processed in the foreground
jobs	Lists the jobs being run in the background
kill	Cancels a job running in the background, it takes argument either the user job number or the system process number
ps	Reports a snapshot of the current processes
top	Display Linux tasks

C.3.9 User Account Commands

chmod	Modify properties for users
chown	Change file owner and group